

GVP-MAAA: MULTI-AGENT ACADEMIC ASSISTANT

A project report submitted in partial fulfilment of the requirement for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING(AI&ML)

Submitted by

K.V.S. MOHITH (5221412021)

E.PURNIMA (5221412057)

S. VEERA BADRAIAH (5221412052)

T. SANDEEP (5221412049)

Under the Esteemed Guidance of

CH. BHANU KUMAR B. Tech, M. Tech

Assistant Professor



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING(AI&ML)

ENGINEERING AND TECHNOLOGY PROGRAM

GAYATRI VIDYA PARISHAD COLLEGE FOR DEGREE AND PG COURSES (A)

Rushikonda, Visakhapatnam - 45

(Approved by AICTE| Accredited by NBA| Accredited by NAAC| Affiliated to Andhra
University) An ISO 9001:2015 Certified Institution

2022-2026

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING(AI&ML)

**GAYATRI VIDYA PARISHAD COLLEGE FOR DEGREE AND PG
COURSES(A)**

Rushikonda, Visakhapatnam - 530045



CERTIFICATE

This is to certify that the project report entitled “GVP-MAAA: Multi-Agent Academic Assistant being submitted by K.V.S. MOHITH (5221412021), E. PURNIMA (5221412057), S.VEERA BADRAIAH (5221412052), T. SANDEEP (5221412049) in the partial fulfillment for the award of the Degree of Bachelor of Technology in Computer Science and Engineering (AI&ML) to the Gayatri Vidya Parishad College for Degree and PG Courses, Visakhapatnam is a record of bonafied work carried out under my guidance and supervision.

Project Guide

Ch. Bhanu Kumar
Assistant Professor
B. Tech, M. Tech

Head of the Department

Dr. N.V. Ramana Murty
Professor
M. Tech, Ph. D

External Examiner

VISION AND MISSION OF THE INSTITUTE

VISION

Creating Human Excellence for a Better Society

MISSION

Unfold into a World class organization with strong academic and research base, producing responsible citizens to cater to the changing needs of the society.

VISION AND MISSION OF THE DEPARTMENT

VISION

“To enhance employability, foster innovative research, and nurture entrepreneurial thinking by implementing effective teaching and learning practices, aimed at developing cutting-edge technology solutions that benefit both industry and society.”

MISSION

“Delivering high-quality, value-driven technical education to shape technology professionals who think creatively and lead with inspiration.

Utilizing rational thinking to design and develop cutting-edge products in collaboration with industry stakeholders, aligning with global standards and demands.

Enhancing core skills in the fields of Artificial Intelligence and Machine Learning”.

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

- **PEO1: Lead in diverse professional roles at reputed industries and address challenges by applying principles of computer science, mathematics, artificial intelligence, machine learning, and scientific investigation to deliver industry-ready solutions using advanced technologies.**
- **PEO2: Exhibit effective communication, teamwork, leadership skills, and ethical behaviour while achieving professional growth through lifelong learning and continuing education.**
- **PEO3: Pursue advanced education in leading graduate programs across engineering and interdisciplinary fields to excel as researchers, experts, educators, and entrepreneurs, while recognizing the importance of and actively engaging in lifelong learning and professional development.**

PROGRAM SPECIFIC OUTCOMES (PSOs)

- **PSO 1: Design and develop optimized solutions for computationally intensive applications.**
- **PSO 2: Apply tools and techniques of Artificial Intelligence and Machine Learning to address multidisciplinary challenges**

DECLARATION

We hereby declare that the project entitled “GVP-MAAA: Multi-Agent Academic Assistant” submitted in partial fulfilment of the requirements for the award of Bachelor of Technology in Computer Science and Engineering (AI&ML), to Engineering and Technology Program, Gayatri Vidya Parishad College for Degree & PG Courses (A). We assure that this project is not submitted in any other University or College.

Name & Signature of the Students

K.V.S.MOHITH (5221412021)

E.PURNIMA (5221412057)

S.VEERA BADRAIAH (5221412052)

T. SANDEEP (5221412049)

ACKNOWLEDGEMENTS

With great pleasure we want to take this opportunity to express our heartfelt gratitude to all the people who helped in making main project work a grand success.

First of all, we express our deep sense of people who helped in making main project work a grand success. First gratitude to **Sri. Ch. Bhanu Kumar**, Assistant Professor, for his constant guidance throughout our main project work.

We are grateful to **Dr. D.S.S.N. Raju**, Assistant Professor, Department of Computer Science and Engineering (AI&ML) as project coordinator, for his valuable suggestions and guidance given by him during the execution of this main project.

We would like to thank **Dr. N.V. Ramana Murty**, Professor, Head of the Department of Computer Science and Engineering (AI&ML), for being moral support throughout the period of our study.

We are highly indebted to **Dr. P.V. Vinay**, Director – E&T and **Prof. K.S. Bose**, Principal for giving us the permission to carry out this main project in the college.

We would like to thank all the Teaching and Non-Teaching staff of CSE and CSE (AI&ML) Department for sharing their knowledge with us.

K.V.S. MOHITH	(5221412021)
E.PURNIMA	(5221412057)
S. VEERA BADRAIAH	(5221412052)
T. SANDEEP	(5221412049)

ABSTRACT

Educational institutions handle various types of academic data such as attendance, marks, schedules, and student records. In many cases, traditional systems are manual or lack advanced automation, which can lead to delays and reduced efficiency.

GVP-MAAA (Multi-Agent Academic Assistant) is a web-based system developed to simplify academic management using a multi-agent approach and role-based dashboards. The platform supports attendance tracking, performance monitoring, and early identification of students who may require academic support. It provides centralized data access, timely notifications, and basic analytical insights to assist administrators and faculty members in decision-making. The system aims to improve transparency, reduce manual workload, and enhance overall academic management within the institution.

TABLE OF CONTENTS

CHAPTER	Page No
1.Introduction	1
1.1 Introduction to GVP-MAAA	2
2. Literature Survey	3
2.1 Research Gap	4
2.2 Motivation For Proposal System	5
3.Requirements and Analysis	6
3.1 Existing System	7
3.2 Proposed System	7
3.3 Functional Requirements	9
3.4 Non-Functional Requirements	10
3.5 Analysis	19
4. System Design	23
4.1 Design Goals	24
4.2 System Architecture	25
4.3 System Design	25
4.4 UML Diagrams	27
4.5 Data Flow Diagrams	34
4.6 Flow Chart	36
5.Implementation	37
5.1 Coding Approach	38
5.2 Information Handling	38
5.3 Programming Style	39
5.4 Verification and Validation	39
5.5 Implementation of Project	40
5.6 System Workflow and Process Flow	42
6. Testing	44
6.1 Testing Activities	43
6.2 Testing Types	44

7.Results	48
7.1 Landing Page	49
7.2 signup Page	49
7.3 Loginin Page	50
7.4 Student dashboard Page	52
7.5 Faculty dashboard Page	57
7.6 Admin dashboard Page	64
 8. Sample Code	
8.1 Code for Auth.py	70
8.2 Code for Database.py	72
8.3 Code for Main.py	72
8.4 Code for Models.py	79
8.5 Code for Schemas.py	101
8.6 Code for Security. Py	123
 9. Conclusion	149
10. References	151

LIST OF TABLES

Title	Page No
6.3 Test cases	47

LIST OF FIGURES

Title	Page No
3.4.1.1 Intel Processor Chip	11
3.4.1.2 Postgre Sql	12
3.4.1.3 User Devices	12
3.4.2.1 Python	15
3.4.2.2 FastAPI	15
3.4.2.3 React JS	16
3.4.2.4 Postgre Sql	17
3.4.2.5 Lang Chain	18
3.4.2.6 Open AI	18
3.4.2.7 Ollama	19
3.4.2.8 Rest API	19
4.3.1 System's Block Diagram	26
4.4.1.1 Use Case Diagram	29
4.4.2.1 Class Diagram	29
4.4.3.1 Activity Diagram	30
4.4.4.1 Sequence Diagram - Student	31
4.4.4.2 Sequence Diagram - Faculty	32
4.4.4.3 Sequence Diagram - Admin	33
4.5.1 DFD Level 0	34
4.5.2 DFD Level 1	35
4.6.1 Flow chart of the system	36
6.2.1 Black Box Testing	46
7.1.1 Landing Page	49
7.2.1 Student Signup Page	49
7.2.2 Teacher Signup Page	50
7.3.1 Student Login Page	50
7.3.2 Teacher Login Page	51
7.3.3 Admin Login Page	51
7.4.1 Student over View Page	52
7.4.2 Student Attendance Page	52
7.4.3 Student Assignment Page	53
7.4.4 Student Timetable Page	53

7.4.5 Student Marks Page	54
7.4.6 Student Placements Page	54
7.4.7 Student Event page	55
7.4.8 Student Insight Page	55
7.4.9 Student Resource Page	56
7.4.10 Student Alerts Page	56
7.4.11 Student Logout Page	57
7.5.1 Faculty Overview Page	57
7.5.2 Faculty Timetable Page	58
7.5.3 Faculty Attendance Page	58
7.5.4 Faculty Assignment Page	59
7.5.5 Faculty Marks Page	59
7.5.6 Faculty Resource Page	60
7.5.7 Faculty Events Page	60
7.5.8 Faculty Insights Page	61
7.5.9 Faculty Placements Page	61
7.5.10 Faculty Coordinator Page	62
7.5.11 Faculty Upload Resource Page	62
7.5.12 Faculty Send Alerts Page	63
7.5.13 Faculty Alerts Page	63
7.5.14 Faculty Logout Page	64
7.6.1 Admin Overview Page	64
7.6.2 Admin Student Page	65
7.6.3 Admin Teacher Page	65
7.6.4 Admin Timetable Page	66
7.6.5 Admin Alerts Page	66
7.6.6 Admin Insights Page	67
7.6.7 Admin Placements Page	67
7.6.8 Admin Settings Page	68
7.6.9 Admin Logout Page	68

1.INTRODUCTION

1.INTRODUCTION

1.1Introduction to GVP-MAAA

Academic institutions today manage enormous volumes of information including attendance records, marks, timetables, announcements, and academic resources. In most institutions, these processes are handled manually or through disconnected systems, resulting in delays, data duplication, limited realtime access, and increased workloads for both students and faculty members. Students frequently struggle to continuously track their academic progress — attendance percentages, subject-wise performance, and examination schedules — without having to approach faculty or administration individually. Faculty members spend considerable time on repetitive administrative tasks such as uploading attendance and entering marks, instead of focusing on academic guidance. Administrators lack centralized visibility into institutional performance trends, making proactive decision-making difficult. To overcome these challenges, the GVP Multi-Agent Academic Assistant (GVP-MAAA) is proposed — a webbased intelligent system that automates and integrates academic management across all stakeholder categories. The system is built on a multi-agent framework where five specialized autonomous agents coordinate to manage distinct academic functions, combined with role-based dashboards and an AI-powered chatbot for natural-language interaction. The five agents — Notification, Analytics, Attendance, Placement, and Resource — each expose role-context-aware capabilities for Students, Faculty, and Administrators. The chatbot dynamically detects which agent to invoke based on keyword matching against the user's natural-language query, ensuring contextually appropriate responses

AI-powered tools and applications are widely used in modern farming practices. For example, image recognition models can identify crop diseases by analysing leaf images, while predictive algorithms can recommend the best crops to grow based on soil type, weather conditions, and historical data. AI can also be integrated with sensors, drones, and satellite imagery to monitor large agricultural fields in real time. These smart systems help farmers reduce the use of fertilizers, pesticides, and water, leading to sustainable farming practices. Overall, Artificial Intelligence in agriculture improves efficiency, reduces labour costs, increases crop yield, and supports farmers in making data-driven decisions for better agricultural management.

2.LITERATURE SURVEY

2.LITERATURE SURVEY

The rapid advancement of Artificial Intelligence and data-driven technologies has significantly influenced the education sector, particularly in the areas of student performance analysis, personalized learning, and intelligent academic support systems. Several research studies have explored how machine learning, data mining, and intelligent systems can be used to improve student outcomes and enhance decision-making in academic environments.

Early work by Cortez and Silva [1] demonstrated the use of data mining techniques to predict student performance based on academic and socio-economic factors. Their study highlighted the effectiveness of classification models in identifying students at risk of poor performance. Similarly, Romero and Ventura [2] provided a comprehensive review of educational data mining, emphasizing its potential in extracting meaningful patterns from academic data to support institutional decision-making. These studies form the foundation for modern academic analytics systems.

Further research by Pandey and Pal [3] and Kaur et al. [4] explored the use of classification algorithms such as decision trees and Naïve Bayes to categorize students into performance groups. While these approaches are effective in prediction, they primarily focus on static analysis and lack real-time adaptability. Xing et al. [5] extended this concept by introducing temporal prediction models in online learning environments, enabling early identification of students likely to drop out. However, these systems are typically limited to specific platforms such as MOOCs and do not provide a comprehensive academic support framework.

The concept of learning analytics has gained significant attention in recent years. Siemens [6] described learning analytics as a discipline focused on measuring, collecting, and analyzing learner data to improve educational outcomes. Ferguson [8] and Viberg et al. [7] further emphasized the role of analytics in enhancing student engagement, identifying learning gaps, and supporting personalized education. Despite their effectiveness, most learning analytics systems are designed primarily for reporting and visualization, rather than proactive intervention.

Artificial Intelligence has also been widely applied in education to create intelligent tutoring systems and automated support tools. Holmes et al. [9] and Zawacki-Richter et al. [10] discussed the growing role of AI in transforming traditional education into a more adaptive and personalized experience. Woolf [11] introduced intelligent tutoring systems that provide customized learning paths based on student behavior.

However, many of these systems focus on instruction rather than integrating academic performance monitoring with institutional workflows.

Recommender systems have been proposed as a solution to personalize learning experiences. Manouselis et al. [12] and Drachsler et al. [13] explored how recommendation techniques can be used to suggest learning materials and courses based on user preferences and performance. While effective in personalization, these systems often operate independently and do not integrate with broader academic management systems.

In addition, multi-agent systems have been studied as a way to build distributed and intelligent applications. Wooldridge [14] defined multi-agent systems as a collection of autonomous agents that interact to solve complex problems. Oprea [15] demonstrated the application of multi-agent systems in educational environments, where different agents handle specific tasks such as monitoring, recommendation, and communication. However, most implementations remain conceptual or limited in scope.

2.1 Research Gap

From the above studies, it is evident that existing systems primarily focus on individual aspects such as prediction, analytics, recommendation, or tutoring. There is a lack of integrated systems that combine these capabilities into a unified platform. Additionally, many systems do not provide real-time, role-based insights for different stakeholders such as students, faculty, and administrators.

2.2 Motivation for Proposed System

To address these limitations, the proposed system (GVP-MAAA) introduces a multi-agent academic intelligence framework that integrates performance analysis, intelligent recommendations, and interactive assistance into a single platform. Unlike existing approaches, the system focuses not only on analyzing data but also on providing actionable insights and proactive academic support through specialized agents

3.REQUIREMENT ANALYSIS

3.REQUIREMENTS AND ANALYSIS

3.1 Existing system

In the current academic environment, most institutions rely on traditional systems such as basic student portals and manual evaluation methods to manage academic performance. These systems primarily focus on storing and displaying data such as marks, attendance, and course details. While they provide essential functionality, they lack the ability to analyze student data in a meaningful way.

summaries and attendance tracking. However, these systems are often limited to static reports and do not offer predictive insights or personalized guidance. Students are required to interpret their own performance without receiving actionable suggestions for improvement.

Additionally, existing systems do Some modern platforms have introduced basic analytics features, such as grade not effectively support faculty and administrators in identifying at-risk students at an early stage. The absence of intelligent monitoring makes it difficult to take timely intervention measures. Most systems also lack integration between different modules, resulting in fragmented information and inefficient workflows.

Another major limitation is the absence of interactive support. Students often need guidance regarding their academic progress, but current systems do not provide conversational or AI-based assistance. As a result, students depend heavily on faculty or external resources for clarification and decision-making.

3.2 Proposed system

The proposed system, GVP-MAAA (Multi-Agent Academic Assistant), is designed to overcome the limitations of existing academic systems by introducing intelligence, automation, and integration.

The system provides a unified platform where students, faculty, and administrators can access academic data along with meaningful insights. It utilizes a multi-agent architecture where different agents are responsible for specific tasks such as performance analysis, notification handling, recommendation generation, and user interaction.

The system continuously analyzes academic data such as marks, attendance, and engagement levels to identify patterns and trends. Based on this analysis, it generates actionable insights that help students improve their performance and assist faculty in monitoring student progress more effectively.

Unlike traditional systems, the proposed solution focuses on proactive support rather than passive data display. It includes an AI-based assistant that allows users to interact with the

system using natural language queries, making the platform more accessible and user-friendly.

Overall, the system aims to transform academic management from a data storage approach into an intelligent decision-support system.

3.2.1 Project Scope

The scope of the project is to develop a web-based academic intelligence system that supports multiple user roles, including students, faculty, and administrators.

The system covers key functionalities such as:

- Academic performance tracking
- Attendance monitoring
- Intelligent analysis and insights
- AI-based query handling
- Role-based dashboards

The system is designed to operate within an institutional environment and can be extended in the future to include advanced features such as machine learning-based prediction models and integration with external educational platforms.

3.2.2 Project Objectives

- To develop an intelligent system for analyzing student academic performance
- To provide real-time insights based on academic data
- To assist students in identifying strengths and areas for improvement
- To support faculty in monitoring and guiding student progress
- To implement a multi-agent architecture for efficient task handling
- To integrate an AI-based assistant for interactive user support
- To design a user-friendly interface for seamless interaction

3.3 Functional Requirements

The system must provide the following functionalities:

- **User Authentication and Role Management**
The system should allow secure login for students, faculty, and administrators with role-based access to functionalities.
- **Academic Data Management**
The system should store and manage academic data such as marks, attendance, and subject details through a centralized data module.
- **Performance Analysis**
The system should analyze academic data to identify trends, evaluate performance, and highlight students who require attention.
- **Dashboard Visualization**
Users should be able to view their academic data and insights through interactive and easy-to-understand dashboards.
- **AI-Based Assistant**
The system should allow users to interact using natural language queries and receive relevant academic information and guidance.
- **Notification System**
The system should provide alerts and updates related to academic performance, attendance, and important activities.
- **Recommendation Support (Added – important)**
The system should provide basic recommendations to students based on their performance, helping them improve in weak areas.
- **Multi-Agent Processing Support (Added – your USP)**
The system should utilize multiple agents such as data management, analysis, and notification components to handle tasks efficiently and ensure smooth system operation.

3.3.1 Actors

The system includes both human users and intelligent agents:

- **Human Actors**

- Student – Views performance, receives recommendations, interacts with AI assistant
- Faculty – Updates data, monitors student progress, receives alerts
- Administrator – Manages system operations and users
- System Actors (Agents)
- Data Management Agent
- Performance Analysis Agent
- Recommendation Agent
- Notification Agent
- AI Assistant Agent

3.3.2 Use cases

The system starts when a user logs into the platform. Based on the role, the user interacts with different system modules.

- When academic data is accessed or updated:

The Data Management Agent handles storage and retrieval

The Performance Analysis Agent processes the data

The Recommendation Agent generates suggestions

The Notification Agent sends alerts if needed

- If the user interacts with the chatbot:

The AI Assistant Agent processes the query

Retrieves relevant data

Provides a response in natural language

The process continues until the user completes the task or logs out.

3.4 Non-Functional Requirements:

- Performance – The system should respond quickly to user requests and data processing
- Accuracy – The system should provide reliable and consistent analysis results
- Scalability – The system should handle multiple users simultaneously

- **Security** – User data should be protected with proper authentication and authorization
- **Usability** – The interface should be simple and easy to use
- **Reliability** – The system should function consistently without failures

3.4.1 Hardware Specifications

- **User Device** (Desktop/Laptop/Mobile) – Used by students, faculty, and administrators to access the system, view dashboards, manage academic and placement data, and interact with the application through the web interface. The system is designed to be responsive and accessible across multiple devices.
- **Computer/Server** – Runs the FastAPI backend, processes user requests, executes multi-agent operations such as data handling, performance analysis, and notification processing. The server is designed to handle multiple concurrent users and can be scaled based on system demand.
- **Database Storage** – Stores student details, academic records, placement data, user authentication information, and system logs using PostgreSQL. The database supports efficient data retrieval, concurrent access, and ensures data consistency across the system.

Computer / Server

- A high-performance computer or server is required to handle backend processing, multi-agent operations, API handling, and real-time data management. It should have the following specifications:
- **Processor:** Intel i5 / Ryzen 5 or higher for efficient request handling and execution of backend services.
- **RAM:** Minimum 8GB RAM to support multiple concurrent users and smooth operation of multi-agent components.
- **Storage:** SSD (256GB or more) for faster data access, storage, and retrieval.
- **GPU (Optional):** Not mandatory, as the system primarily relies on backend processing and API-based intelligence. Can be used if advanced AI/ML models are integrated in future enhancements.



Fig. 3.4.1.1 Intel Processor Chip

Database Storage

- **Database Technology**

The system uses PostgreSQL as a relational database for storing student information, academic records, placement data, and user-related details. It provides reliability, scalability, and efficient handling of structured data.

- **Backend Integration**

FastAPI is used for server-side processing to handle database operations such as data insertion, retrieval, updating records, and managing communication between the frontend and the database.

- **Storage Capacity**

A minimum of 256 GB storage is recommended to efficiently store academic data, system logs, and user information, with the ability to scale based on institutional requirements.

- **Data Management**

The system ensures efficient data handling through optimized queries and transaction management, supporting multiple concurrent users while maintaining data consistency and integrity



Fig. 3.4.1.2 Postgre Sql

User Devices

- **Device Types**

The system can be accessed using desktops, laptops (Windows/macOS), tablets, and smartphones, providing flexibility for students, faculty, and administrators to interact with the application from different platforms.

- **Display and Input**

Devices must support a clear display to view dashboards, academic data, and insights, along with input methods such as keyboard, mouse, or touch for smooth interaction with the system.

- **Internet Connectivity**

A stable internet connection is required to access the web application, communicate with the backend server, and receive real-time updates, notifications, and responses from the system

- **Browser Compatibility**

The system should be accessible through standard web browsers such as Chrome, Edge, Safari, or Firefox, ensuring compatibility and consistent performance across different devices.



Fig. 3.4.1.3 User Devices

3.4.2 Software Specifications

- **React (Vite)** – Frontend framework used to develop a responsive and interactive web-based user interface for students, faculty, and administrators.
- **Python** – Core programming language used for backend development, data handling, and implementing system logic.
- **Fast API** – Backend framework built using Python, used to create RESTful APIs, handle business logic, and manage communication between frontend, database, and system agents.
- **PostgreSQL** – Relational database used to store academic data, user information, placement records, and system logs with high reliability and scalability.

- **Multi-Agent Architecture** – The system is designed using a multi-agent approach where different agents (Data, Analysis, Recommendation, Notification) handle specific tasks independently while coordinating with each other.
- **Lang Chain** – Framework used to manage interactions between language models and system components, enabling structured and intelligent responses.
- **OpenAI API (ChatGPT)** – Used to provide natural language understanding and conversational capabilities for the AI assistant.
- **Ollama** – Used as an optional local language model integration for running AI models locally and reducing dependency on external services.
- **REST API Architecture** – Enables communication between frontend and backend using HTTP requests for efficient data exchange.
- **Web Browser** – The system is accessed through modern web browsers such as Chrome, Edge, Safari, or Firefox.

Python

Python is a high-level, interpreted programming language widely used for backend development, data processing, and building intelligent systems. It is known for its simple and readable syntax, which allows developers to write efficient and maintainable code. Python supports multiple programming paradigms such as object-oriented and functional programming, making it flexible for different types of applications. In this project, Python is used as the core language for implementing backend logic and system operations.

Python provides strong support for web frameworks such as FastAPI, which enables the development of high-performance APIs. It also supports seamless integration with databases and external services, making it suitable for building full-stack applications. In this system, Python handles data processing, API communication, and coordination between different modules including the multi-agent components.

Another advantage of Python is its extensive library support and active developer community, which simplifies development and debugging. It allows rapid development and easy scalability, making it suitable for both small and large-scale systems. Overall, Python plays a central role in ensuring the efficiency and flexibility of the proposed system.



Fig. 3.4.2.1 Python

Fast API

FastAPI is a modern, high-performance web framework built using Python, designed for developing RESTful APIs efficiently. It is known for its speed, simplicity, and automatic documentation generation. FastAPI enables developers to build scalable backend systems with minimal code while maintaining high performance. In this project, FastAPI is used to handle backend operations and manage communication between frontend and database.

FastAPI supports asynchronous programming, which allows the system to handle multiple user requests simultaneously. This is particularly useful in applications where multiple users interact with the system at the same time. It also provides built-in validation and serialization, ensuring that data exchanged between components is accurate and structured.

In the proposed system, FastAPI plays a key role in managing multi-agent interactions, processing user requests, and delivering responses efficiently. Its lightweight nature and performance make it suitable for real-time applications and scalable systems.



Fig. 3.4.2.2 FastAPI

React (Vite)

React is a popular JavaScript library used for building dynamic and interactive user interfaces. It allows

developers to create reusable components, improving code organization and maintainability. In this project, React is used to design the frontend interface for students, faculty, and administrators.

Vite is used as a build tool for React, providing faster development and optimized performance. It enables quick loading and efficient rendering of components, enhancing user experience. The combination of React and Vite ensures that the application is responsive and user-friendly across different devices.

React also supports seamless integration with backend APIs, allowing real-time data updates and smooth interaction with the system. This makes it suitable for building modern web applications with interactive dashboards.



Fig. 3.4.2.3 React JS

PostgreSQL

PostgreSQL is a powerful open-source relational database management system used for storing and managing structured data. It is known for its reliability, scalability, and support for complex queries. In this project, PostgreSQL is used to store academic data, user details, and system-related information.

The database ensures efficient data retrieval and supports multiple users accessing the system simultaneously. It maintains data consistency and integrity through transactions and constraints. PostgreSQL also provides advanced features such as indexing and query optimization, which improve system performance.

Its scalability makes it suitable for handling large datasets and growing user bases. Overall, PostgreSQL plays a crucial role in maintaining the data backbone of the system.



Fig. 3.4.2.4 Postgre Sql

Multi-Agent Architecture

The system is designed using a multi-agent architecture, where different agents are responsible for handling specific tasks independently. This approach improves modularity, scalability, and system efficiency. Each agent performs a dedicated function such as data management, performance analysis, recommendation generation, and notification handling.

The agents interact with each other to ensure smooth workflow and decision-making. For example, the data agent provides input to the analysis agent, which then triggers recommendation and notification agents. This distributed approach reduces system complexity and improves maintainability.

The multi-agent design allows the system to be extended easily by adding new agents in the future. It plays a key role in making the system intelligent and adaptive.

LangChain

LangChain is a framework used to build applications powered by language models. It helps in managing interactions between large language models and system components. In this project, LangChain is used to structure and control how the AI assistant processes user queries.

It enables chaining of different operations such as data retrieval, processing, and response generation. This makes the system more organized and efficient when handling complex queries. LangChain also supports integration with APIs and databases, allowing the assistant to provide meaningful and context-aware responses.

By using LangChain, the system improves the quality of interaction and ensures better handling of user queries.



Fig. 3.4.2.5 Langchain

OpenAI API (ChatGPT)

The OpenAI API is used to integrate advanced natural language processing capabilities into the system. It enables the AI assistant to understand user queries and generate human-like responses. In this project, it is used to provide academic guidance and answer user questions.

The API allows the system to process natural language input and deliver meaningful outputs, improving user experience. It enhances interaction by making the system more intuitive and accessible.

This integration adds an intelligent conversational layer to the system, making it more user-friendly and interactive.



Fig. 3.4.2.6 OpenAI

Ollama

Ollama is used to run language models locally, providing an alternative to cloud-based AI services. It allows the system to perform AI-based interactions without relying entirely on external APIs. This improves flexibility and reduces dependency on internet connectivity.

In this project, Ollama is considered as an optional component for future enhancement, enabling local processing of AI queries. It can improve performance and data privacy when fully implemented.



Fig. 3.4.2.7 Ollama

REST API Architecture

REST API architecture is used to enable communication between the frontend and backend systems. It allows data to be exchanged using standard HTTP methods such as GET, POST, PUT, and DELETE.

This architecture ensures that the system remains modular and scalable. It also simplifies integration between different components such as frontend, backend, and database. REST APIs play a crucial role in maintaining smooth and efficient data flow within the system.



Fig. 3.4.2.8 Rest API

3.5 ANALYSIS

The analysis phase defines the requirements and functionalities of the proposed GVP-MAAA (Multi-Agent Academic Assistant System), independent of how the system will be implemented. This phase focuses on understanding the challenges faced in academic environments, such as difficulty in tracking student performance, lack of timely insights, and absence of intelligent

guidance for students and faculty.

The goal of the analysis phase is to clearly define the system requirements and develop a model that represents how the system will operate from the user's perspective. The system aims to provide meaningful insights, improve decision-making, and offer interactive support through a multi-agent architecture.

The outcome of this phase is a requirement specification document that describes the system behaviour, inputs, outputs, and interactions between different components. The analysis model represents the system in terms of functionality, objects, and behaviour.

The analysis model focuses on representing the system from the perspective of students, faculty, and administrators, ensuring that the developed system effectively addresses real academic challenges.

3.5.1 Entity Objects

Entity objects represent the core data and information stored and processed by the system. These objects contain essential academic and user-related data that remain persistent within the database.

Examples include:

Student details (marks, attendance, performance records)

Subject and course information

Placement and academic records

User profiles and authentication data

These objects form the foundation of the system and are used by different agents for analysis and decision-making.

3.5.2 Boundary Objects

Boundary objects represent the interaction between users and the system. They act as interfaces through which users provide inputs and receive outputs.

These interfaces collect inputs such as:

User login credentials

Academic queries through the AI assistant

Requests for performance analysis

They display outputs such as:

Dashboards and performance insights

Recommendations and alerts

Responses from the AI assistant

The boundary objects ensure smooth communication between users and the system.

3.5.3 Control Objects

Control objects manage the communication between boundary objects and entity objects. They coordinate system operations and ensure that user requests are processed correctly.

In this project, control objects include:

Data Management Controller – Handles storage and retrieval of academic data

Performance Analysis Controller – Processes academic data and generates insights

Recommendation Controller – Provides suggestions based on performance

Notification Controller – Manages alerts and updates

AI Interaction Controller – Handles user queries and generates responses

These components interact with the database and system agents to produce meaningful outputs.

3.5.4 Object Interaction

Object interaction describes how different components within the system communicate to complete a specific task. Interaction diagrams such as sequence diagrams represent the flow of actions between the user interface, backend system, and database.

For example, when a student requests performance analysis:

The user interface sends the request to the backend

The system retrieves data through the Data Management module

The Performance Analysis component processes the data

The Recommendation module generates suggestions

The results are displayed to the user

Similarly, when a user interacts with the AI assistant, the system processes the query and returns a relevant response using the AI module.

3.5.5 Object Behaviour

Object behaviour describes how the system changes its state based on user actions or system events. State transitions represent different stages of system operation.

In this project, the system moves through states such as:

User login

Data retrieval

Performance analysis

Recommendation generation

Notification triggering

AI response generation

Each state represents a step in the workflow, and transitions occur when specific actions are performed by the user or triggered by the system. This ensures smooth and structured system operation.

4.SYSTEM DESIGN

4.SYSTEM DESIGN

System Design is the process of defining the architecture, components, modules, interfaces, and data of a system to satisfy the specified requirements.

In System design, developers:

- Define the design goals of the project
- Decompose the system into smaller subsystems
- Design hardware/software strategies
- Design persistent data management strategies
- Design global control flow strategies
- Design access control policies and
- Design strategies for handling boundary conditions.

System design is not algorithmic. It is composed of several activities. They are:

- Identify design goals
- Design the initial subsystem decomposition
- Refine the subsystem decomposition to address the design goals

System Design is the transformation of the analysis model into a system design model. Developers define the design goals of the project and divide the system into smaller modules such as The GVP-MAAA system follows a **multi-agent, layered architecture** that integrates: Web-based interface, Backend processing, Database storage, AI-based modules as architected, structured, and implemented to meet the functional and non-functional requirements.

Developers also select strategies for building the system, such as the software and hardware platform on which the system will run, the database management strategy using Firebase, the global control flow of the application, and the access control for user interaction. The result of the system design is a model that includes a clear description of these strategies, subsystem decomposition, and UML diagrams that represent the structure and functioning of GVP-MAAA

4.1 Design Goals

Design goals are the qualities that the system should focus on during development.

- **UserFriendly:**

The interface should be **simple and intuitive**. Users should easily understand how to use the system. Minimal training should be required.

- **Reliability:**

The system should work **consistently without failures**. It should ensure **high availability**. Backup mechanisms should prevent data loss.

- **Efficiency:** □

The system should provide fast response times. Data processing (attendance, marks) should be efficient. The system should handle multiple requests simultaneously..

4.2 System Architecture

The GVP-MAAA system is structured as a three-layer web application: a Presentation Layer, a Multi-Agent Application Layer, and a Data Layer.

Presentation Layer (Frontend)

The Presentation Layer is developed using React.js / Next.js with Tailwind CSS for responsive design. Chart.js and Recharts are used for graphical performance data. The MAAAAgent React component (MAAA_Agent_3.jsx) is the core of this layer — rendering the role selection screen, the agent strip showing all five active agents, the chat message window with per-agent color coding and icon badging, context-aware suggestion chips, and the NLP input area.

Multi-Agent Application Layer (Backend)

The backend is implemented using FastAPI (Python), hosting five agent modules coordinated by an Agent Controller. The chatbot layer (implemented in MAAA_Agent_3.jsx) communicates with the Anthropic Claude API (claude-sonnet-4-20250514) with a role-context-aware system prompt injected per query. Agent detection is performed client-side using the detectAgent() function, and the identified agent context is embedded into the system prompt before the API call.

Data Layer

The Data Layer comprises a MySQL/PostgreSQL relational database storing user profiles, attendance records, marks, timetables, announcements, alerts, and system logs. The schema is normalized to Third Normal Form (3NF). Softdelete mechanisms (is_deleted, deleted_at columns) preserve data integrity across user management operations.

4.3 System Design

This diagram illustrates the overall structure of the system, including the frontend interface, backend server, multiagent modules, and database. It shows how different components interact to provide academic services for students, faculty, and administrators. The architecture

ensures smooth communication between modules and efficient management of academic data.

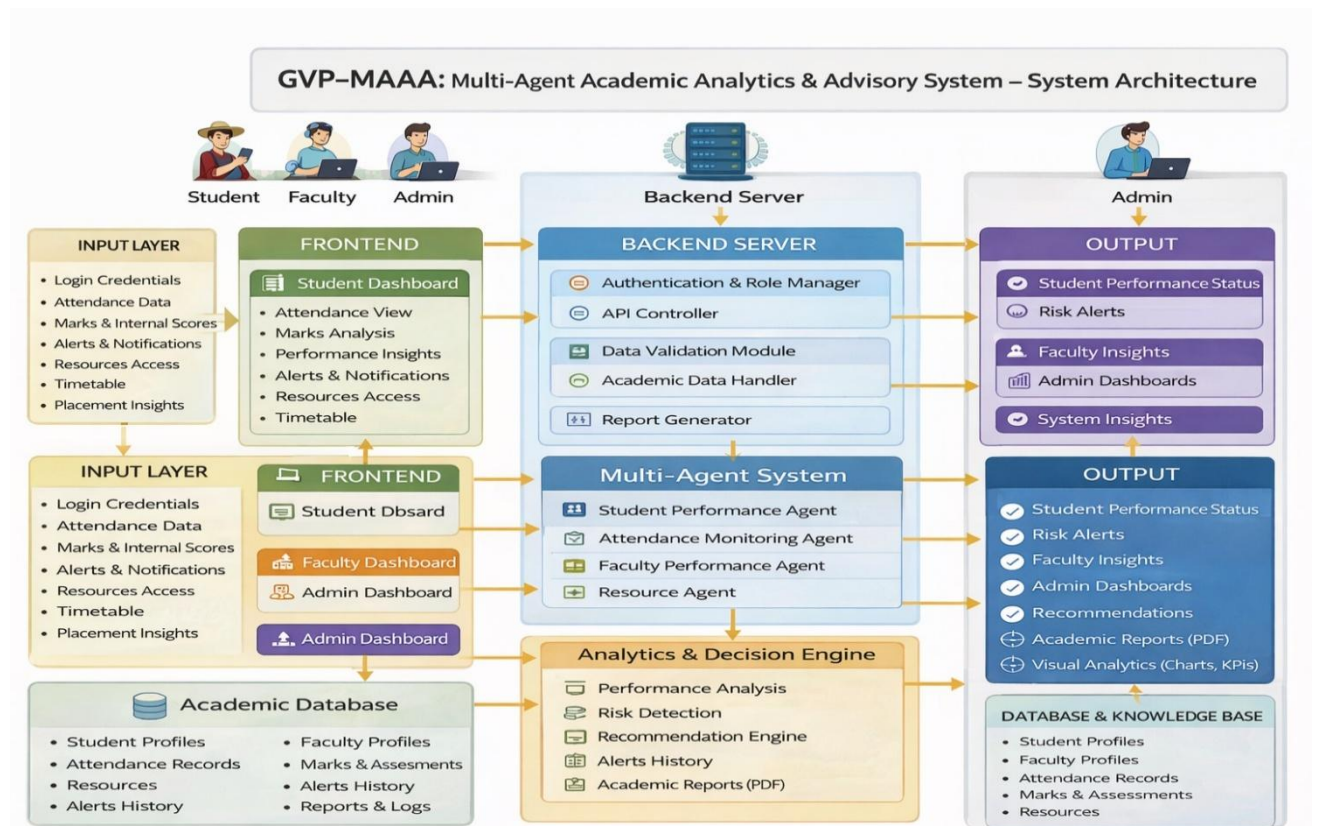


Fig. 4.3.1 System's Block Diagram

4.4 UML Diagrams

A UML diagram is a diagram based on the UML (Unified Modelling Language) with the purpose of visually representing a system along with its main actors, roles, actions, artifacts or classes, in order to better understand, alter, maintain, or document information about the system.

UML is a modern approach to modelling and documenting software. In fact, it's one of the most popular business process modelling techniques. It is based on diagrammatic representations of software components. As the old proverb says: "a picture is worth a thousand words". By using visual representations, we are able to better understand possible flaws or errors in software or business processes.

Mainly UML has been used as a general-purpose modelling language in the field of software engineering. However, it has now found its way into the documentation of several business processes or workflows. For example, activity diagrams, a type of UML diagram, can be used as a replacement for flowcharts. They provide both a more standardized way of modelling workflows as well as a wider range of features to improve readability.

4.4.1 Use Case Diagram

In the Unified Modelling Language (UML), a use case diagram can summarize the details of your system's users (also known as actors) and their interactions with the system. To build one, use a set of specialized symbols and connectors.

- This diagram represents the functional scope of the GVP-MAAA system.
- It shows three main actors: Student, Faculty, and Admin.
- Each actor interacts with the system based on their role-specific permissions.
- Common functionalities like Login, Overview, Alerts, and Logout are shared.
- Faculty and Admin handle data creation and management, while Students mainly view and consume data.
- The diagram clearly defines system boundaries and responsibilities.

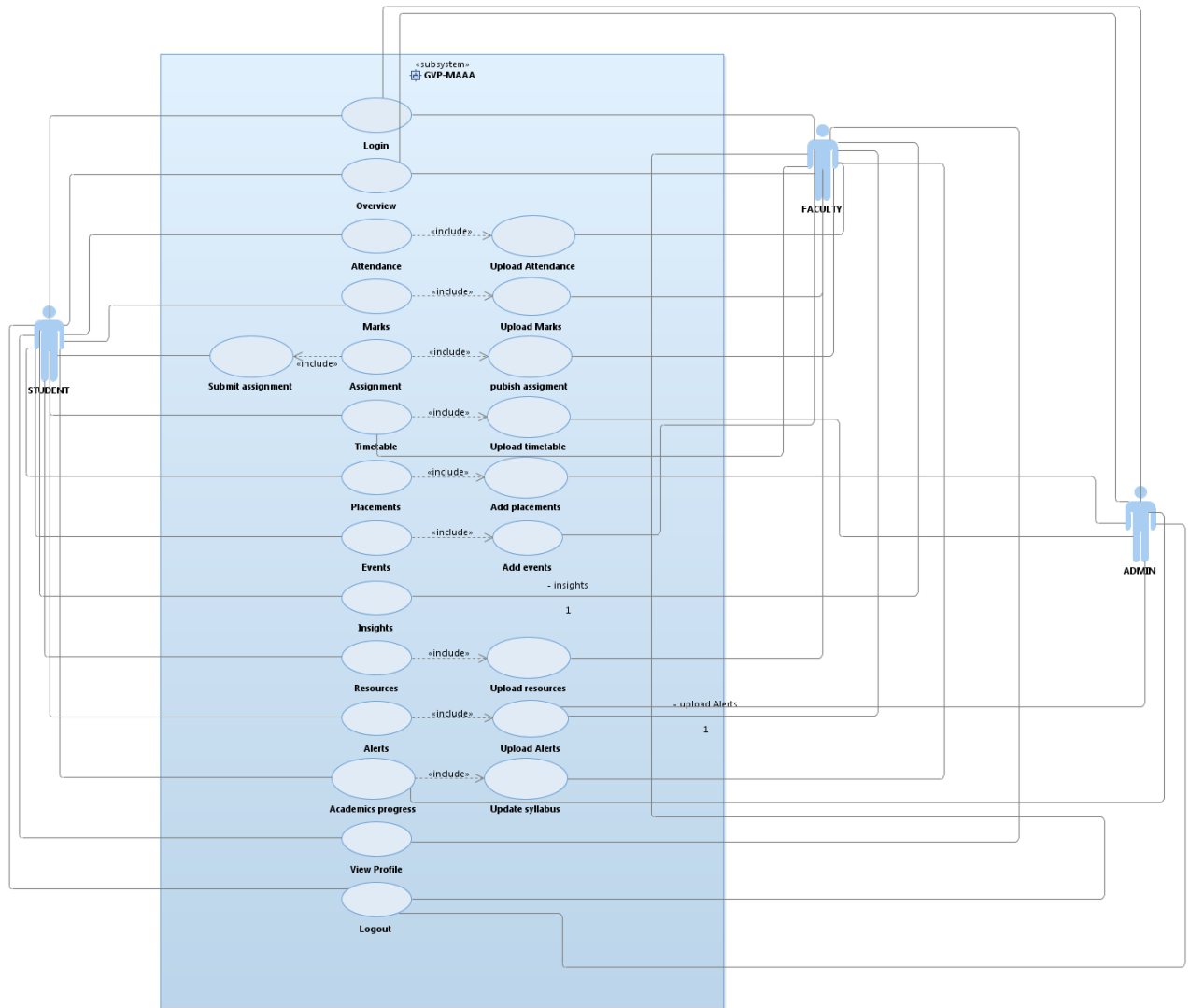


Fig. 4.4.1.1 Use Case Diagram

4.4.2 Class Diagram

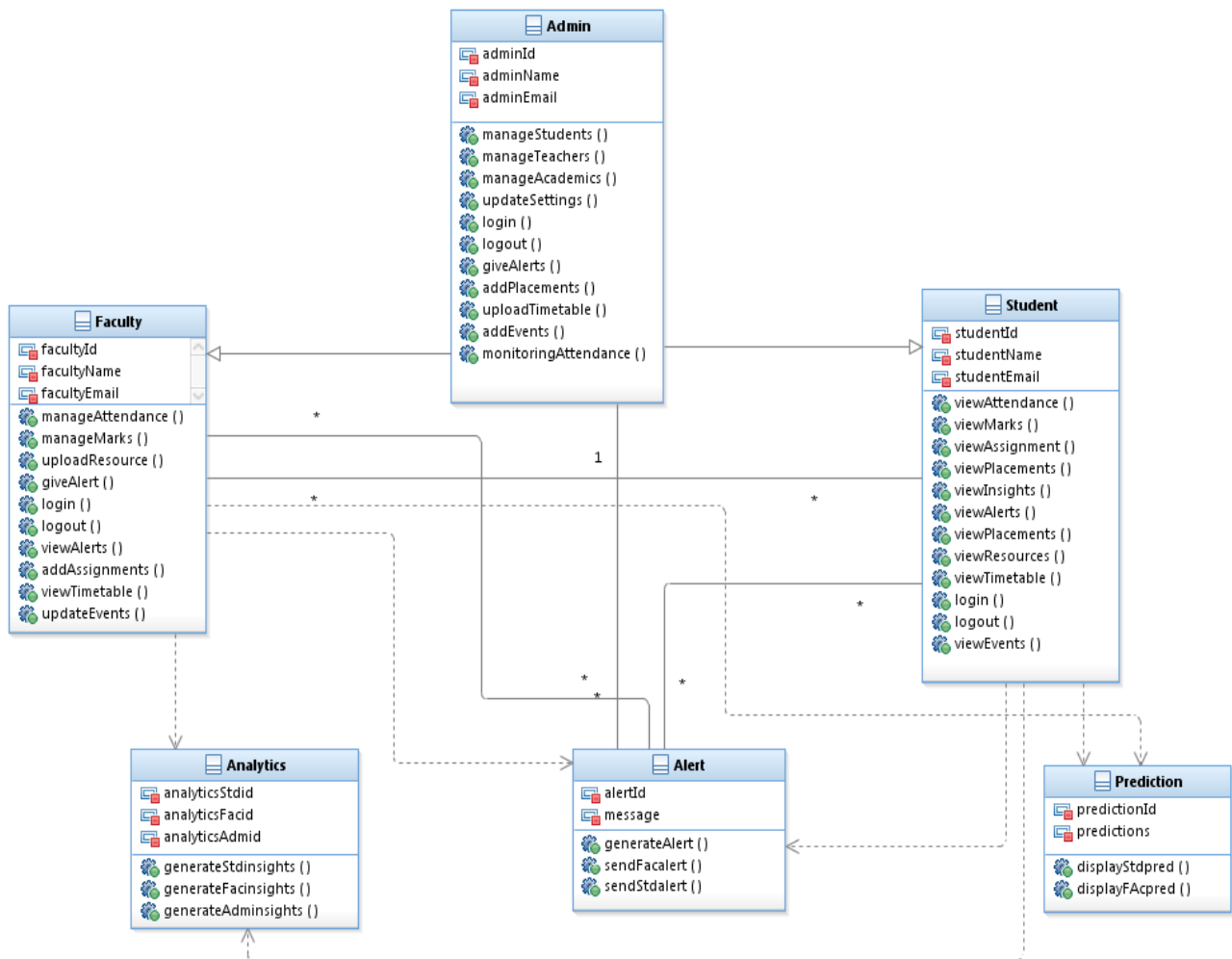


Fig. 4.4.2.1 Class Diagram

- This diagram shows the static structure of the system.
- It defines major classes such as Student, Faculty, Admin, Analytics, Alert, and Prediction.
- Each class contains attributes (data) and methods (operations) relevant to its role.
- Relationships represent associations, dependencies, and interactions between entities.
- Analytics and Prediction classes support insight generation and future trend analysis.
- This diagram helps in understanding backend design and data flow.

4.4.3 Activity Diagram

Activity diagram is basically a flowchart to represent the flow from one activity to another activity. The activity can be described as an operation of the system.

The control flow is drawn from one operation to another. This flow can be sequential, branched, or concurrent. Activity diagrams deal with all type of flow control by using different elements such as fork, join, etc. Activity is a particular operation of the system. Activity diagrams are not only used for visualizing the dynamic nature of a system, but they are also used to construct the executable system by using forward and reverse engineering techniques.

The purpose of an activity diagram can be described as

- Draw the activity flow of a system.
- Describe the sequence from one activity to another.
- Describe the parallel, branched and concurrent flow of the system

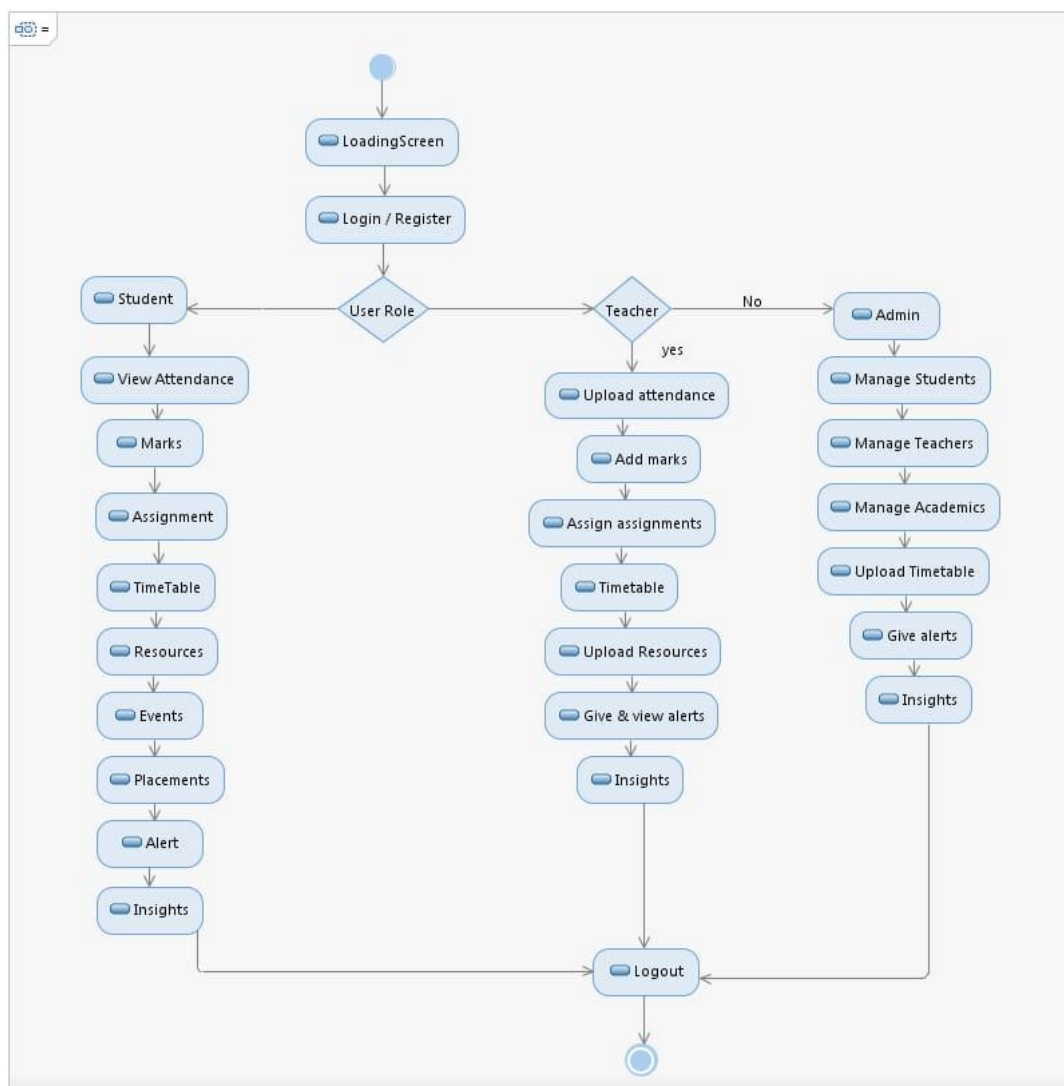


Fig 4.4.3.1 Activity Diagram

4.4.4 Sequence Diagram - Student

- This diagram illustrates the interaction flow for a student user.
- The process starts with authentication (login).
- The student accesses features like attendance, marks, insights, and predictions.
- The backend fetches data, performs analytics and risk evaluation, and returns results.
- Alerts are generated automatically if the student is academically at risk.
- The sequence ends with logout and session termination.

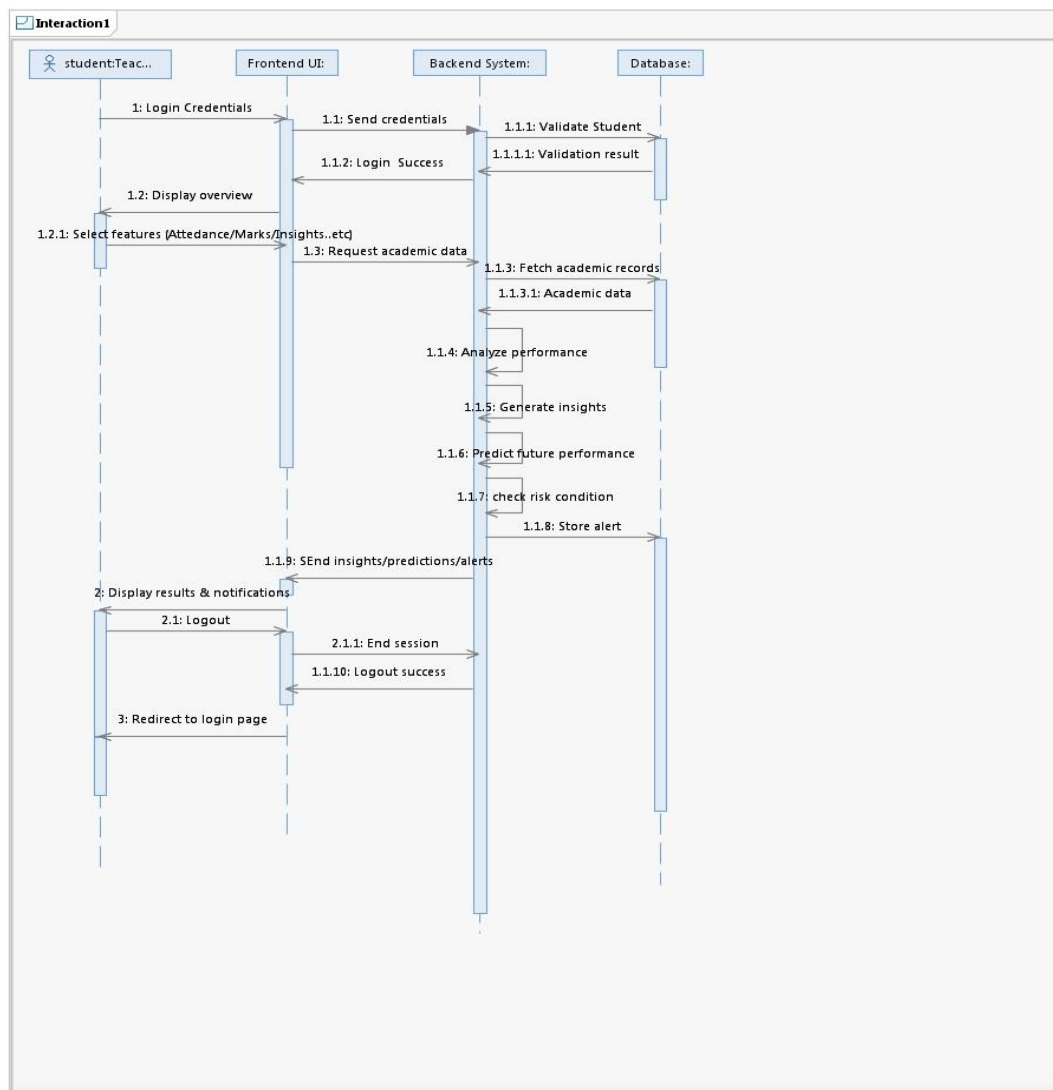


Fig. 4.4.4.1 Sequence Diagram Student

Sequence Diagram - Faculty

- This diagram explains the workflow of a faculty member.
- Faculty logs in and manages attendance, marks, and academic resources.
- Faculty can upload resources, view analytics, and monitor student performance.
- Insights are generated by the backend using academic data.
- Faculty can send alerts to students when required.
- The sequence concludes with a secure logout process.

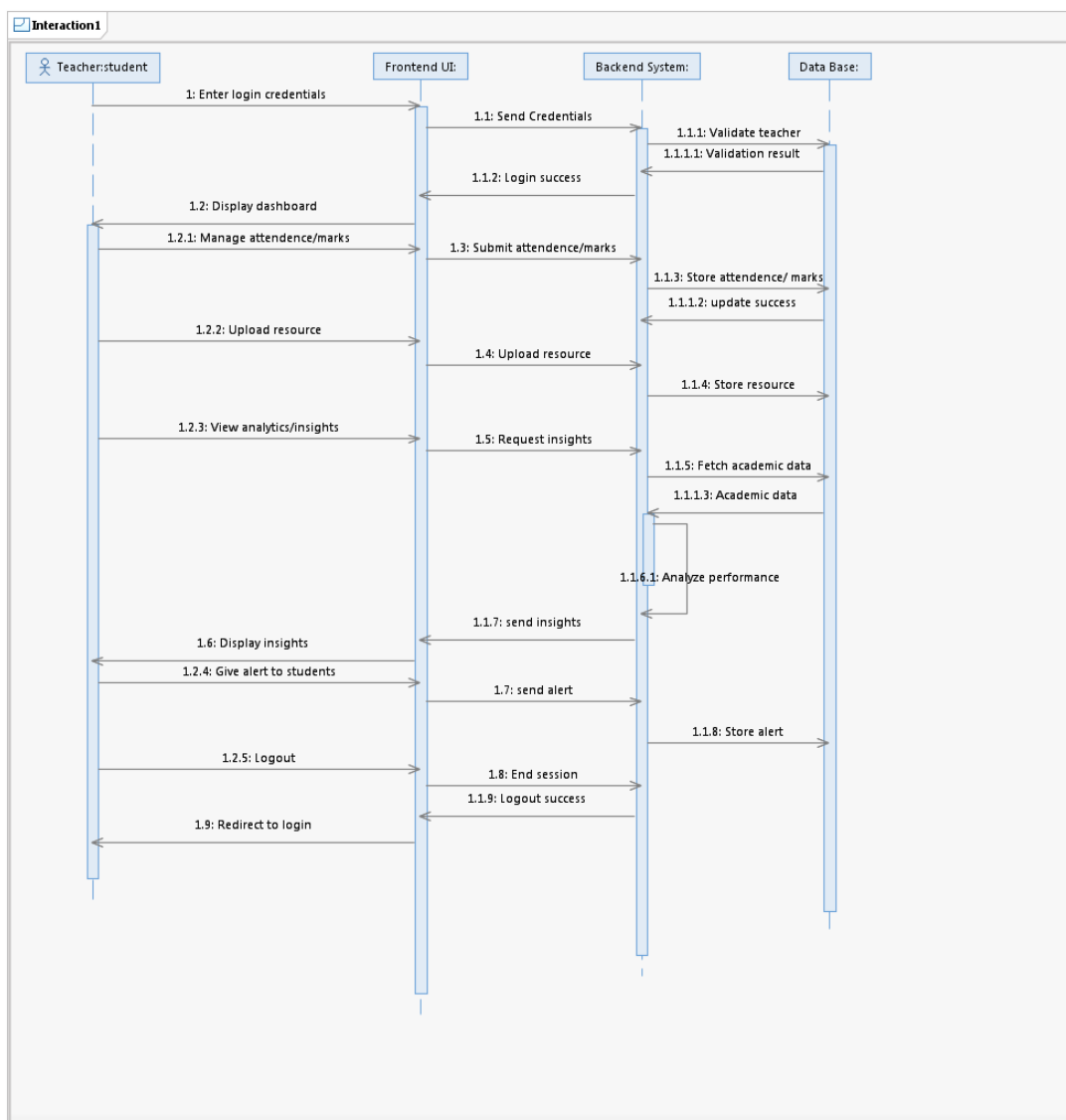


Fig.4.4.4.2Sequence Diagram - Faculty

Sequence Diagram - Admin

- This diagram represents the admin-level system interactions.
- Admin performs user management (students and faculty).
- Admin monitors overall academic performance and trends.
- The system generates analytics, predictions, and reports for decision-making.
- Admin can send alerts and generate downloadable reports.
- The diagram ends with proper session closure and logout.

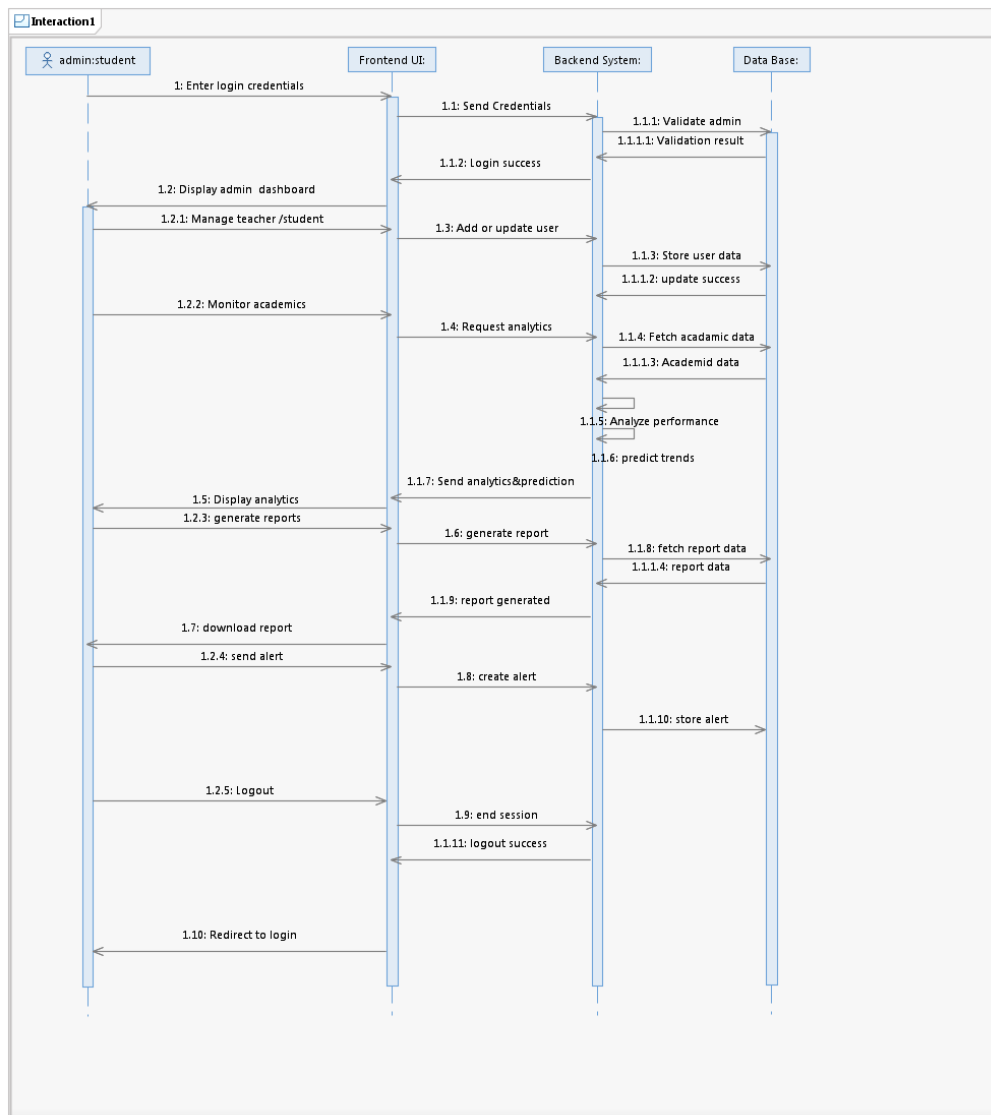


Fig.4.4.4.3Sequence Diagram - Admin

4.5 Data Flow Diagrams

Data Flow Diagram (DFD) represents the flow of data within a system and illustrates how information is processed, stored, and transferred between different components. It uses standard symbols such as rectangles (external entities), circles (processes), arrows (data flow), and cylinders (data storage) to provide a clear understanding of system functionality.

DFDs help in visualizing how data moves through the system and how different modules interact with each other. They can be designed at different levels, starting from a high-level overview and gradually expanding into detailed processes. In this project, the DFD is used to represent the working of the GVP-MAAA system, including user interactions, system processing, and data storage.

4.5.1 DFD Level-0

DFD Level 0, also known as the Context Diagram, provides a high-level overview of the entire system. It represents the system as a single process and shows its interaction with external entities.

In this project, the external entities include Students, Faculty, and Administrators, who interact with the system by providing inputs such as login requests, academic data, and queries. The system processes these inputs and generates outputs such as insights, recommendations, and notifications.

The diagram also shows the interaction between the system and the database, where data is stored and retrieved. This level focuses on providing a clear and simplified view of how the system communicates with users and manages data.

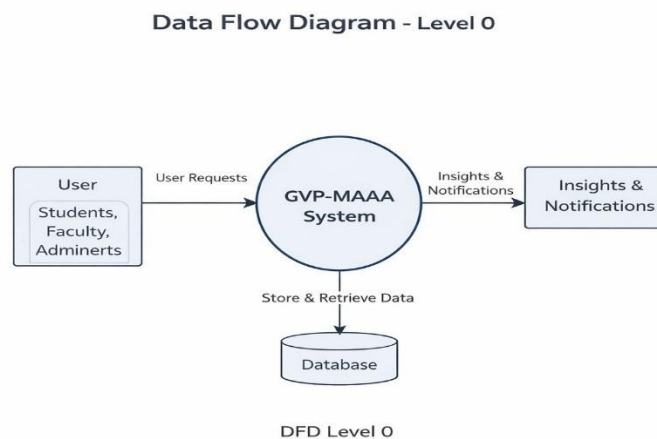


Fig. 4.5.1 DFD Level -0

4.5.1 DFD Level-1

DFD Level 1 provides a more detailed representation of the system by breaking down the main process into multiple sub-processes. It highlights the internal functioning of the system and shows how data flows between different modules.

In the proposed system, the main process is divided into multiple components based on the multi-agent architecture. These include:

Data Management Agent – Handles storage and retrieval of academic data

Analysis Agent – Processes data and generates performance insights

Recommendation Agent – Provides suggestions based on analysis

Notification Agent – Sends alerts and updates to users

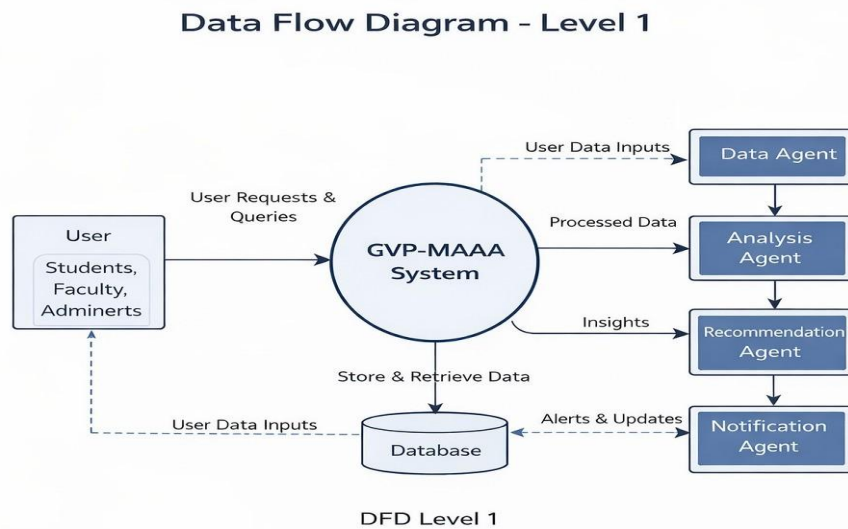


Fig 4.5.2 DFD Level 1

4.6 Flow Chart

A flowchart is a type of diagram that represents an algorithm, workflow or process. Flowchart can also be defined as a diagrammatic representation of an algorithm. The flowchart shows the steps as boxes of various kinds, and their order by connecting the boxes with arrows. This diagrammatic representation illustrates a solution model to a given problem. Flowcharts are used in analysing, designing, documenting or managing a process or program in various fields.

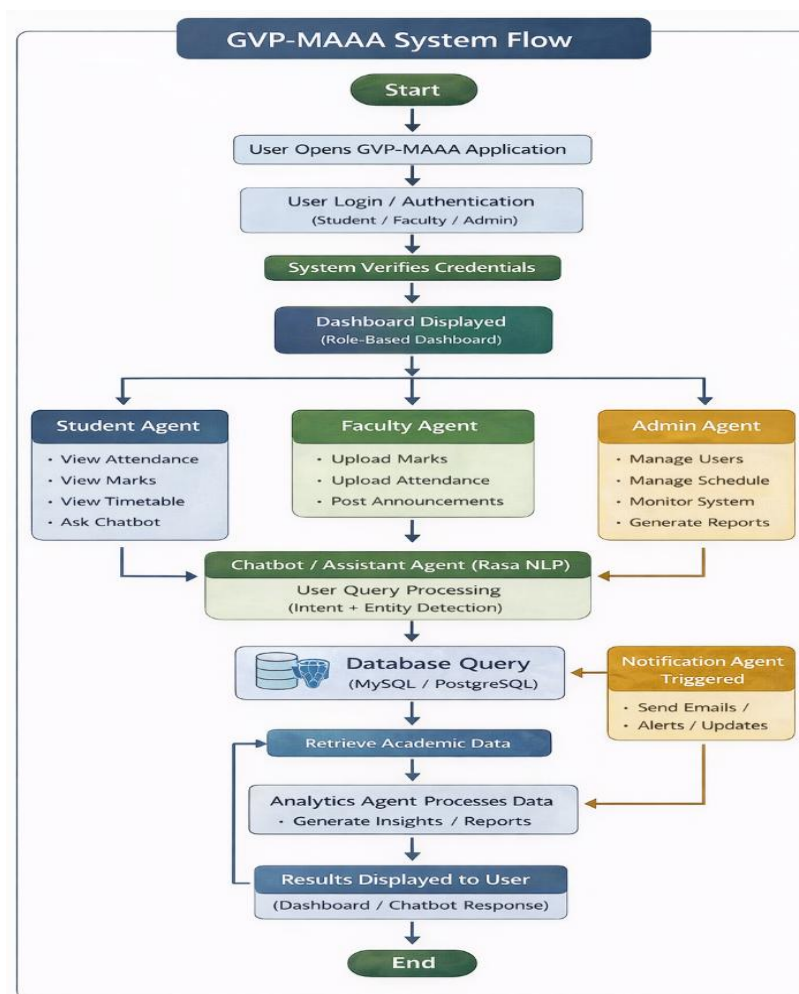


Fig. 4.6.1 Flow chart of the system

5.IMPLEMENTATION

IMPLEMENTATION

The implementation phase focuses on converting the system design into a working application using appropriate technologies and tools. It involves developing the frontend, backend, database integration, and multi-agent components to ensure the system functions as intended.

The main objective of implementation is to build a reliable and scalable system that can handle user interactions, process academic data, and generate meaningful insights. This phase plays a crucial role in determining the performance, usability, and maintainability of the system .

5.1 Coding Approach

The system follows a bottom-up approach for development. In this approach, individual modules such as user authentication, data management, analysis, and notification systems are developed independently and later integrated into a complete system.

Each module is designed as a separate component, making it easier to test, debug, and maintain. The multi-agent architecture further supports this approach by allowing each agent to operate independently while collaborating with other components.

5.2 Information Handling

Efficient data handling is essential for the system's performance. The system manages various types of data such as:

Student academic records (marks, attendance)

User details and authentication data

Placement and event information

PostgreSQL is used to store and manage this data, ensuring consistency and fast retrieval. The backend (FastAPI) processes user requests and interacts with the database to perform operations such as data insertion, retrieval, and updates.

5.3 Programming Style

A structured and modular programming style is followed to ensure code quality and maintainability. Proper naming conventions, code organization, and separation of concerns are maintained throughout the system.

Each module, including frontend components, backend APIs, and agent logic, is developed in a way that is easy to understand and extend. This improves readability and allows future enhancements such as adding new agents or features.

5.4 Verification and Validation

Verification ensures that the system is developed correctly, while validation ensures that the system meets user requirements.

The system includes validation at multiple levels:

Form-level validation: Ensures all required inputs are provided before processing

Field-level validation: Validates individual inputs such as login credentials and data entries

These validation mechanisms reduce errors and improve the overall user experience.

5.5 Implementation of Project

The proposed system GVP-MAAA (Multi-Agent Academic Assistant) is implemented as a web-based application that integrates frontend, backend, database, and AI components.

The system consists of the following modules::

1. User Management Module

Provides login and role-based access for students, faculty, and administrators

Ensures secure authentication and authorization

Allows users to access features based on their roles

2. Academic Data Management Module

Allows faculty/admin to manage student data such as marks and attendance

Stores and retrieves data using PostgreSQL

Maintains structured academic records

3. Performance Analysis Module (Analysis Agent)

Processes academic data to evaluate student performance

Identifies trends and weak areas

Generates meaningful insights for decision-making

4. Recommendation Module (Recommendation Agent)

Provides suggestions based on student performance

Helps students improve in weak subjects

Supports personalized academic guidance

5. Notification Module (Notification Agent)

Sends alerts related to performance, attendance, and updates

Notifies both students and faculty

Ensures timely communication

6. AI Assistant Module

Allows users to interact using natural language

Uses OpenAI API / LangChain for intelligent responses

Provides academic guidance and query handling

7. Frontend Interface (React)

Provides a responsive and user-friendly interface

Displays dashboards, insights, and reports

Enables smooth interaction with the system

8. Backend System (FastAPI)

Handles API requests and business logic

Manages communication between frontend, agents, and database

Supports scalable and efficient processing

9. Database System (PostgreSQL)

Stores all system data securely

Supports multiple users and concurrent operations

Ensures data integrity and consistency

10. Integrated Multi-Agent Workflow

Data flows from user → backend → agents → database

Agents process data and generate outputs

Results (insights, recommendations, alerts) are delivered to users

This integration ensures that the system operates efficiently and provides intelligent outputs.

5.6 System Workflow & Process Flow

The user logs into the system using valid credentials

The user selects required features such as viewing data or requesting insights

The system processes the request through backend and agent modules

Academic data is retrieved and analyzed

Insights, recommendations, and notifications are generated

Results are displayed through the dashboard or AI assistant

The user can interact further or logout from the system

6.TESTING

6.TESTING

Testing is the process of identifying differences between expected system behavior and actual system performance. It plays a critical role in ensuring the reliability, accuracy, and efficiency of the developed system. The application must be thoroughly tested to detect and eliminate errors before deployment .

6.1 Testing Activities

Testing of the system was carried out in a structured and incremental manner. Since the system consists of multiple components such as frontend (React), backend (FastAPI), database (PostgreSQL), and multi-agent modules, each component was tested independently before integration.

The following approach was followed:

Individual modules like login, dashboard, and data management were tested

Backend APIs were tested using FastAPI Swagger UI

Multi-agent components (Data Agent, Analysis Agent, Recommendation Agent, Notification Agent) were tested separately

Finally, all modules were integrated and tested as a complete system

This approach helped in early detection of errors and ensured smooth system integration.

6.2 Testing Types Unit Testing

Equivalence testing was used to categorize inputs into valid and invalid groups.

Examples:

Valid vs invalid login credentials

Correct vs incorrect academic data inputs

Valid vs unsupported AI queries

This reduces redundant test cases while maintaining coverage.

6.2.1 Boundary Testing

Boundary testing was applied to validate edge conditions.

Examples:

Attendance range (0% to 100%)

Marks limits (minimum and maximum values)

Empty or null inputs

This ensures the system behaves correctly at extreme values.

6.2.2 Path testing:

Path testing was used to verify different execution paths.

Examples:

Login → Dashboard → Insights → Recommendations

User query → AI Agent → Response

Data input → Agent processing → Notification

This ensures all logical flows are working correctly.

Integrating Testing

Integration testing was performed by combining different modules and verifying their interactions.

Frontend ↔ Backend API communication tested

Backend ↔ Database interaction validated

Backend ↔ Multi-agent system communication verified

A top-down integration approach was followed, where higher-level modules were tested first and then integrated with lower-level components.

API Testing (FastAPI Swagger)

API testing was performed using the built-in FastAPI Swagger UI, which provides an interactive interface for testing all backend endpoints.

- Each API endpoint was tested with different input scenarios
- Request and response formats were validated
- Error handling responses were verified

Authentication APIs were tested for security and access control

Swagger UI allowed real-time testing of APIs without requiring external tools like Postman, ensuring faster development and debugging.

Validation Testing

Validation testing ensures that the system meets user expectations and requirements.

Tested real academic scenarios (marks, attendance, insights)
Verified outputs generated by agents
Checked AI-based responses for relevance and correctness
Validation was mainly performed using black-box testing technique.

System Testing

System testing ensures that the entire application works as expected.

Key Activities:

Functional Testing

Verified login and role-based access (Student, Faculty, Admin)

Tested academic data management features

Validated dashboard insights and visualizations

Checked AI assistant responses

Performance Testing

Evaluated response time of APIs

Tested system under multiple user requests

Pilot Testing

Tested system with a small group of users

Feedback was collected and improvements were made

Acceptance Testing

Ensured system meets project objectives

Verified usability and correctness of outputs

Black Box Testing

Black box testing was used to test system functionality without considering internal implementation.

Focus areas:

Input validation

Output correctness

Error handling

Errors identified include:

Missing or incorrect functions

Interface errors

Database access issues

Performance delays

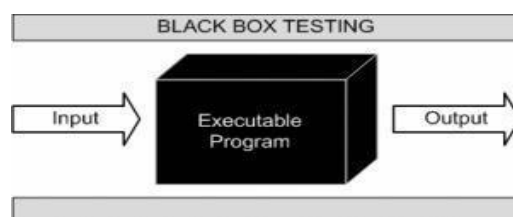


Fig. 6.2.1 Black Box Testing

6.3 Test cases

Test Case	Test Case Description	Test Data	Expected Result	Actual Result	Pass/Fail
1	Login System	Valid credentials	Login successful	Login successful	PASS
2	Invalid Login	Wrong credentials	Error message	Error displayed	PASS
3	Data Entry	Academic data	Stored correctly	Stored successfully	PASS
4	Data Retrieval	Fetch data	Data displayed	Data shown	PASS
5	Data Agent	Raw input data	Processed data	Processed correctly	PASS
6	Analysis Agent	Academic data	Insights generated	Insights correct	PASS
7	Recommendation Agent	Performance data	Suggestions generated	Recommendations shown	PASS
8	Notification Agent	Trigger alerts	Alerts sent	Alerts displayed	PASS
9	AI Assistant	User query	Relevant response	Response generated	PASS
10	Dashboard	Load data	Visualization shown	Working correctly	PASS
11	API Testing	API request	Valid response	Error shown	PASS

7.RESULTS

7.1 Landing page:

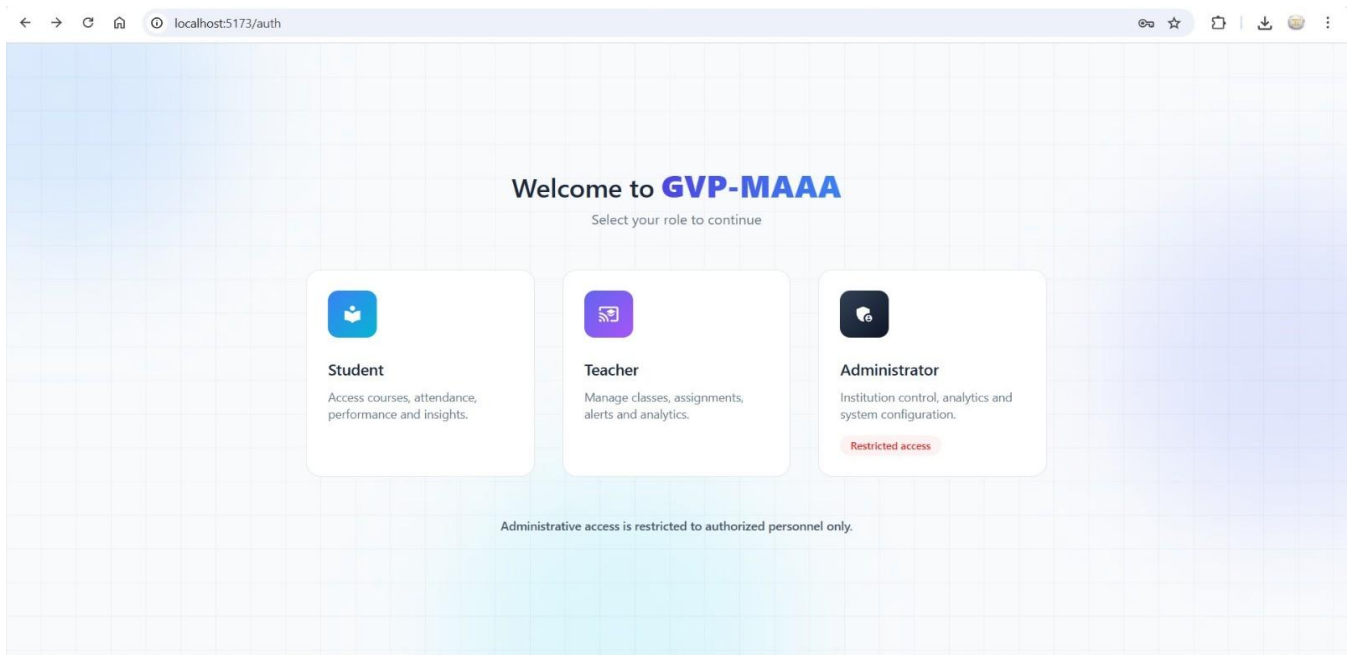


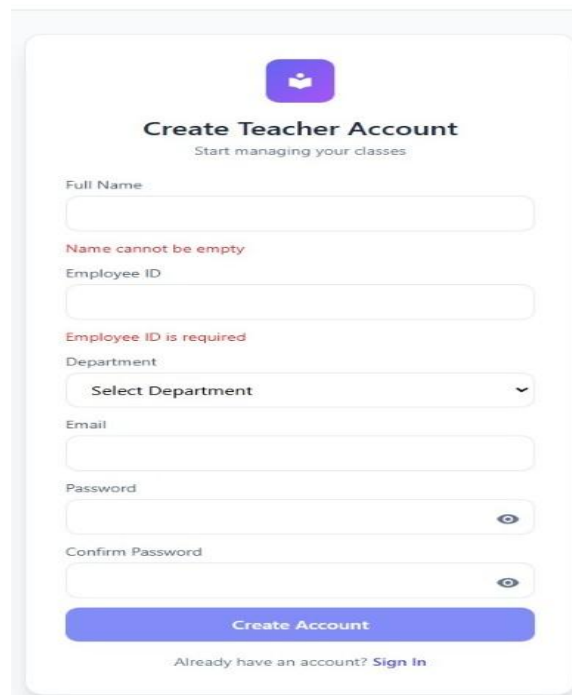
Fig. 7.1.1 Landing Page

7.2 Signup Page Student sign up page:

The image shows a 'Create Student Account' form. At the top, there is a blue icon of a book with a person inside. Below it, the title 'Create Student Account' is in bold, followed by the subtitle 'Start accessing your academic dashboard'. The form has five input fields: 'Full Name' (with a red error message 'Name cannot be empty'), 'Roll Number', 'College Email', 'Password' (with a toggle icon), and 'Confirm Password' (with a toggle icon). At the bottom, there is a blue button labeled 'Create Account' and a link 'Already have an account? Sign In'.

Fig. 7.2.1 Student signup page

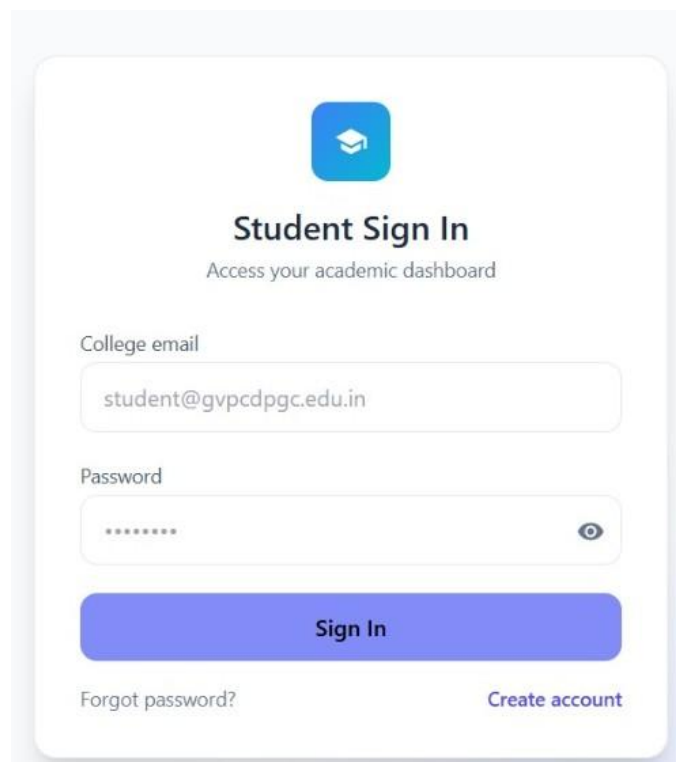
Teacher signup page:



The form is titled "Create Teacher Account" with a subtitle "Start managing your classes". It features a purple icon of a person with a book. The fields include: "Full Name" (with a red error message "Name cannot be empty"), "Employee ID" (with a red error message "Employee ID is required"), "Department" (a dropdown menu showing "Select Department"), "Email", "Password" (with an eye icon), and "Confirm Password" (with an eye icon). A purple "Create Account" button is at the bottom, followed by a link "Already have an account? Sign In".

Fig. 7.2.2 Teacher signup page

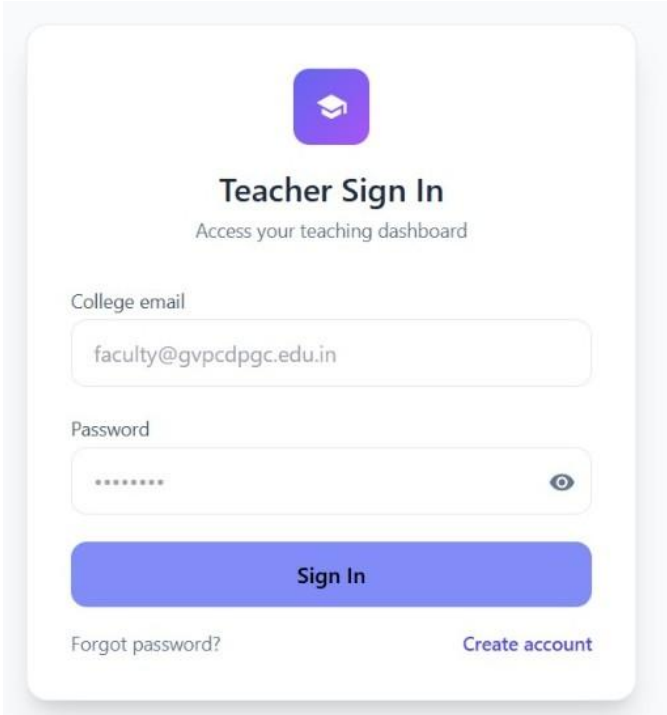
7.3 Login pages: Student login page:



The form is titled "Student Sign In" with a subtitle "Access your academic dashboard". It features a blue icon of a graduation cap. The fields include: "College email" (with the example "student@gvpcdpgc.edu.in") and "Password" (with a redacted password "*****" and an eye icon). A purple "Sign In" button is at the bottom. Below the button are links for "Forgot password?" and "Create account".

Fig. 7.3.1 Student login page

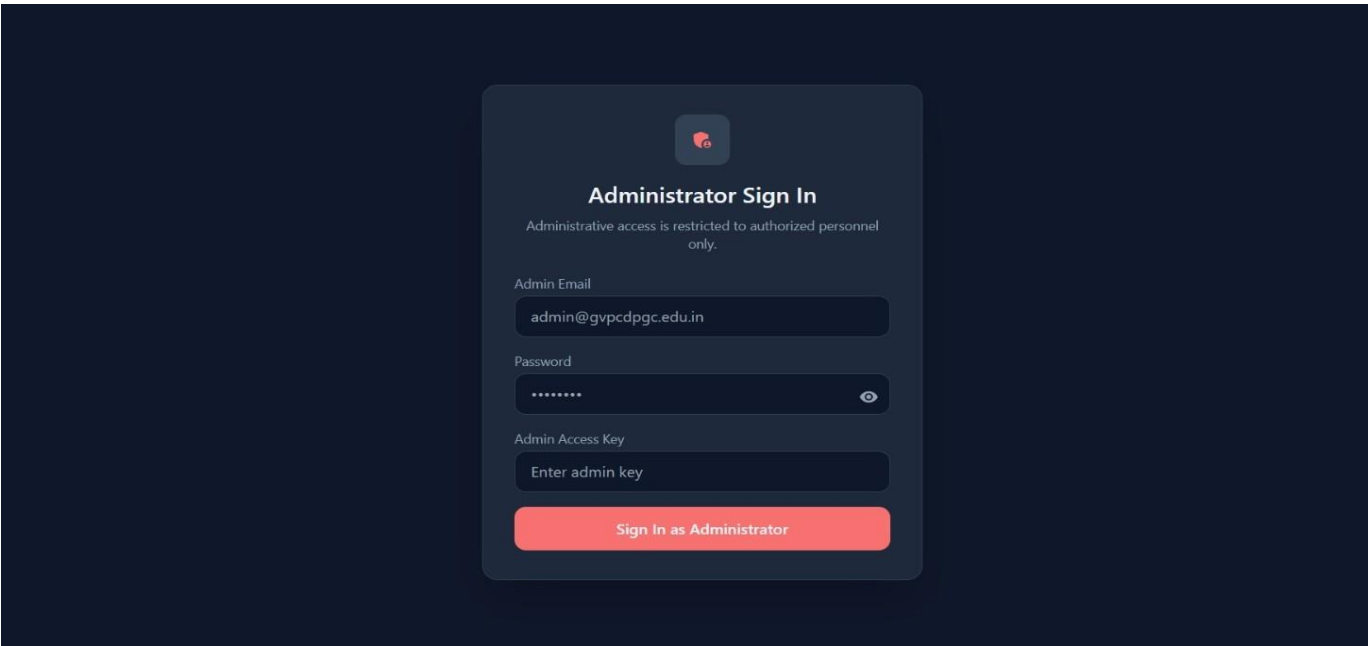
Teacher login page:



The image shows a 'Teacher Sign In' form. At the top is a purple square icon with a white graduation cap. Below it, the text 'Teacher Sign In' is centered, followed by the subtitle 'Access your teaching dashboard'. The form contains two input fields: 'College email' with the value 'faculty@gvpcdpgc.edu.in' and 'Password' with masked characters '*****'. A blue 'Sign In' button is positioned below the password field. At the bottom, there are two links: 'Forgot password?' on the left and 'Create account' on the right.

Fig. 7.3.2 Facutly Login Page

Admin login page:



The image shows an 'Administrator Sign In' form on a dark background. At the top is a dark blue square icon with a red 'G'. Below it, the text 'Administrator Sign In' is centered, followed by the subtitle 'Administrative access is restricted to authorized personnel only.'. The form contains three input fields: 'Admin Email' with the value 'admin@gvpcdpgc.edu.in', 'Password' with masked characters '*****', and 'Admin Access Key' with the placeholder text 'Enter admin key'. A red 'Sign In as Administrator' button is positioned below the 'Admin Access Key' field.

Fig. 7.3.3 Admin Login Page

7.4 Student Dashboard

Student Overview Page:

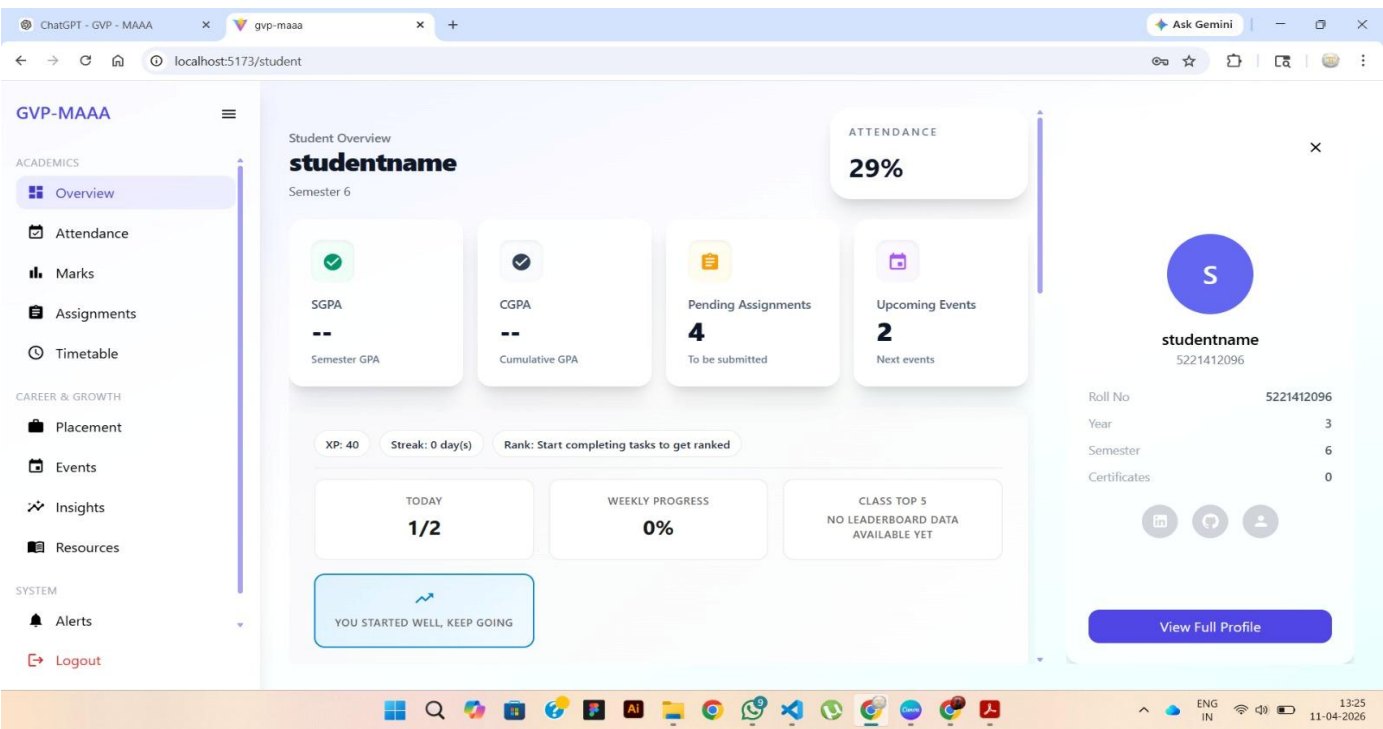


Fig. 7.4.1 Student Overview Page

Student attendance:

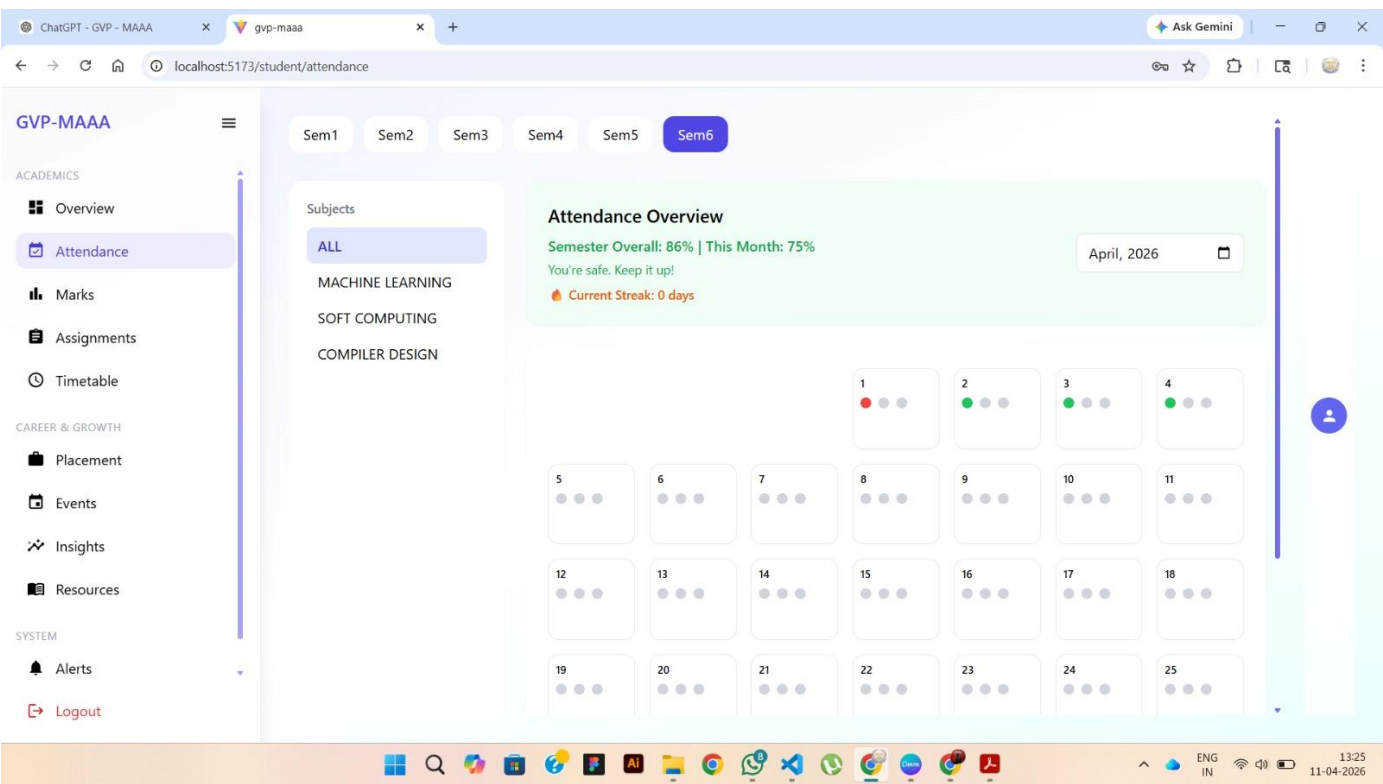


Fig. 7.4.2 Student Attendance Page

Student Assignment:

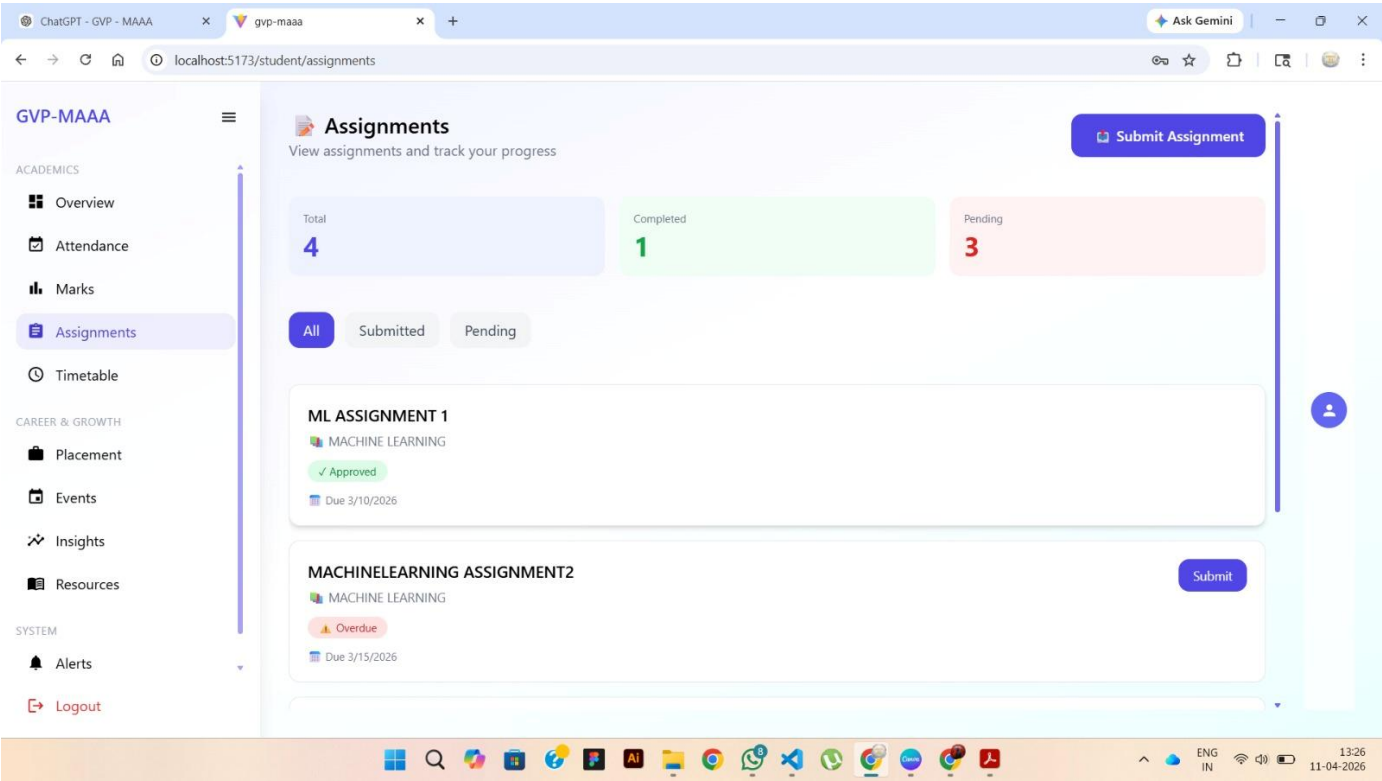


Fig. 7.4.3 Student Assignment Page

Student Timetable:

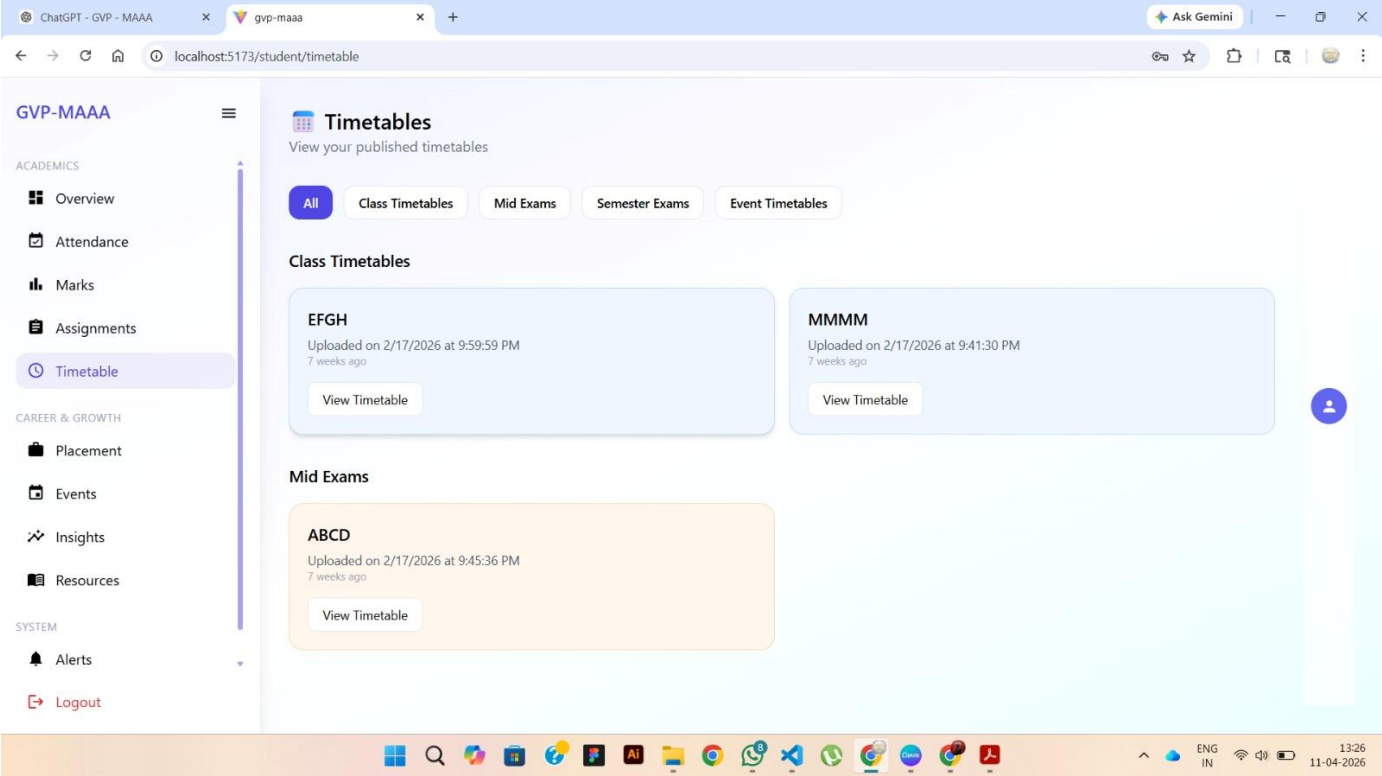


Fig. 7.4.4 Student Timetable Page

Student marks:

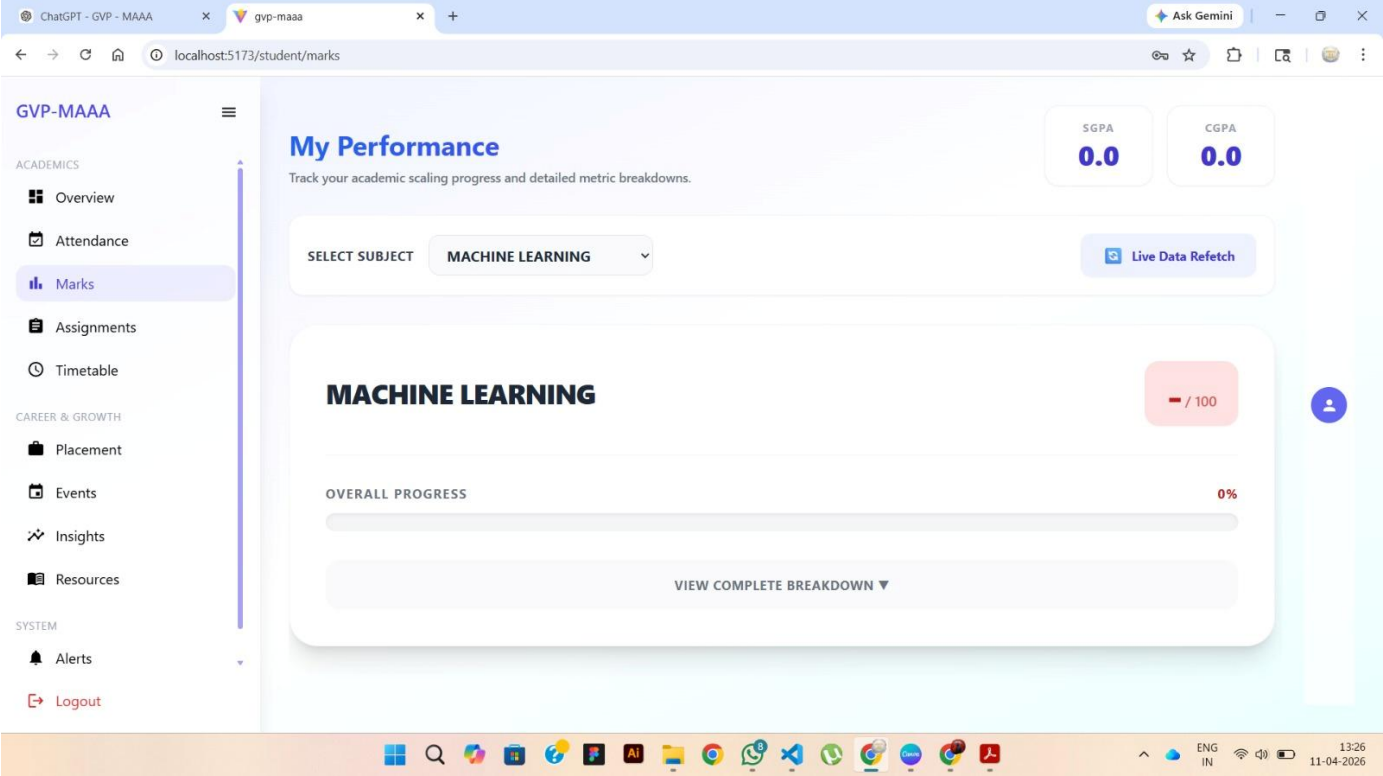


Fig. 7.4.5 Student Mark Page

Student Placements:

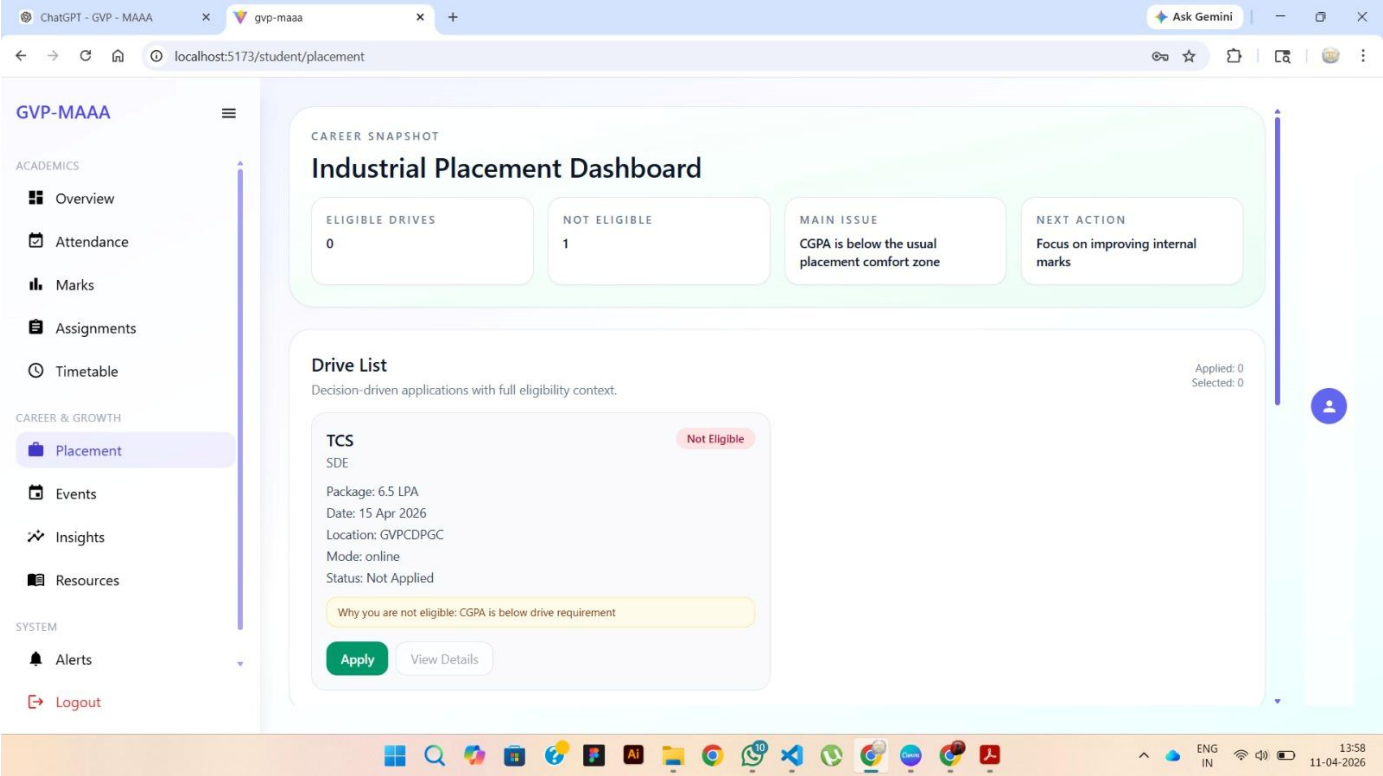


Fig. 7.4.6 Student Placement Page

Student events:

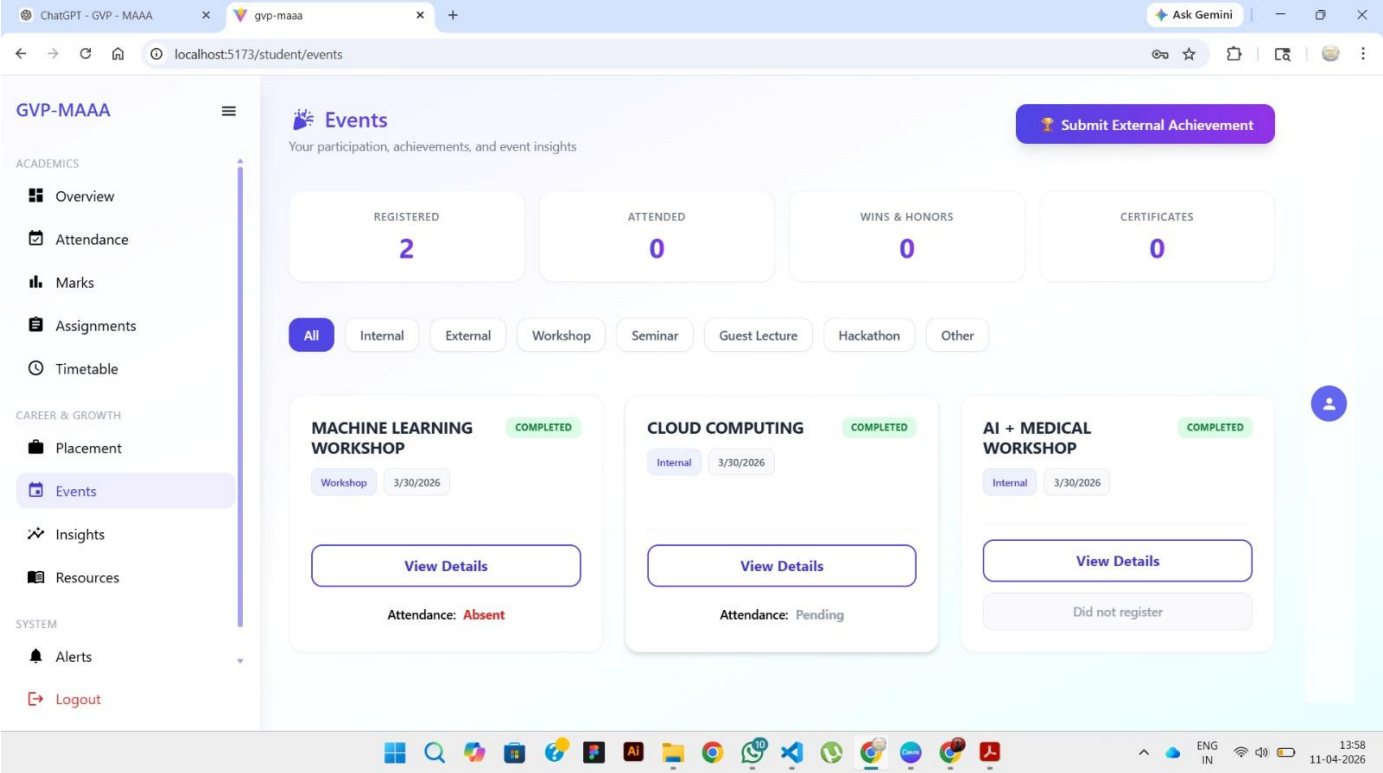


Fig. 7.4.7 Student Event Page

Student Insights:

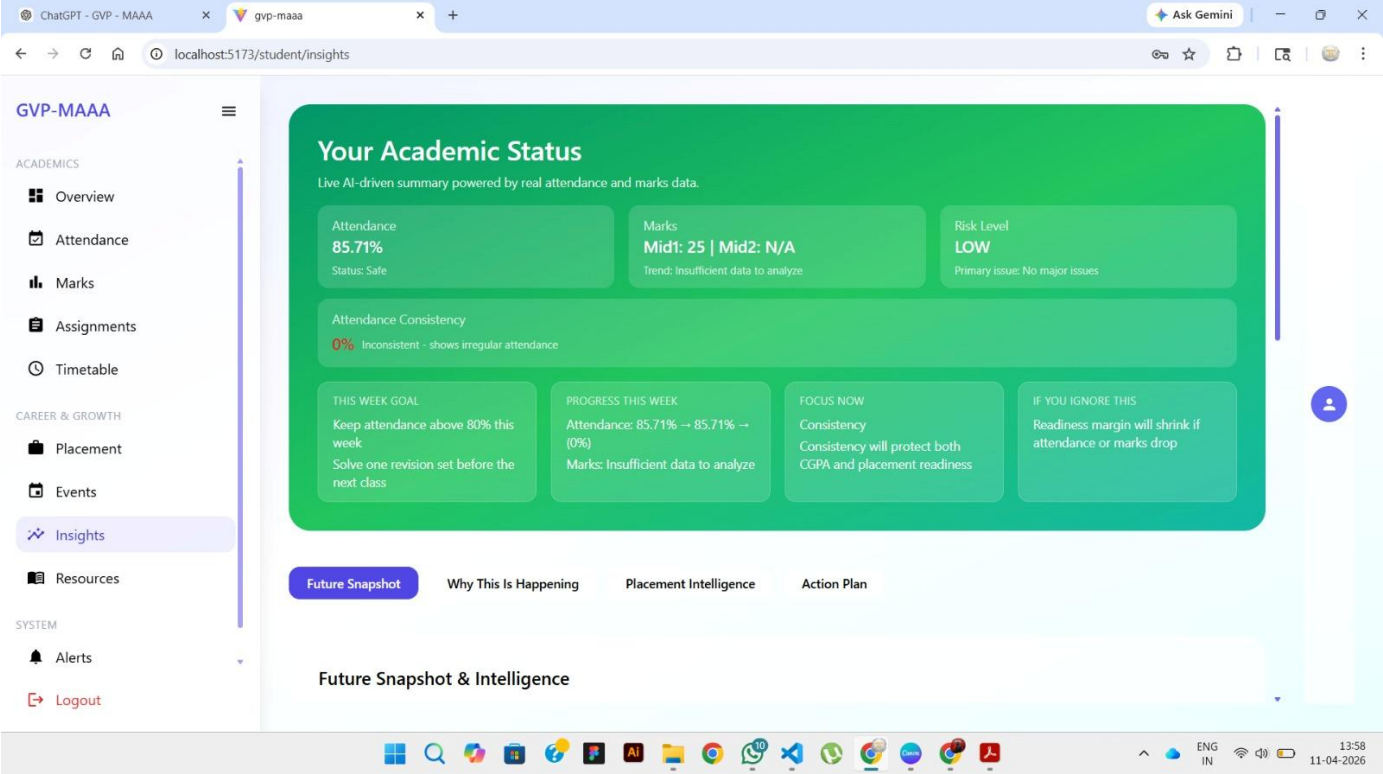


Fig. 7.4.8 Student Insights Page

Student Resources:

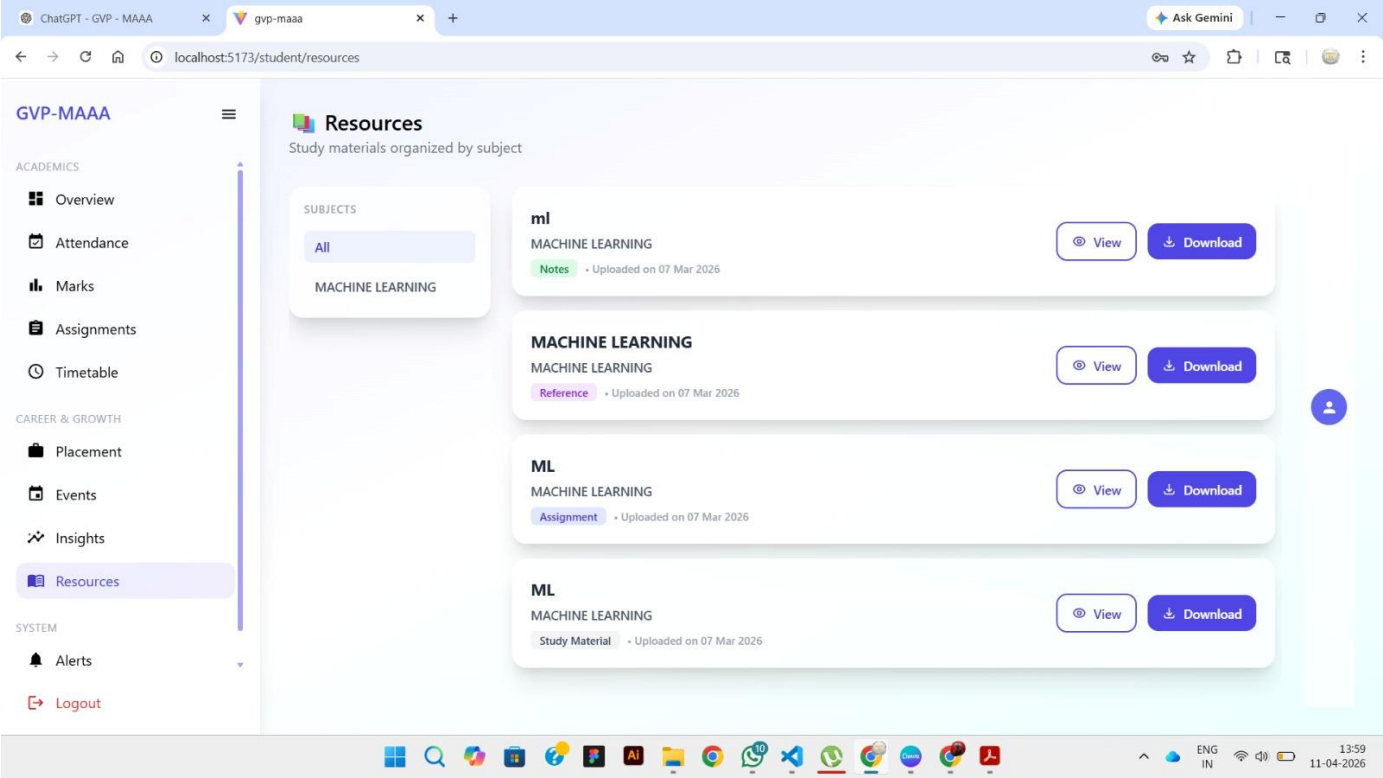


Fig. 7.4.9 Student Resource Page

Student Alerts:

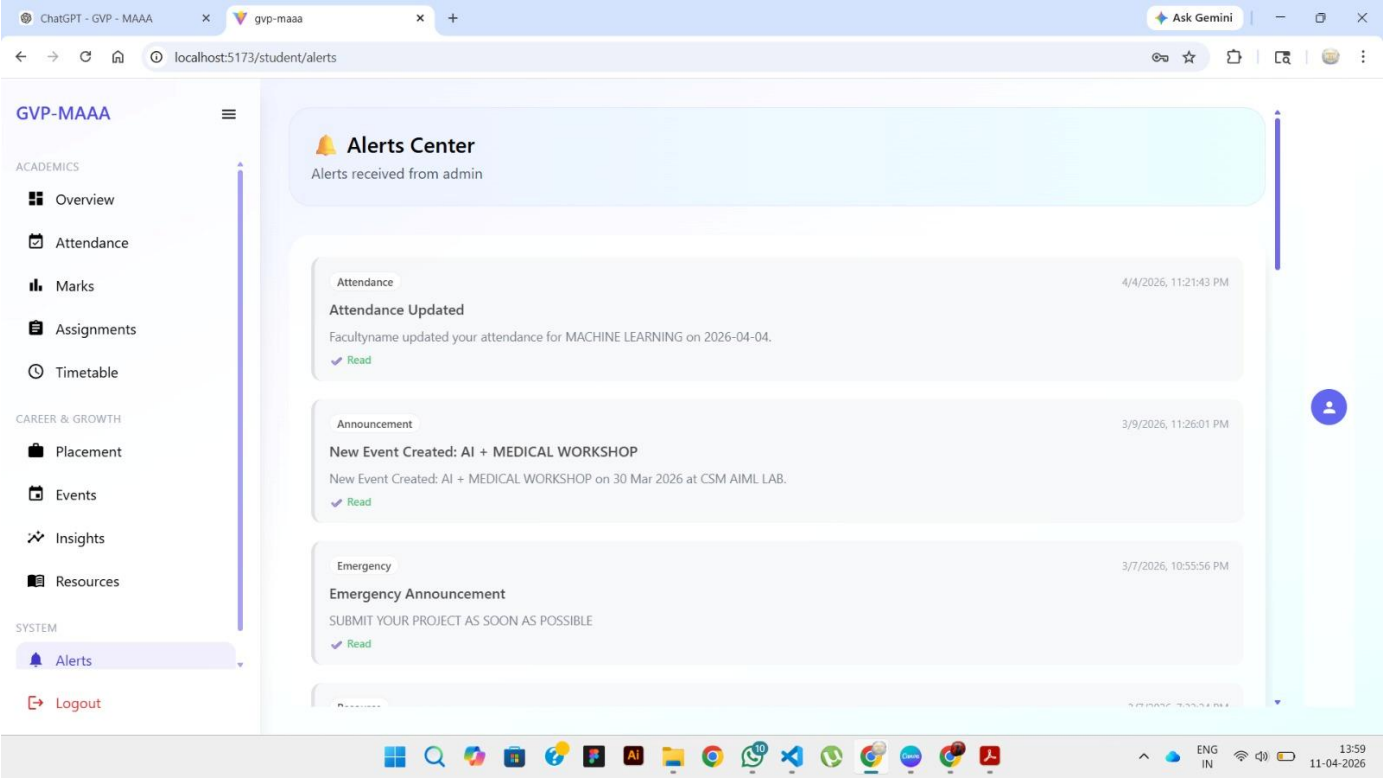


Fig. 7.4.10 Student Alerts Page

Student Logout page:

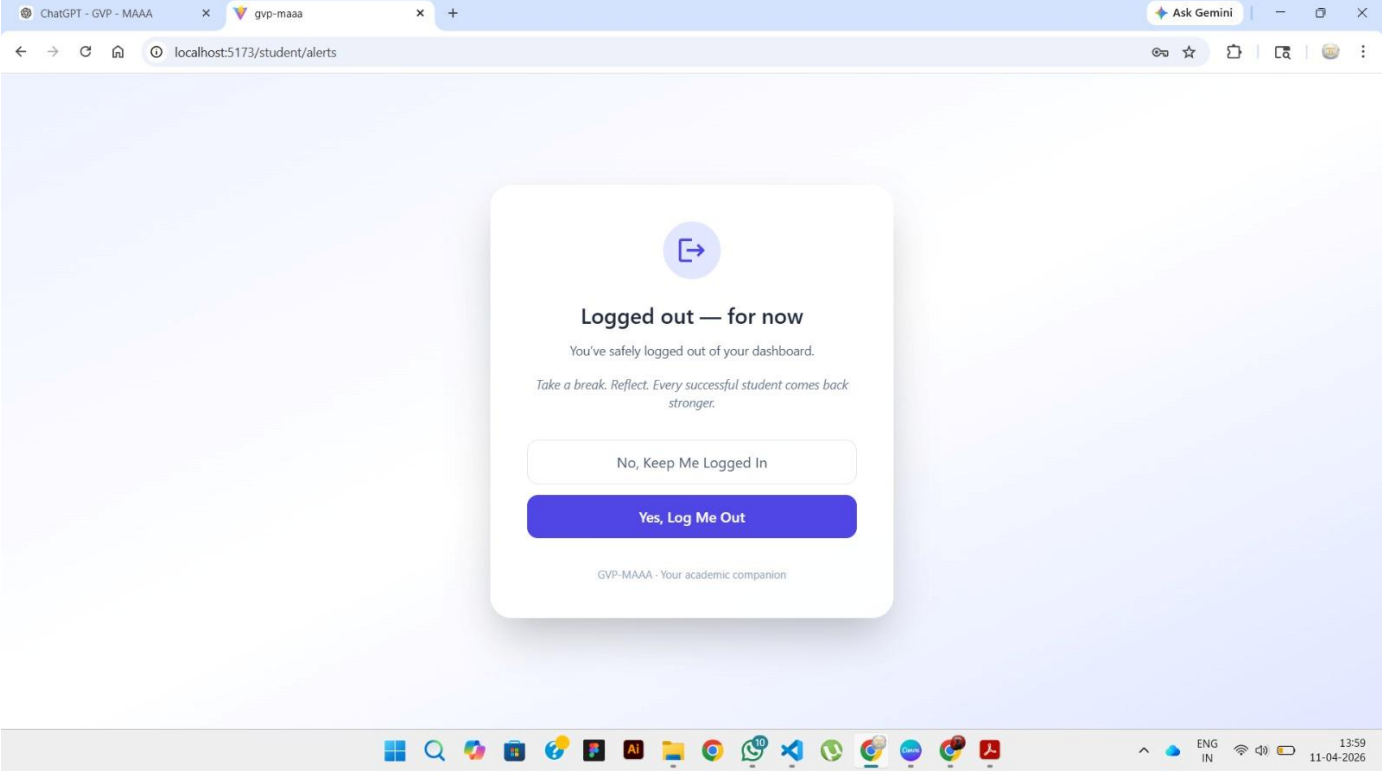


Fig. 7.4.11 Student Logout Page

7.5. Faculty Dashboard
Faculty Overview Page:

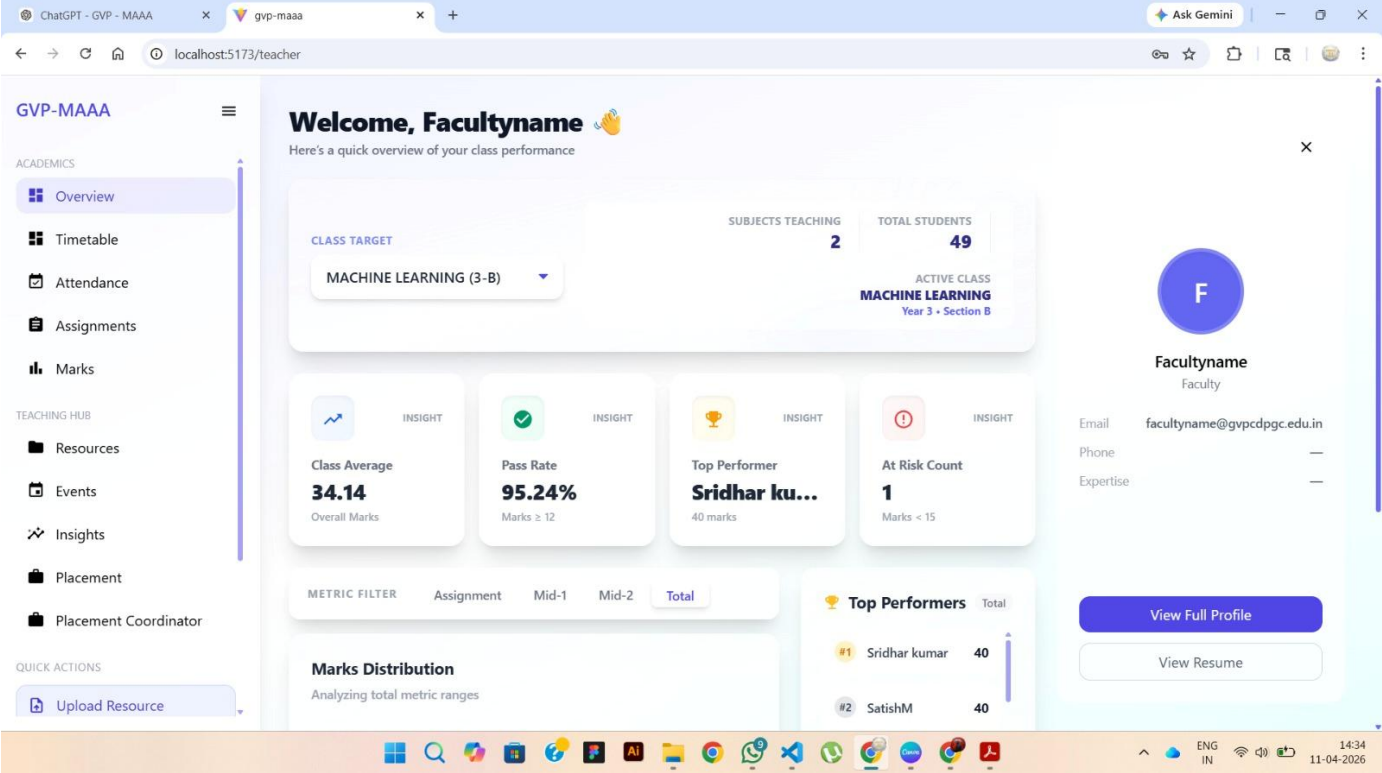


Fig. 7.5.1 Faculty Overview Page

Faculty timetable:

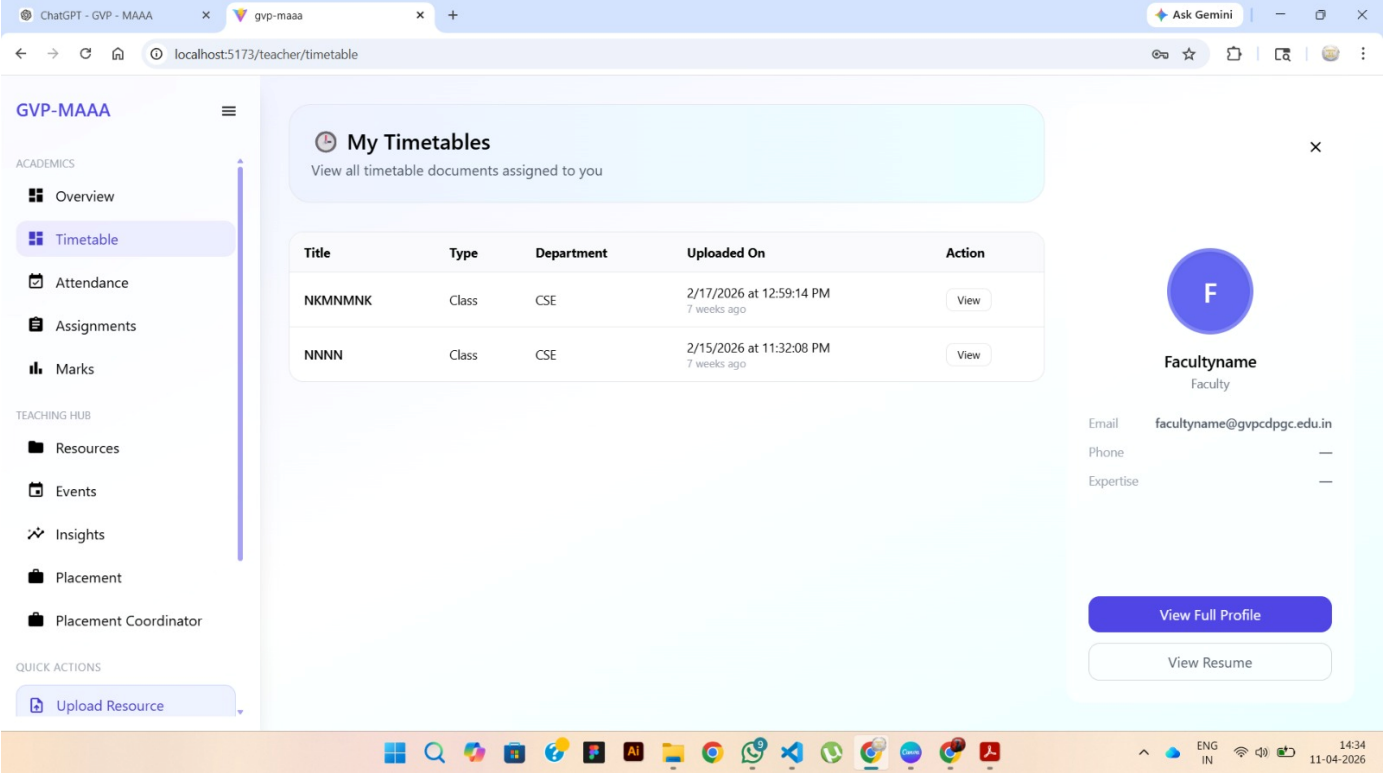


Fig. 7.5.2 Facult Timetable Page

Faculty Attendance:

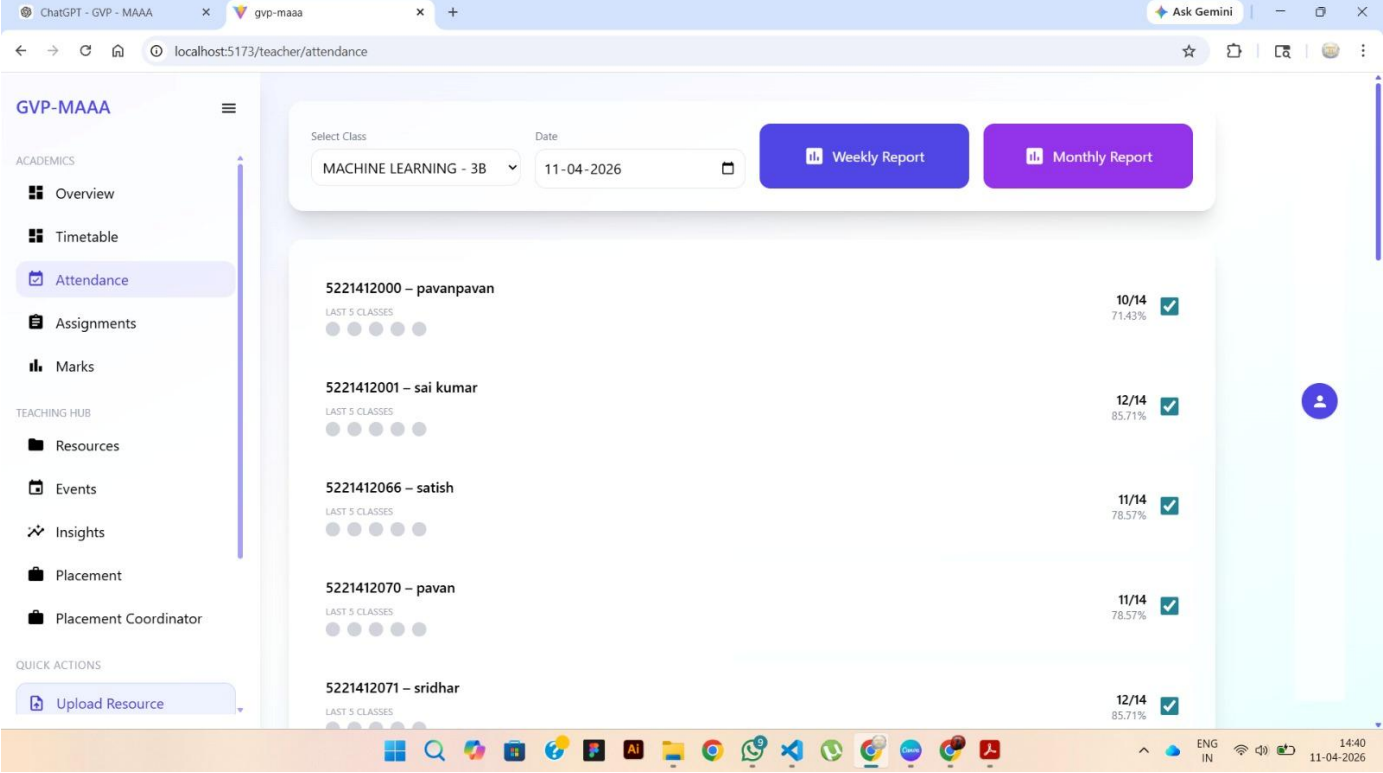


Fig. 7.5.3 Faculty Attendance Page

Faculty Assignments:

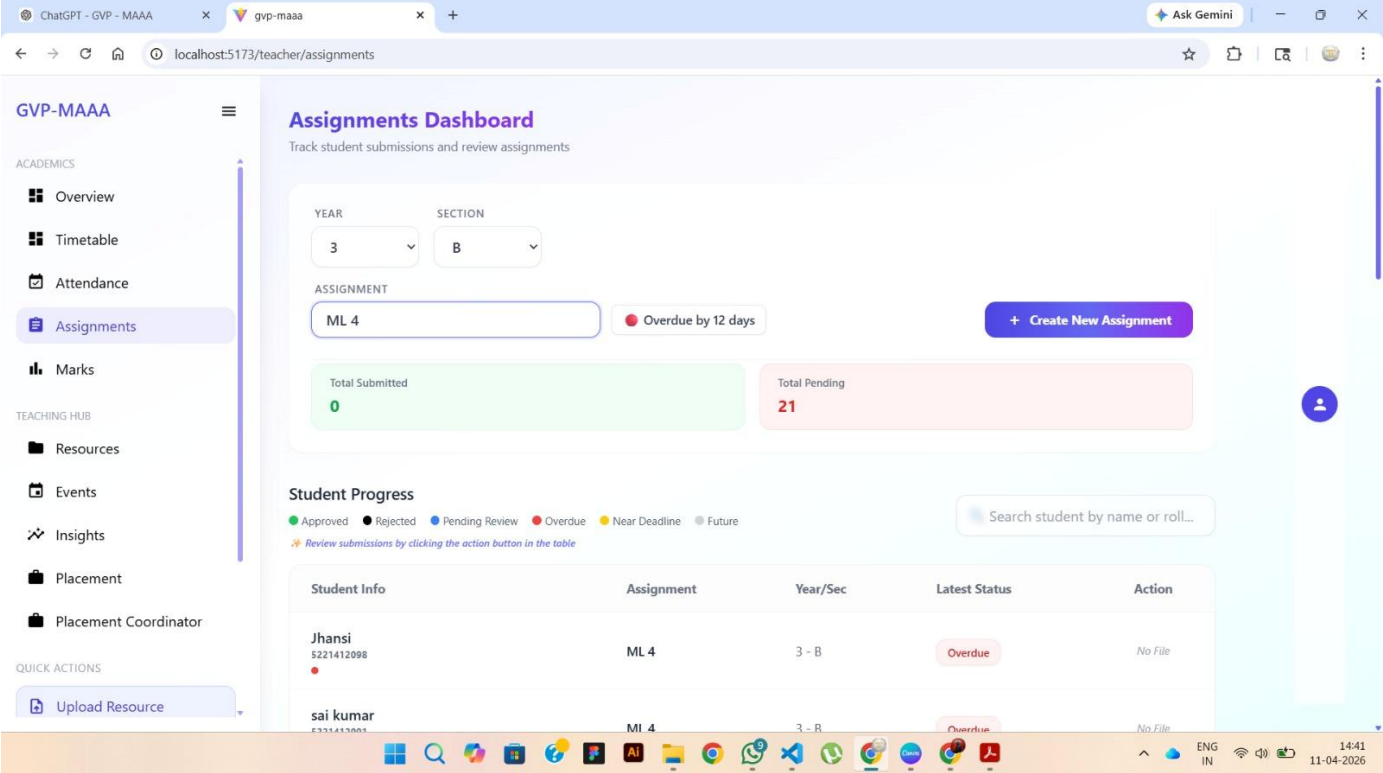


Fig. 7.5.4 Faculty Assignment Page

Faculty Marks:

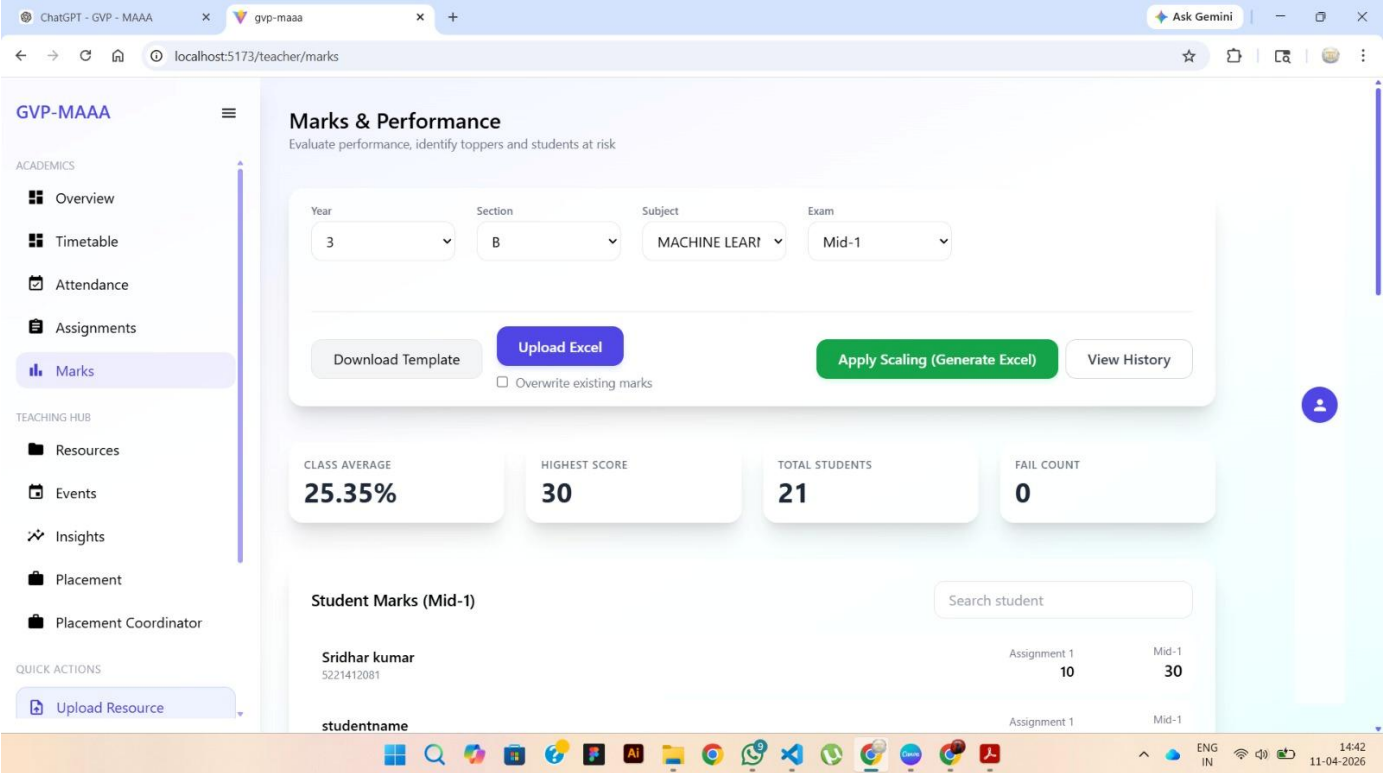


Fig. 7.5.5 Facult Marks Page

Teacher Resources:

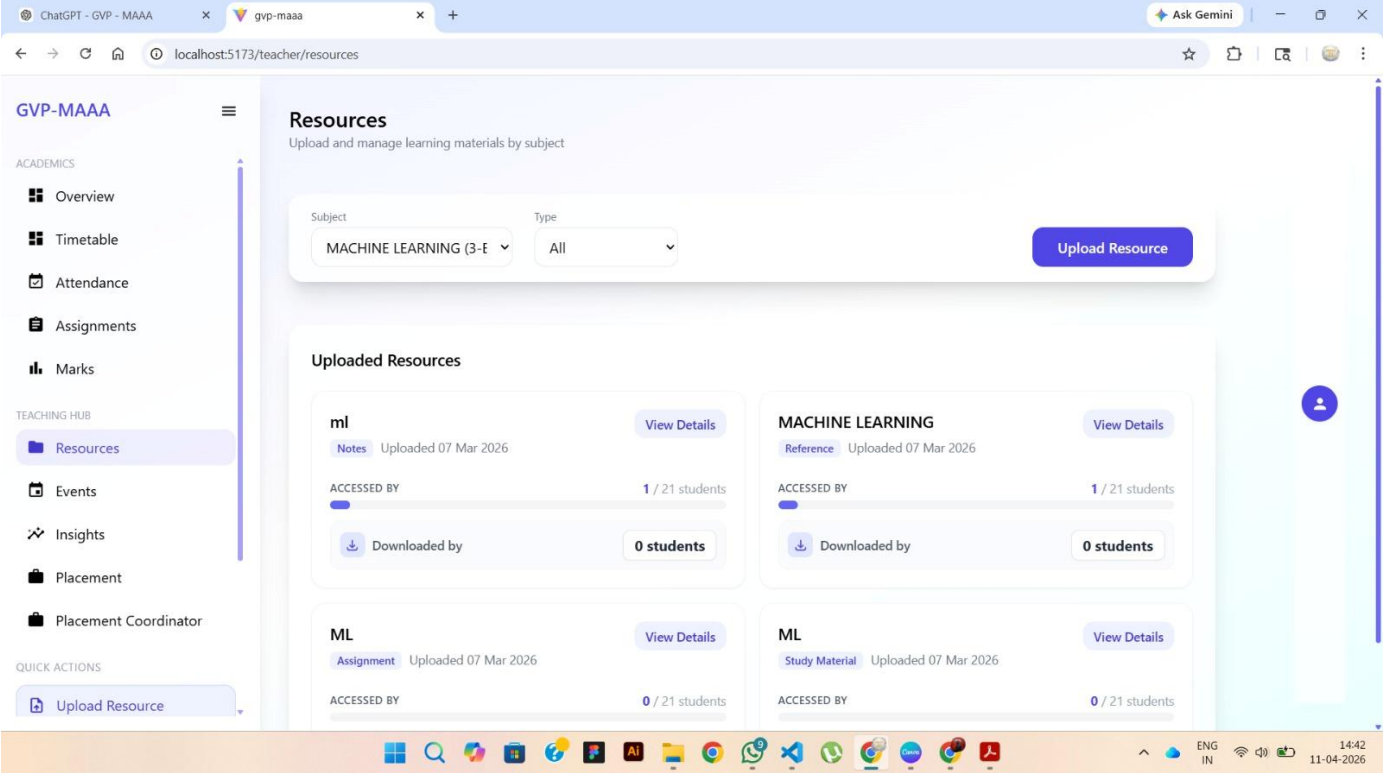


Fig. 7.5.6 Faculty Resource Page

Faculty Events:

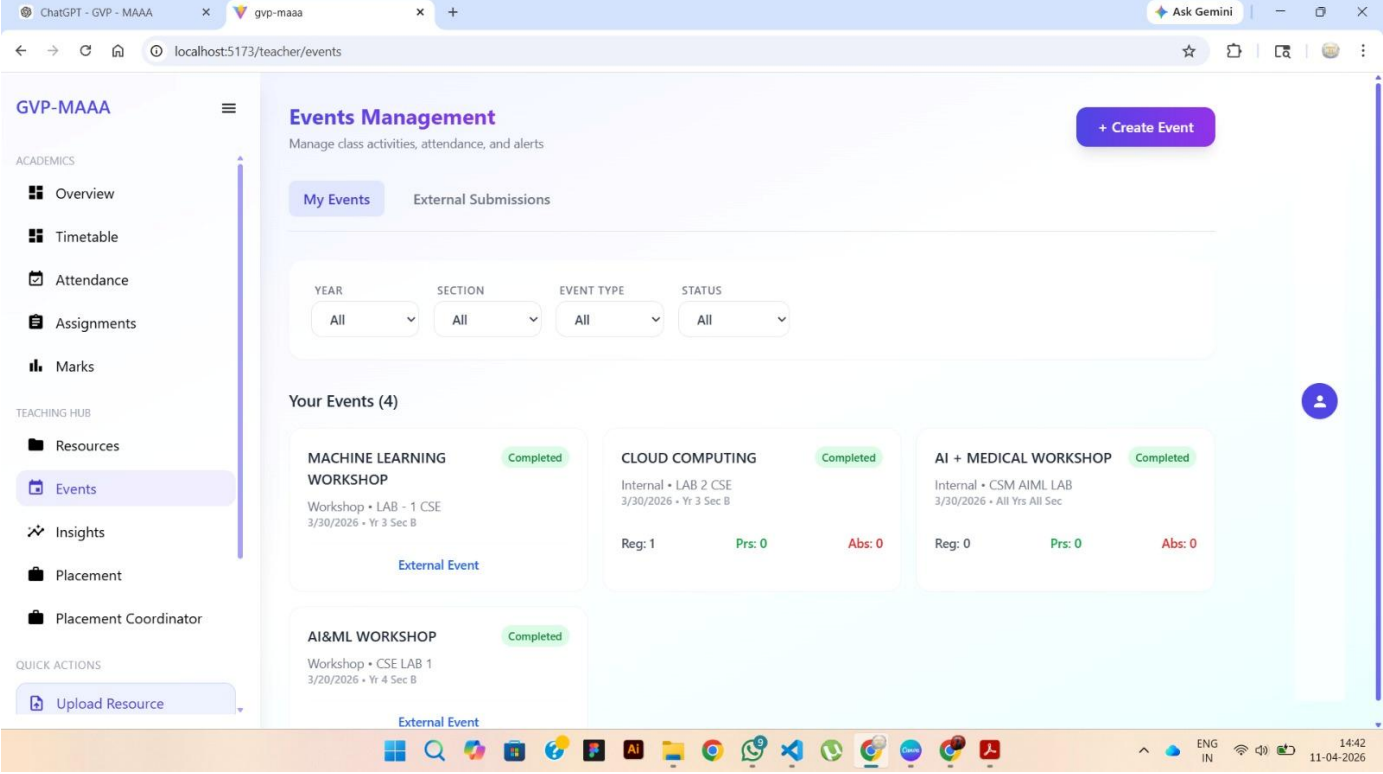


Fig. 7.5.7 Faculty Events Page

Faculty Insights:

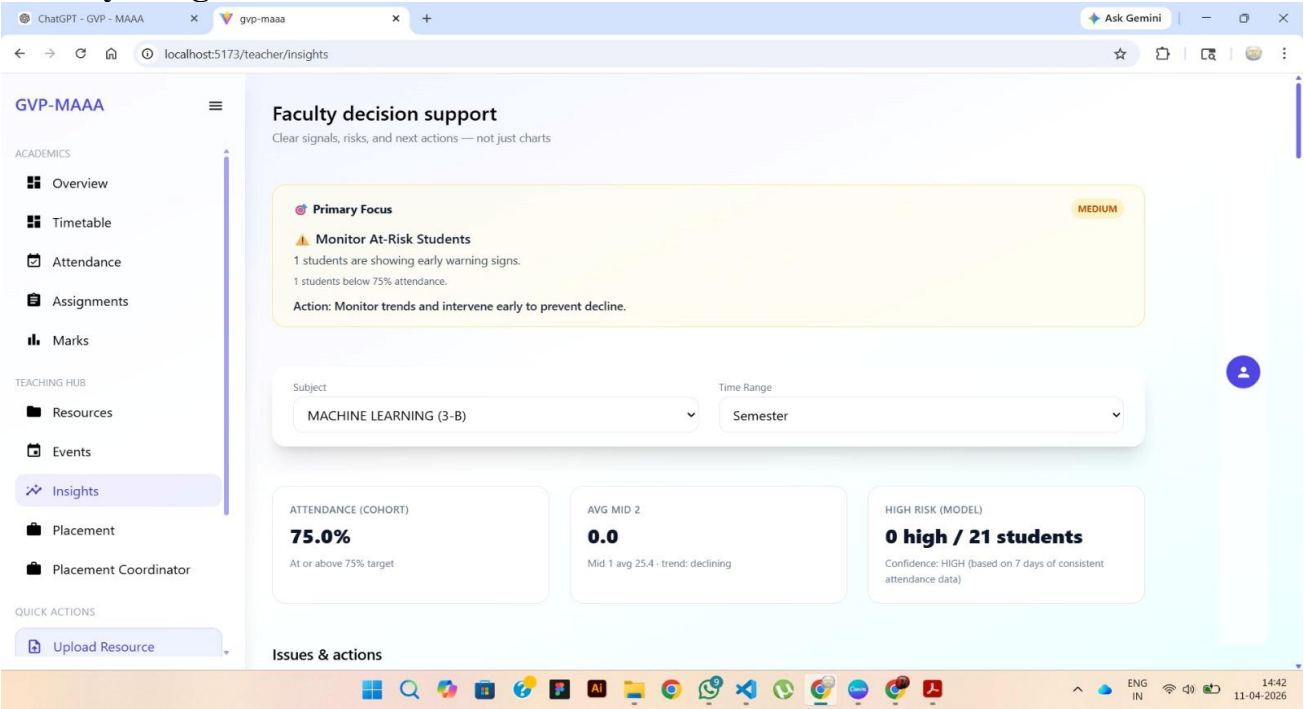


Fig. 7.5.8 Facutly Insights Page

Faculty Placements:

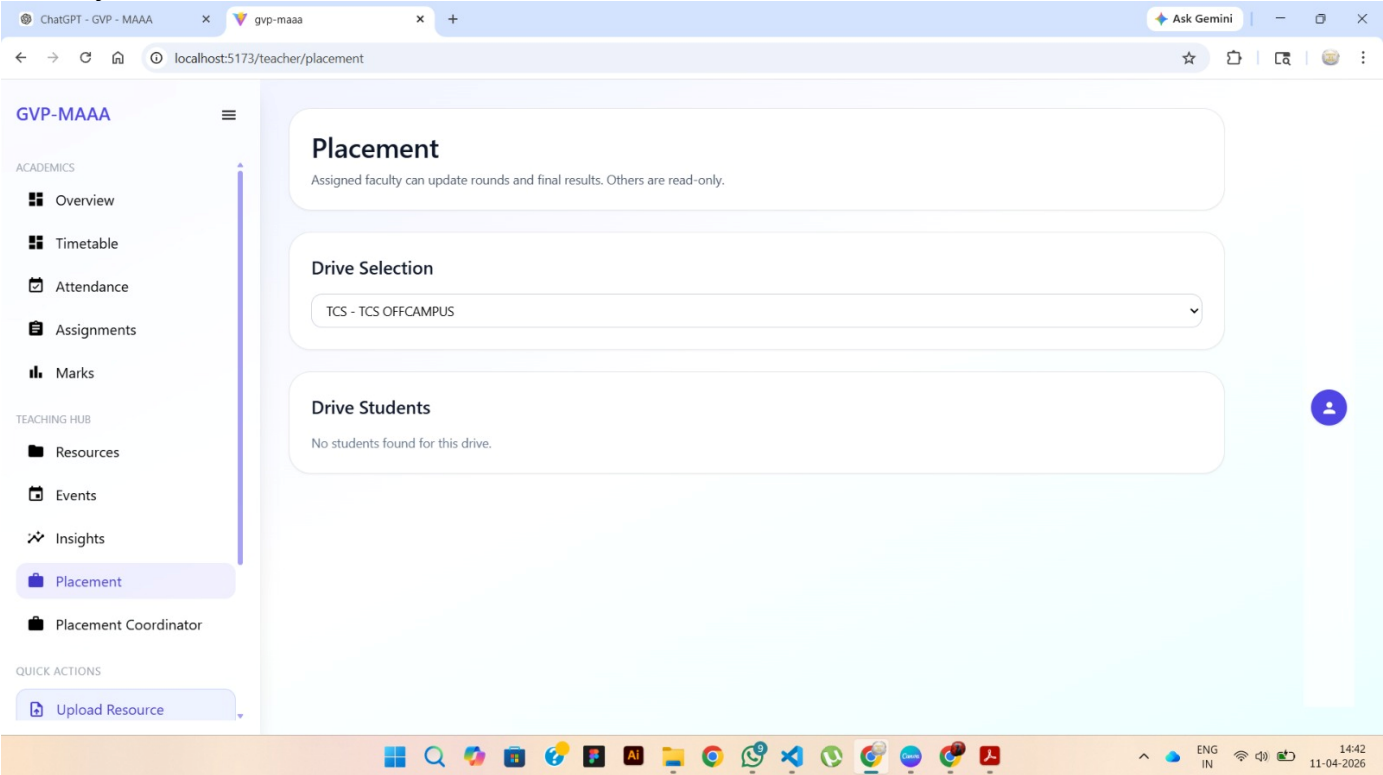


Fig. 7.5.9 Facutly Placement Page

Faculty Placement Coordinator

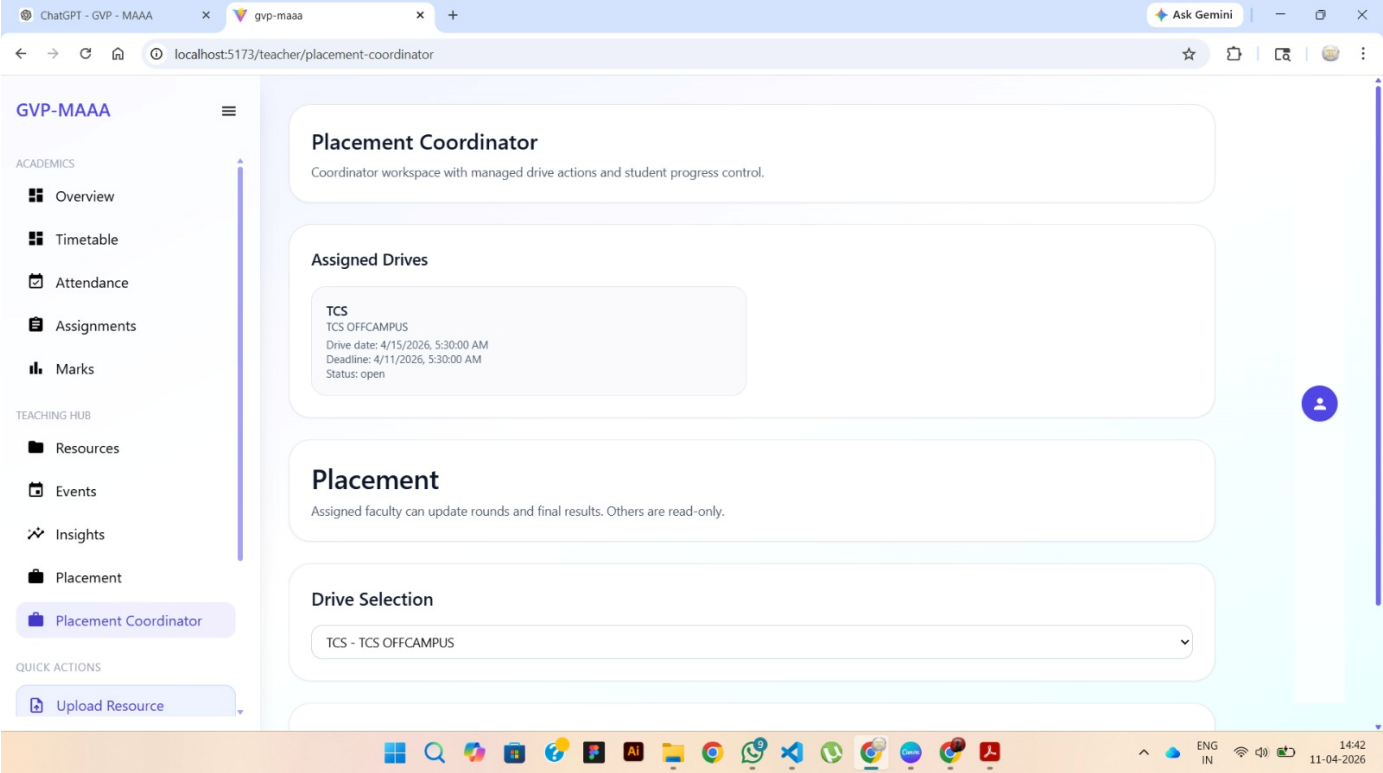


Fig. 7.5.10 Facutly Placement Coordinator Page

Faculty Upload resources:

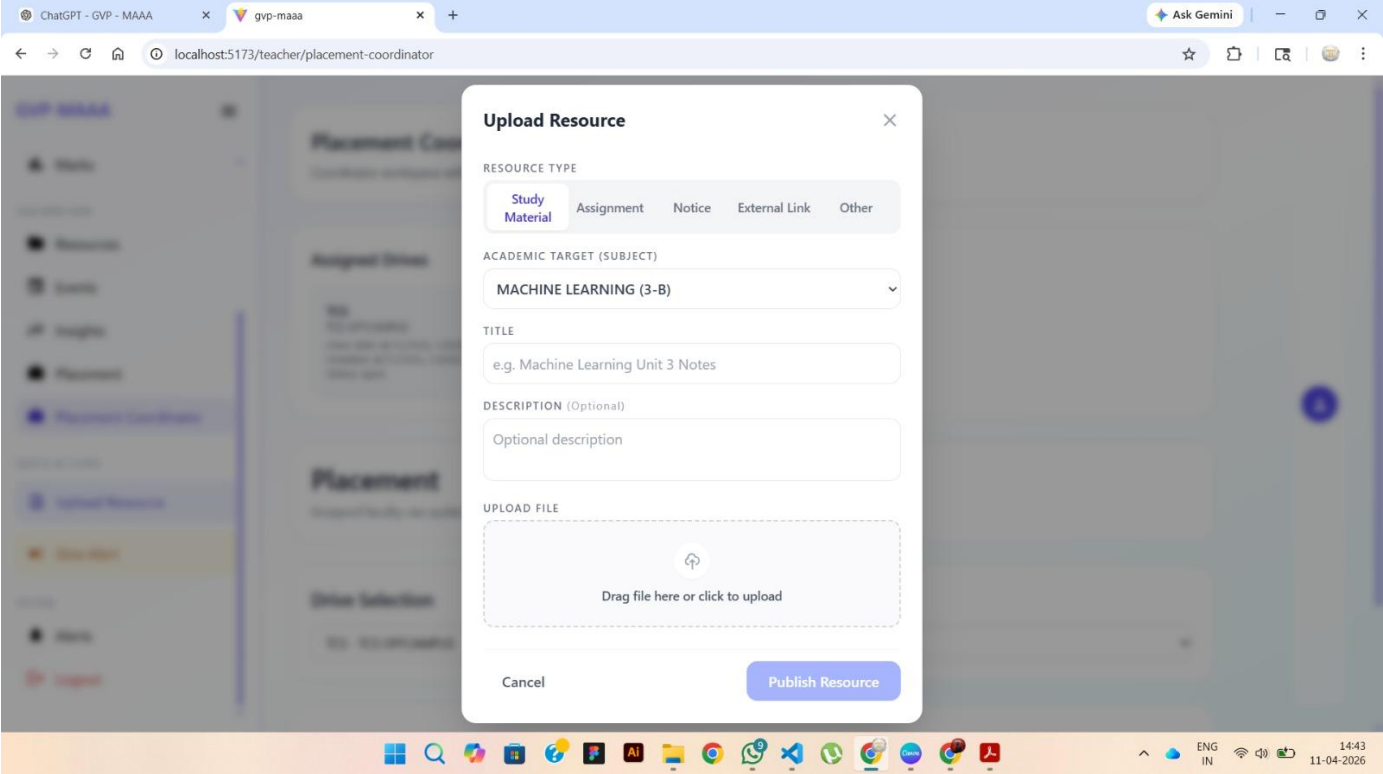


Fig. 7.5.11 Facutly Upload Resource Page

Faculty Send Alerts:

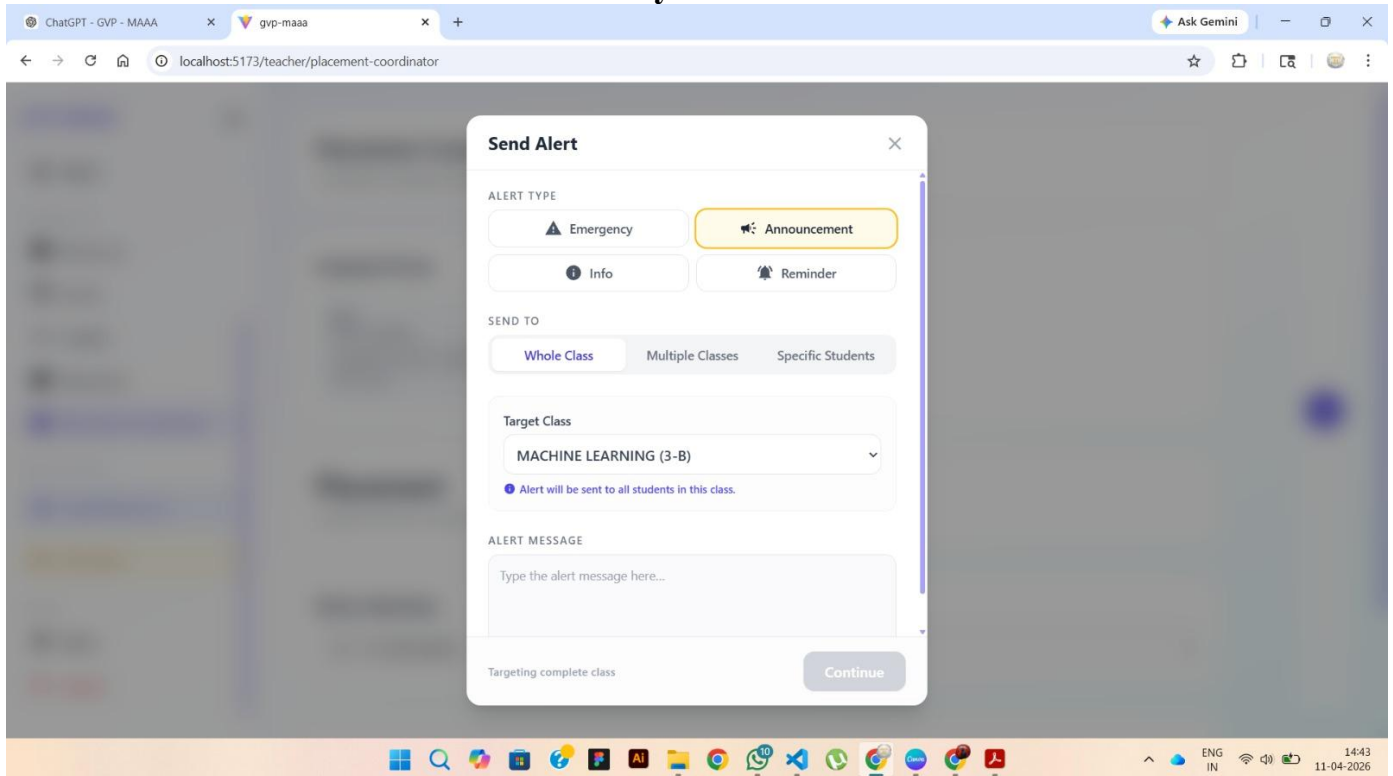


Fig. 7.5.12 Faculty Send Alert Page

Faculty Alerts

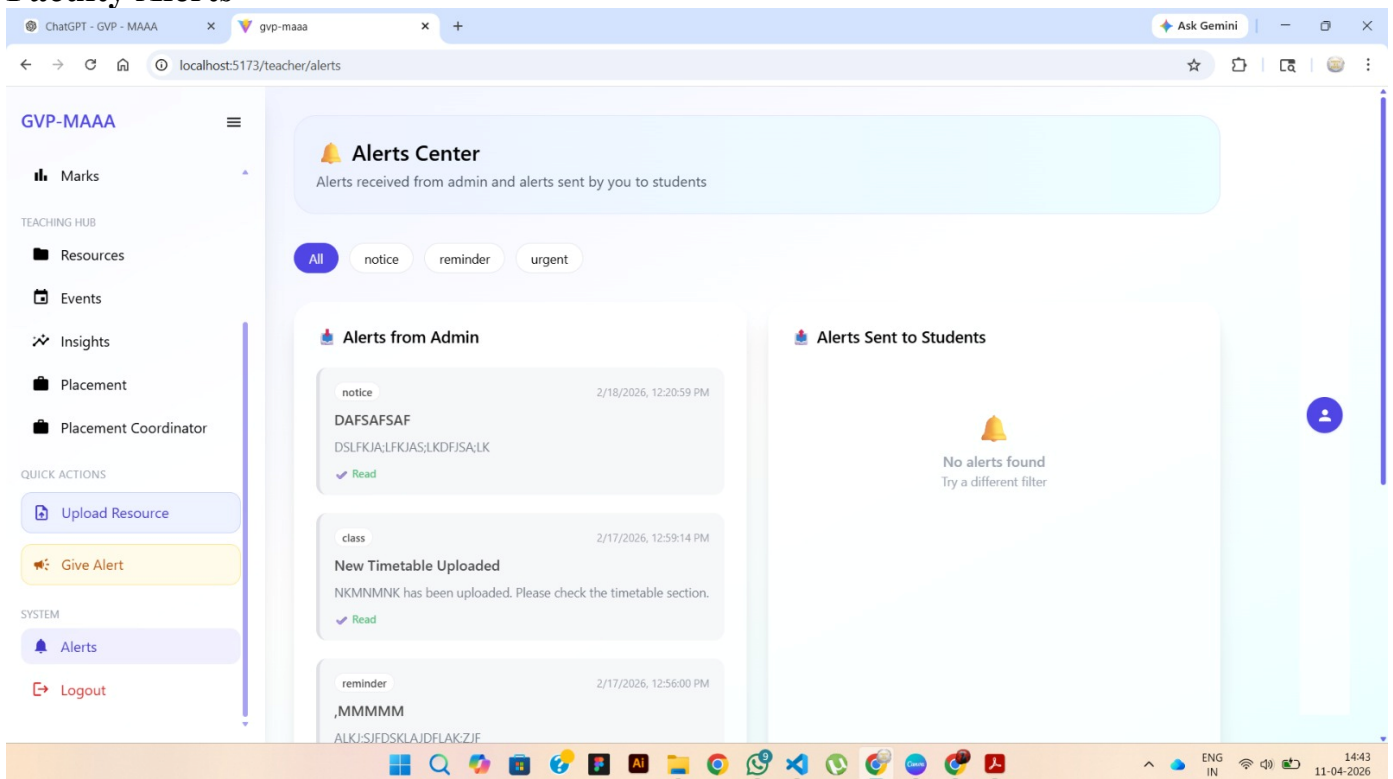


Fig. 7.5.13 Faculty Alerts Page

Teacher logout Page:

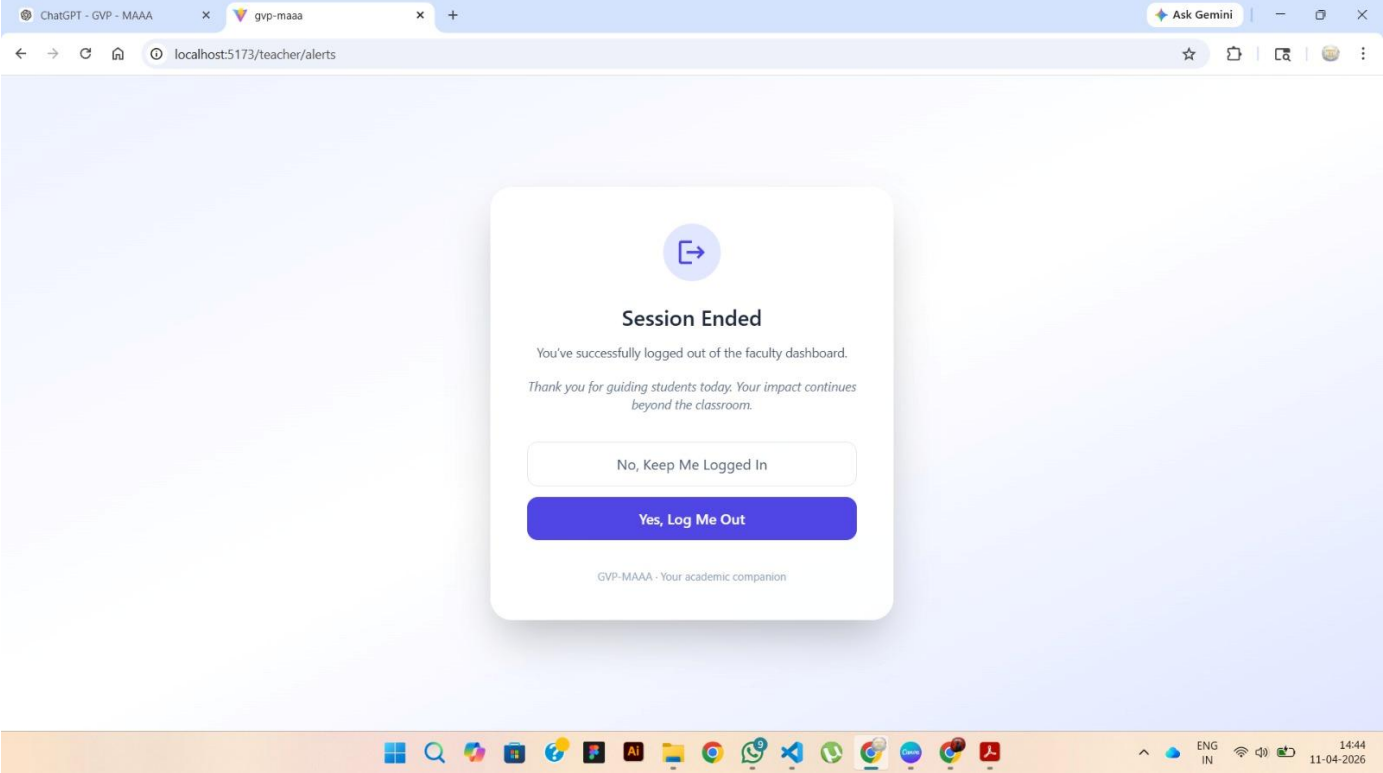


Fig. 7.5.14 Facutly Logout Page

7.6 Admin Dashboard: Admin Overview

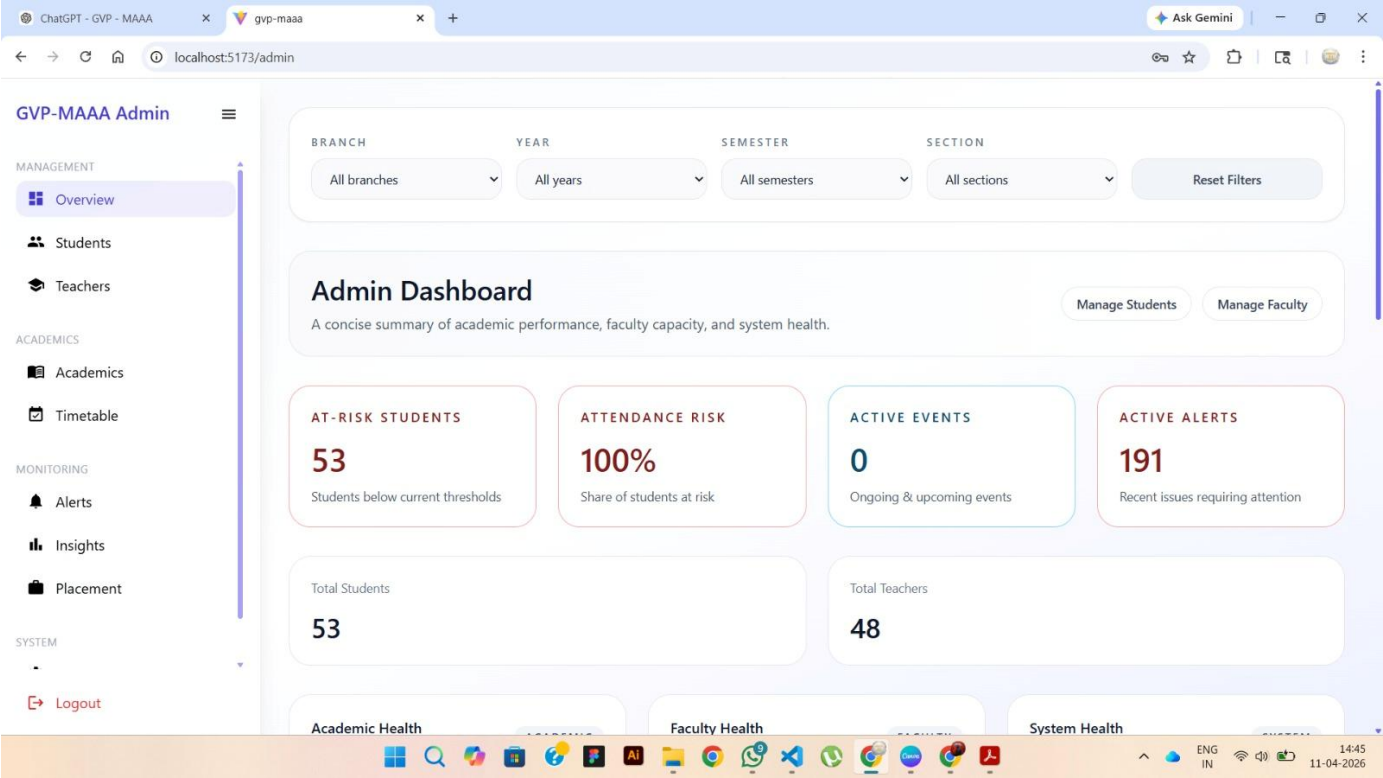


Fig. 7.6.1 Admin Overview Page

Admin-Students :

The screenshot shows the 'Admin-Students' page. The left sidebar has a menu with 'MANAGEMENT' (Overview, Students, Teachers), 'ACADEMICS' (Academics, Timetable), 'MONITORING' (Alerts, Insights, Placement), and 'SYSTEM' (Logout). The main content area is titled 'Student Management' with the subtitle 'Monitor performance, risks, and insights'. It features buttons for 'Update Students', 'Delete Students', 'Notify Students', and 'Download Risk Students Report'. Below these are three dropdown menus set to 'All' and a search bar 'Search by name or roll no'. A table lists 7 students with columns: Roll, Name, Year, Sec, Attendance, CGPA, and Risk. All students have 0% attendance and are marked 'At Risk'.

Roll	Name	Year	Sec	Attendance	CGPA	Risk
21XX1A0506	Student_1	3	A	0%	0	At Risk
21XX1A0507	Student_2	3	A	0%	0	At Risk
21XX1A0508	Student_3	3	A	0%	0	At Risk
21XX1A0509	Student_4	3	A	0%	0	At Risk
21XX1A0510	Student_5	3	A	0%	0	At Risk
21XX1A0511	Student_6	3	A	0%	0	At Risk
21XX1A0512	Student_7	3	A	0%	0	At Risk

Fig. 7.6.2 Admin Students Page

Admin-Teacher :

The screenshot shows the 'Admin-Teacher' page. The left sidebar is identical to the previous page. The main content area is titled 'Teacher Management' with the subtitle 'Academic performance, class health & intervention insights'. It features buttons for 'Update Teachers', 'Delete Teachers', 'Assign Work', and 'Notify Teachers'. Below these are five department buttons: 'All', 'CSE', 'CSM', 'MECH', and 'ECE', along with a search bar 'Search teacher name or designation'. A table lists 5 faculty members with columns: Teacher, Department, Classes, Students, Avg Attendance, and Condition. All faculty are in the 'CIVIL' department, have 0 classes and 0 students, and are in 'Stable' condition.

Teacher	Department	Classes	Students	Avg Attendance	Condition
Faculty_1 Assistant Professor	CIVIL	0	0	NaN%	Stable
Faculty_2 Assistant Professor	CIVIL	0	0	NaN%	Stable
Faculty_3 Assistant Professor	CIVIL	0	0	NaN%	Stable
Faculty_4 Assistant Professor	CIVIL	0	0	NaN%	Stable
Faculty_5 Assistant Professor	CIVIL	0	0	NaN%	Stable

Fig. 7.6.3 Admin Teacher Page

Admin-Timetable :

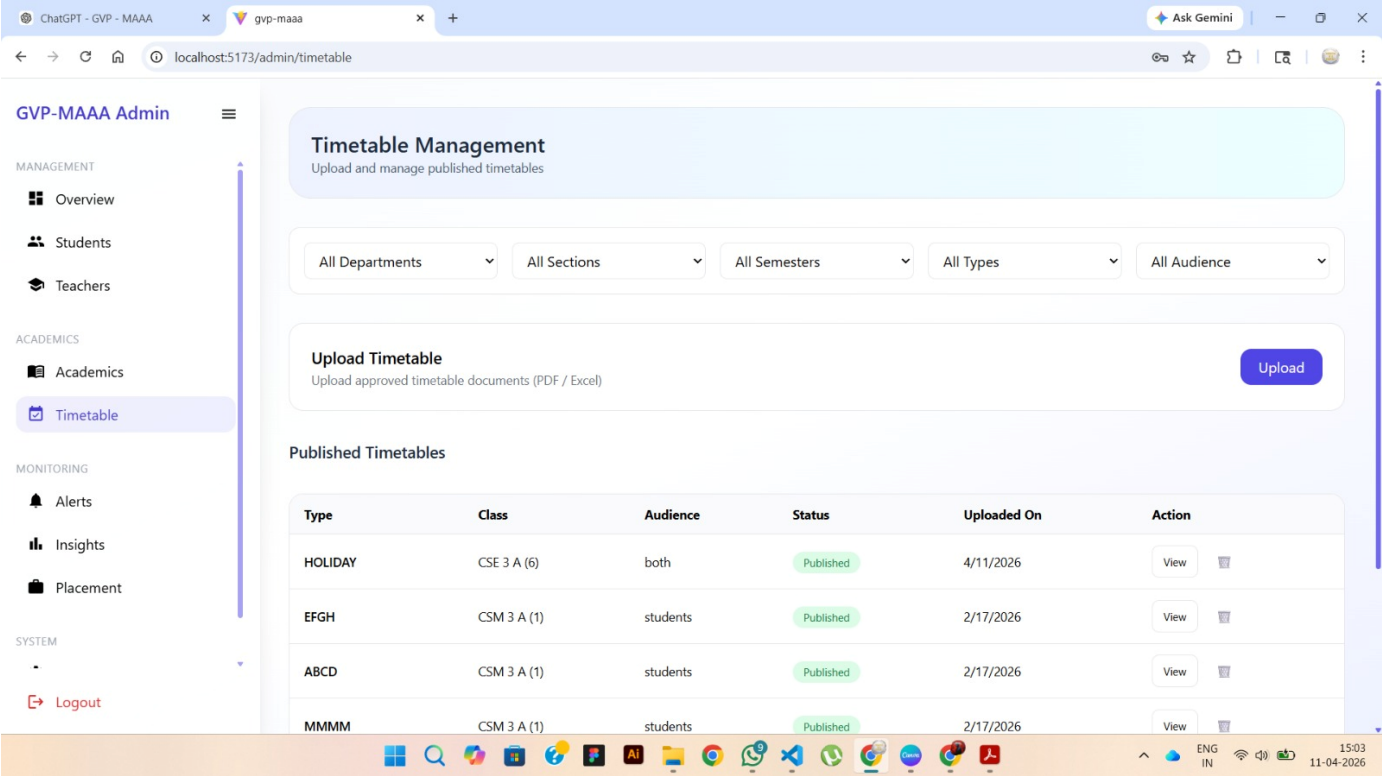


Fig. 7.6.4 Admin Timetable Page

Admin-Alerts:

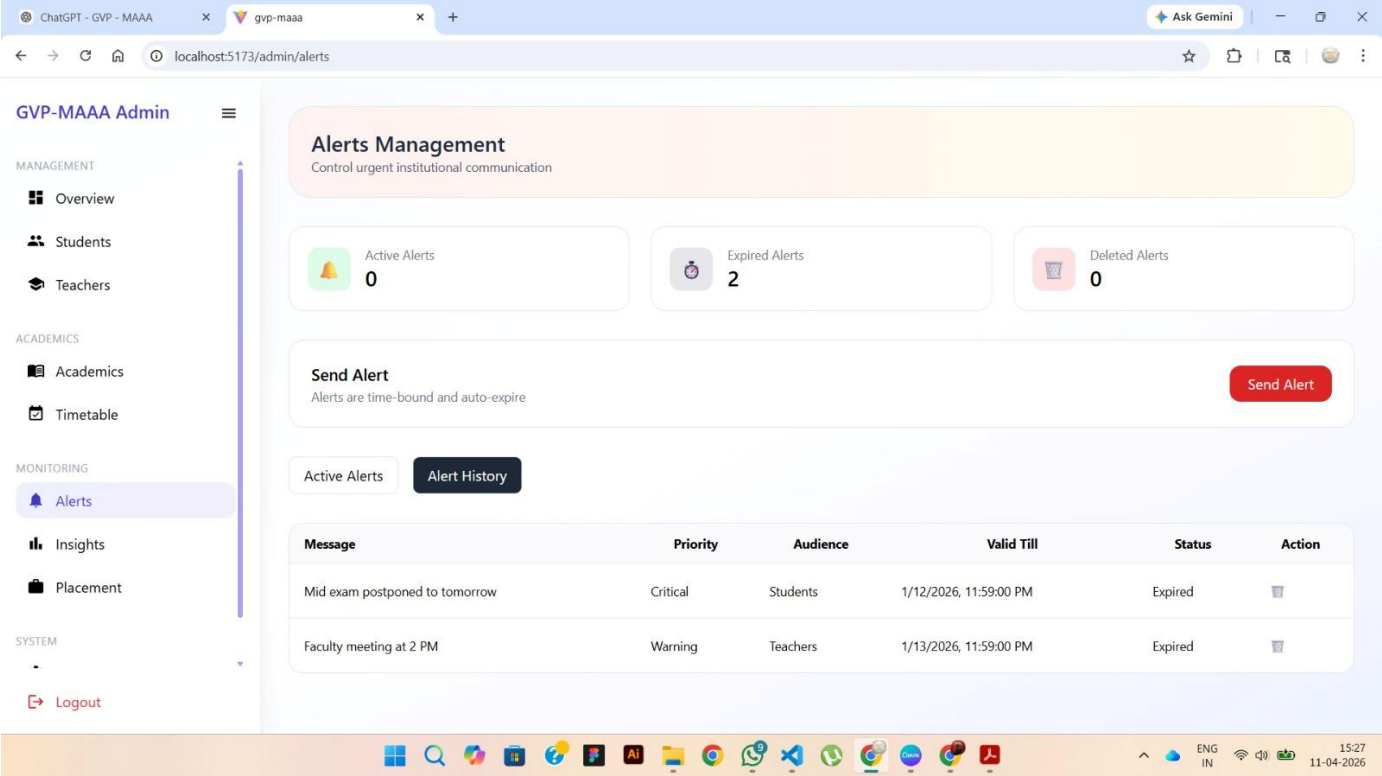


Fig. 7.6.5 Admin Alerts Page

Admin-insights:

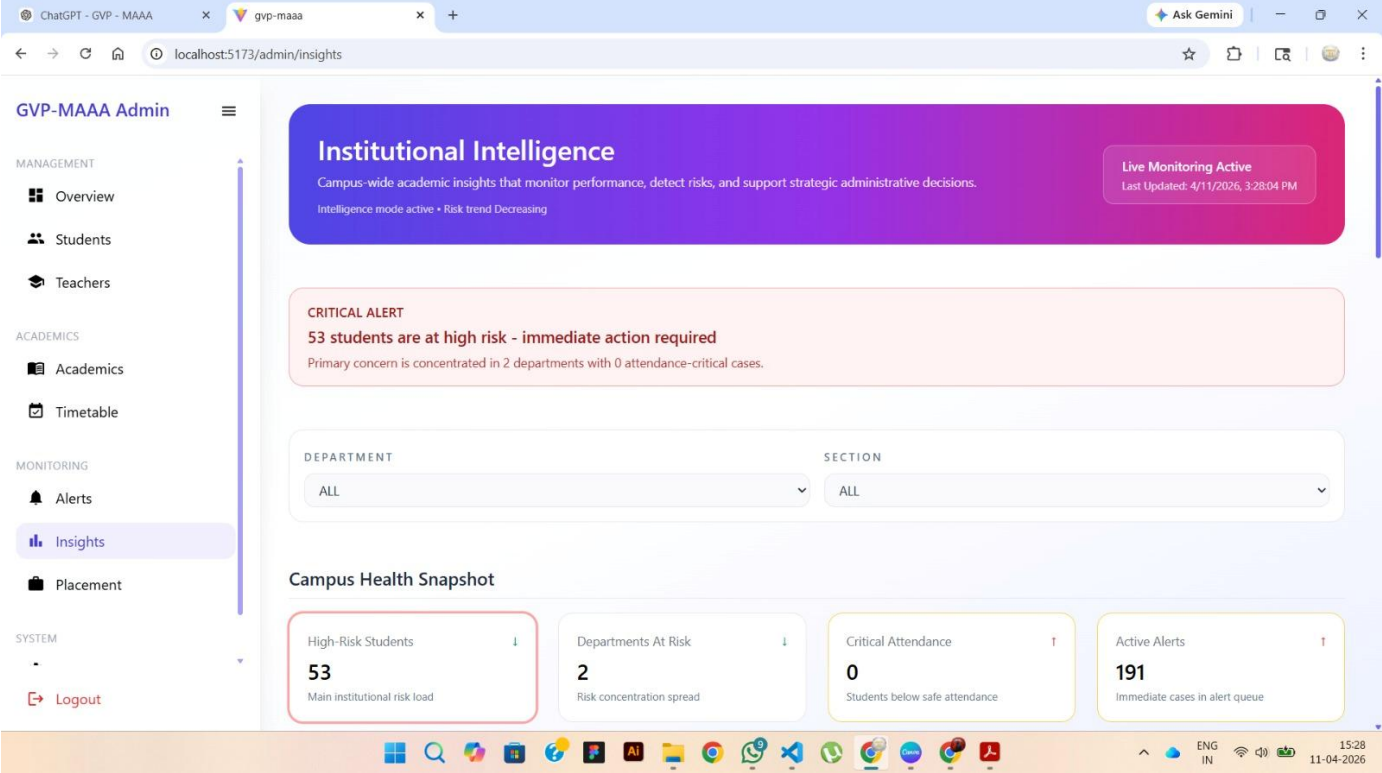


Fig. 7.6.6 Admin Insights Page

Admin-placements:

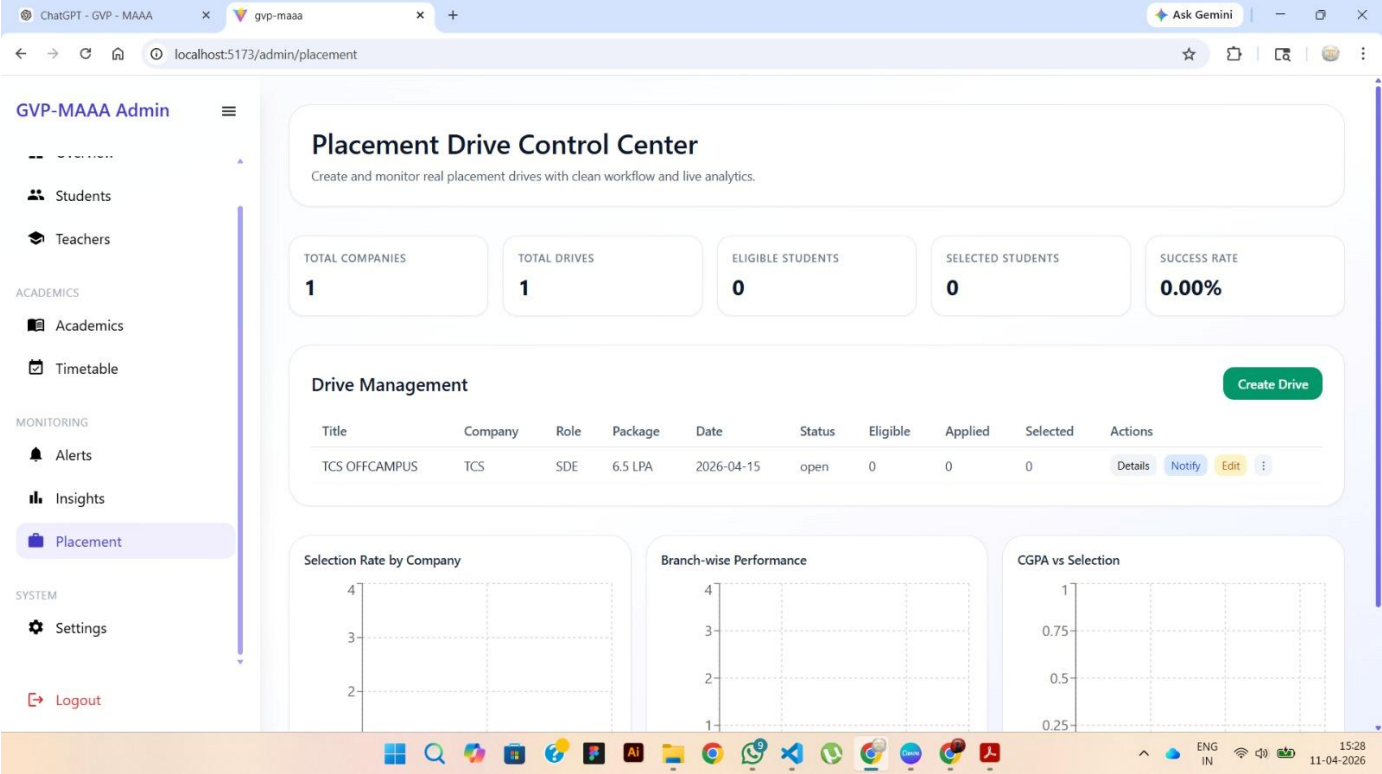


Fig. 7.6.7 Admin Placement Page

Admin Settings:

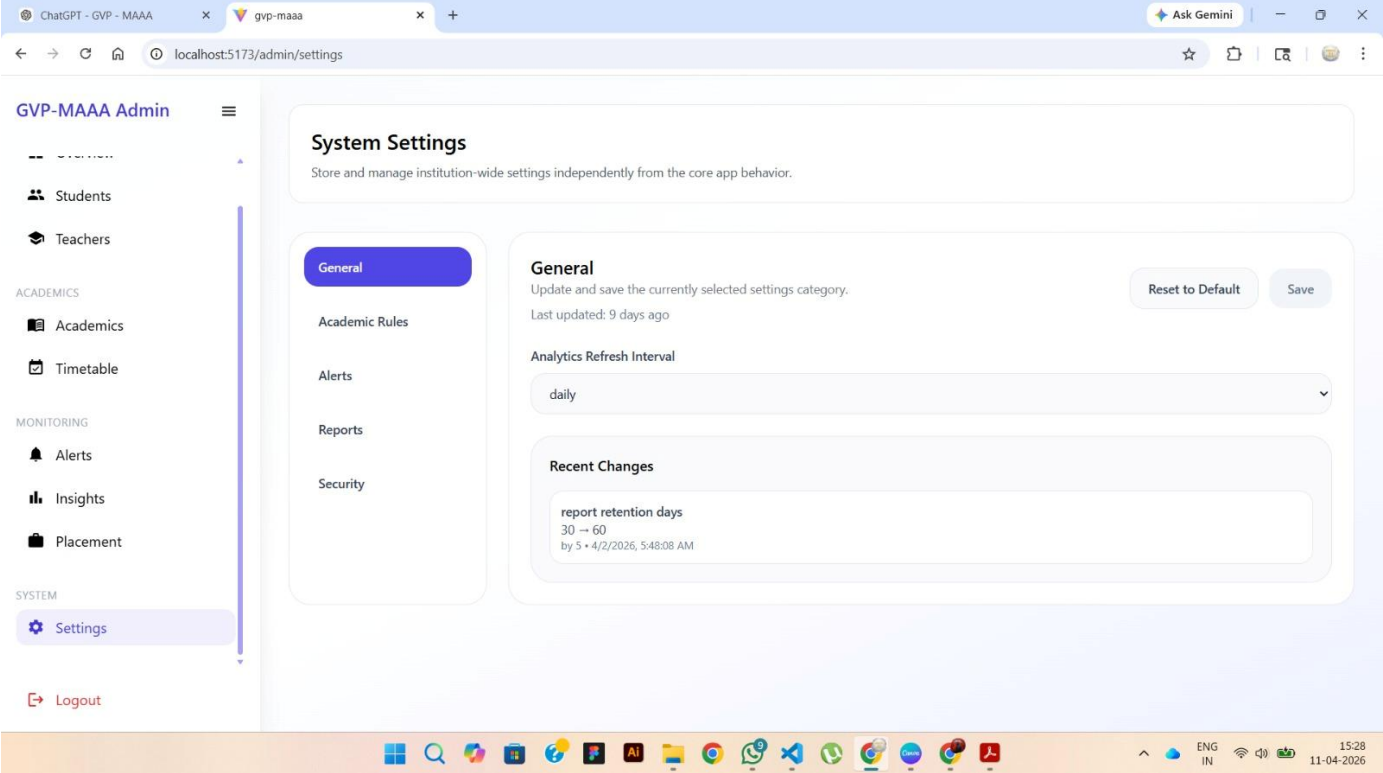


Fig. 7.6.8 Admin Settings Page

Admin-logout :

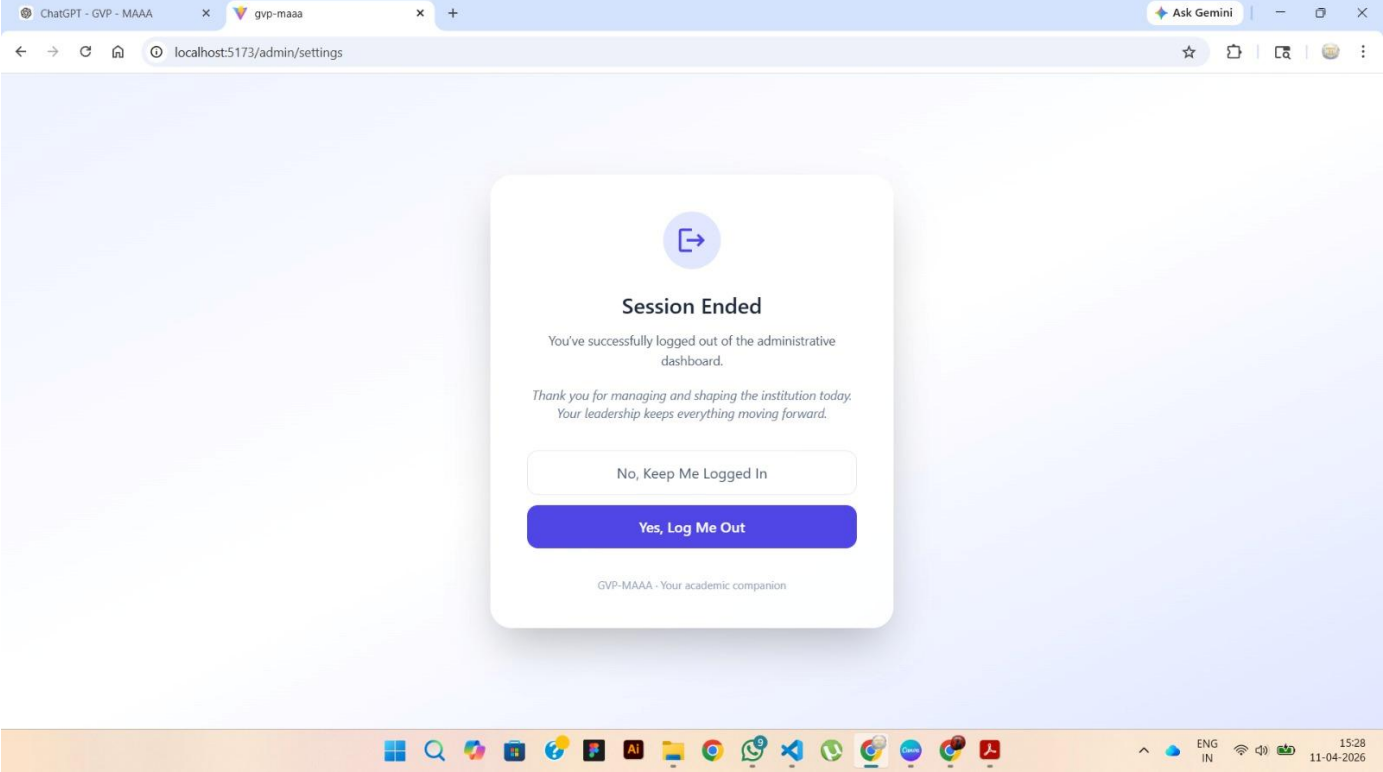


Fig. 7.6.9 Admin Logout Page

8.SAMPLE CODE

8.SAMPLE CODE

8.1 auth.py

```
from datetime import datetime, timedelta

from jose import jwt, JWTError

from fastapi import Depends, HTTPException, status
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials

SECRET_KEY = "supersecretkey" # later move to .env
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 120
RESET_TOKEN_EXPIRE_MINUTES = 15

security = HTTPBearer() # 👉 THIS IS IMPORTANT

def create_access_token(data: dict):
    to_encode = data.copy()
    expire = datetime.utcnow() + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    to_encode.update({"exp": expire})
    encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
    return encoded_jwt

def create_reset_token(email: str):
    expire = datetime.utcnow() + timedelta(minutes=RESET_TOKEN_EXPIRE_MINUTES)

    payload = {
        "sub": email, # subject = user email
        "exp": expire
    }

    token = jwt.encode(payload, SECRET_KEY, algorithm=ALGORITHM)
    return token
```

```

def verify_reset_token(token: str):
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        email: str = payload.get("sub")

        if email is None:
            raise HTTPException(status_code=400, detail="Invalid reset token")

        return email

    except JWTError:
        raise HTTPException(
            status_code=400,
            detail="Reset token is invalid or expired"
        )

def get_current_user(
    credentials: HTTPAuthorizationCredentials = Depends(security)
):
    token = credentials.credentials

    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        return payload
    except JWTError:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid or expired token"
        )

```

8.2 database.py

```
from sqlalchemy import create_engine
from sqlalchemy.orm import declarative_base, sessionmaker

DATABASE_URL = "postgresql://postgres:31012025@127.0.0.1:5432/gvp_maaa"

engine = create_engine(DATABASE_URL, echo=True)

SessionLocal = sessionmaker(
    autocommit=False,
    autoflush=False,
    bind=engine
)

Base = declarative_base()

#  DB session dependency (VERY IMPORTANT)
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

8.3 main.py

```
# pyre-ignore-all-errors
from fastapi.middleware.cors import CORSMiddleware
from fastapi import FastAPI, Depends, HTTPException, UploadFile, File, Form, Body,
Query, Request # type: ignore
from fastapi.middleware.cors import CORSMiddleware # type: ignore
from io import BytesIO
from fastapi.responses import StreamingResponse # type: ignore
```

```

from sqlalchemy.orm import Session, aliased # type: ignore
from sqlalchemy import text, extract, func, or_, case # type: ignore
from sqlalchemy.exc import IntegrityError # type: ignore
from typing import Optional, List, Dict, Any
from security import hash_password, verify_password # type: ignore
from mail import send_reset_email # type: ignore
from schemas import AlertCreate, AssignFacultyRequest, AssignSubjectRequest,
AttendanceCreate, CoordinatorAssignRequest, CoordinatorExtendRequest,
DriveNotifyFilteredRequest, DriveRoleAssignmentRequest, DriveStudentUpdateRequest,
DriveUpdate, FacultyAssignmentUpdateRequest, ResetPasswordRequest,
StudentNotifyRequest, StudentPromotionRequest, TeacherAdminUpdate,
TeacherDeleteRequest, TimetableCreate,
TimetableResponse, StudentDeleteRequest, SubjectCreate, AttendanceAnalyticsResponse,
MarksUpload, AdminOverviewResponse, DriveCreate, PlacementDriveFeedbackCreate
# type: ignore
import schemas # type: ignore
from datetime import datetime, timedelta, date
from database import engine # type: ignore
try:
    from ml.risk_engine import calculate_risk_movement, calculate_risk_score,
get_attendance_trend_label, get_student_attendance_trend, predict_future_risk
    from ml.prediction_engine import forecast_performance
except ImportError:
    def calculate_risk_movement(*args, **kwargs): return "stable"
    def calculate_risk_score(*args, **kwargs): return (0.0, [])
    def get_attendance_trend_label(*args, **kwargs): return "Fluctuating"
    def get_student_attendance_trend(*args, **kwargs): return [None, None, None, None,
None]
    def predict_future_risk(*args, **kwargs): return 0
    def forecast_performance(*args, **kwargs): return 0.0
try:
    from ml.recommendation_engine import generate_recommendations
except ImportError:
    def generate_recommendations(*args, **kwargs): return []

```

```
from models import ( # type: ignore
    Base,
    Department,
    Alert,
    AlertRecipient,
    Timetable,
    Subject,
    FacultySubject,
    Attendance,
    Student,
    User,
    AttendanceWarning,
    FacultyMonthlyAttendanceAlert,
    Assignment,
    AssignmentSubmission,
    Resource,
    ResourceAccess,
    Event,
    EventAttendance,
    EventRegistration,
    ExternalEventSubmission,
    SystemSetting,
    SettingsAuditLog,
    Mark,
    StudentProgress,
    TaskLog,
    PlacementCompany,
    PlacementDrive,
    PlacementFeedback,
    PlacementStudentProfile,
    PlacementStudentSkill,
    DriveAuditLog,
    DriveCoordinatorMap,
    DriveApplication,
```

```

    DriveFacultyMap,
    AssignmentHistory,
    AuditLog,
    StudentDrive,
)
from apscheduler.schedulers.background import BackgroundScheduler # type: ignore
from reportlab.lib import colors # type: ignore
from reportlab.lib.styles import getSampleStyleSheet # type: ignore
from reportlab.lib.units import inch # type: ignore
from reportlab.platypus import Table, TableStyle # type: ignore
from fastapi.responses import FileResponse # type: ignore
from reportlab.platypus import Table, TableStyle, Paragraph, Spacer, Image # type:
ignore
from reportlab.platypus import SimpleDocTemplate # type: ignore
from reportlab.lib.styles import ParagraphStyle # type: ignore
from reportlab.lib.styles import getSampleStyleSheet # type: ignore
from reportlab.lib.pagesizes import A4 # type: ignore
from reportlab.lib import colors # type: ignore
from reportlab.lib.units import inch # type: ignore
from fastapi.responses import FileResponse # type: ignore
from reportlab.pdfbase.pdfmetrics import stringWidth # type: ignore
from reportlab.pdfgen import canvas # type: ignore
from jose import JWTError, jwt # type: ignore
from datetime import date, timedelta
from services.risk_engine import get_student_risk
from services.alert_rules import generate_student_alerts, NO_DATA_MESSAGE
from services.placement_engine import (
    generate_action_plan,
    get_company_eligibility,
    get_interview_insights,
    get_placement_readiness,
    get_selection_probability,
    get_skill_gap,
)

```

```

from services.admin_insights_engine import (
    build_admin_insights,
    get_admin_departments,
    get_admin_overview_payload,
    get_admin_risk_summary,
    get_recent_admin_alerts,
    run_admin_action,
)
import pandas as pd # type: ignore
import os
import json
import shutil
import uuid
import csv

DEPARTMENT_MAP = {
    11: "CSE",
    12: "CSM",
    14: "ECE",
    15: "MECH",
    1: "CIVIL"
}

DEPARTMENTS_CACHE_TTL_SECONDS = 15 * 60

BRANCH_TO_DEPARTMENT = {name: id for id, name in
    DEPARTMENT_MAP.items()}

def apply_student_filters(query, branch: Optional[str], year: Optional[int], semester:
    Optional[int], section: Optional[str]):
    if branch:
        dept_id = BRANCH_TO_DEPARTMENT.get(branch)
        if dept_id is not None:
            query = query.join(User, User.user_id ==
                Student.student_id).filter(User.department_id == dept_id)

```

```

    else:
        try:
            query = query.join(User, User.user_id ==
Student.student_id).filter(User.department_id == int(branch))
        except Exception:
            query = query.filter(Student.section == branch)

    if year is not None:
        query = query.filter(Student.year == year)
    if semester is not None:
        query = query.filter(Student.semester == semester)
    if section:
        query = query.filter(Student.section == section)
    return query

def apply_alert_filters(query, branch: Optional[str], year: Optional[int], semester:
Optional[int], section: Optional[str]):
    if branch or year is not None or semester is not None or section:
        query = query.join(Student, Student.student_id == Alert.student_id)

    if branch:
        dept_id = BRANCH_TO_DEPARTMENT.get(branch)
        if dept_id is not None:
            query = query.join(User, User.user_id ==
Student.student_id).filter(User.department_id == dept_id)
        else:
            query = query.filter(Alert.department == branch)

    if year is not None:
        query = query.filter(Student.year == year)
    if semester is not None:
        query = query.filter(Student.semester == semester)
    if section:
        query = query.filter(Student.section == section)

```

```

    return query
from dotenv import load_dotenv # type: ignore
load_dotenv()
from database import SessionLocal # type: ignore
from schemas import ( # type: ignore
    LoginRequest,
    StudentSignupRequest,
    TeacherSignupRequest,
    AdminLoginRequest,
    StudentProfileUpdate,
    FacultyProfileUpdate,
    AssignmentCreate,
    AssignmentResponse,
    AssignmentSubmissionCreate,
    AssignmentSubmissionResponse,
    AssignmentDetailResponse,
    StatusUpdateRequest,
    StudentAssignmentSummaryResponse,
    ResourceResponse,
    ResourceAccessRequest,
    EventCreate,
    EventResponse,
    EventAttendanceResponse,
    EventAttendanceUpdate,
    BulkEventAttendanceUpdate,
    EventAlertRequest,
    EventResultUpdate,
    EventRegistrationRequest,
    StudentEventResponse,
    ExternalEventSubmissionCreate,
    ExternalEventSubmissionResponse,
    FacultyExternalSubmissionDetail
)
from models import User, Student, Faculty, Timetable # type: ignore

```

```

from auth import ( # type: ignore
    create_access_token,
    create_reset_token,
    verify_reset_token,
    get_current_user,
    SECRET_KEY,
    ALGORITHM # ✅ ADD THIS
)

```

8.4 Models.py

```

from datetime import datetime
from sqlalchemy import Column, Integer, String, Numeric, ForeignKey, Text, Boolean,
    DateTime, Date, UniqueConstraint, JSON, Float
from sqlalchemy.dialects.postgresql import ARRAY
from sqlalchemy.sql import func
from sqlalchemy.orm import relationship

```

```

from database import Base

```

```

# -----

```

```

# USER (BASE TABLE)

```

```

# -----

```

```

class User(Base):

```

```

    __tablename__ = "users"

```

```

    user_id = Column(Integer, primary_key=True, index=True)

```

```

    name = Column(String, nullable=False)

```

```

    email = Column(String, unique=True, nullable=False)

```

```

    password = Column(String, nullable=False)

```

```

    role = Column(String, nullable=False) # student | faculty | admin

```

```

    department_id = Column(Integer, nullable=False)

```

```

# ✅ NEW (SOFT DELETE)

```

```

is_deleted = Column(Boolean, default=False)
deleted_at = Column(DateTime, nullable=True)

class Department(Base):
    __tablename__ = "departments"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String(100), unique=True, nullable=False)
# -----
# STUDENT (EXTENSION)
# -----
class Student(Base):
    __tablename__ = "students"

    student_id = Column(
        Integer,
        ForeignKey("users.user_id", ondelete="CASCADE"),
        primary_key=True
    )
    roll_no = Column(String, unique=True, nullable=True)
    joining_year = Column(Integer, nullable=False)
    year = Column(Integer, default=1)
    semester = Column(Integer, default=1)
    section = Column(String, nullable=True)
    cgpa = Column(Numeric(3, 2), default=0.00)

# 📌 NEW PROFILE FIELDS
phone = Column(String, nullable=True)
skills = Column(String, nullable=True) # comma separated
certificates = Column(String, nullable=True) # comma separated
linkedin = Column(String, nullable=True)
github = Column(String, nullable=True)
portfolio = Column(String, nullable=True)
bio = Column(String, nullable=True)

```

```

intervention_status = Column(String, nullable=True, default="none")
intervention_type = Column(String, nullable=True)
intervention_last_updated = Column(DateTime, nullable=True)
previous_risk_score = Column(Numeric(5, 2), nullable=True)

#  NEW (SOFT DELETE)
is_deleted = Column(Boolean, default=False)
deleted_at = Column(DateTime, nullable=True)

# -----
# PLACEMENT INTELLIGENCE
# -----
class PlacementStudentProfile(Base):
    __tablename__ = "student_profile"

    student_id = Column(
        Integer,
        ForeignKey("students.student_id", ondelete="CASCADE"),
        primary_key=True,
    )
    cgpa = Column(Numeric(3, 2), nullable=True)
    backlogs = Column(Integer, default=0)
    department = Column(String(50), nullable=True)
    year = Column(Integer, nullable=True)
    updated_at = Column(DateTime(timezone=True), server_default=func.now(),
        onupdate=func.now())

class PlacementInterview(Base):
    __tablename__ = "interviews"

    id = Column(Integer, primary_key=True, index=True)
    student_id = Column(Integer, ForeignKey("students.student_id", ondelete="CASCADE"),
        nullable=False)
    company_name = Column(String(255), nullable=False)

```

```

date = Column(Date, nullable=True)
mode = Column(String(20), nullable=True)
status = Column(String(20), nullable=True)
result = Column(String(20), nullable=True)
round_reached = Column(String(100), nullable=True)
feedback = Column(Text, nullable=True)
weak_area = Column(String(100), nullable=True)
created_at = Column(DateTime(timezone=True), server_default=func.now())

```

```

class PlacementCompany(Base):

```

```

    __tablename__ = "companies"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String(255), unique=True, nullable=False)
    role = Column(String(255), nullable=True)
    package_lpa = Column(Numeric(10, 2), nullable=True)
    min_cgpa = Column(Numeric(3, 2), nullable=False)
    max_backlogs = Column(Integer, default=0)
    branches = Column(ARRAY(String), nullable=True)
    selection_process = Column(ARRAY(String), nullable=True)
    role_type = Column(String(50), nullable=True)
    required_skills = Column(JSON, default=list)
    created_at = Column(DateTime(timezone=True), server_default=func.now())
    updated_at = Column(DateTime(timezone=True), server_default=func.now(),
        onupdate=func.now())

```

```

class PlacementStudentSkill(Base):

```

```

    __tablename__ = "student_skills"

    id = Column(Integer, primary_key=True, index=True)
    student_id = Column(Integer, ForeignKey("students.student_id", ondelete="CASCADE"),
        nullable=False)
    skill_name = Column(String(120), nullable=False)
    level = Column(String(20), nullable=True)

```

```
updated_at = Column(DateTime(timezone=True), server_default=func.now(),
    onupdate=func.now())
```

```
__table_args__ = (
    UniqueConstraint("student_id", "skill_name", name="unique_student_skill"),
)
```

```
class PlacementProgress(Base):
```

```
    __tablename__ = "placement_progress"
```

```
    student_id = Column(Integer, ForeignKey("students.student_id", ondelete="CASCADE"),
        primary_key=True)
```

```
    readiness_score = Column(Numeric(5, 2), nullable=True)
```

```
    last_updated = Column(DateTime(timezone=True), server_default=func.now(),
        onupdate=func.now())
```

```
class PlacementDrive(Base):
```

```
    __tablename__ = "placement_drives"
```

```
    id = Column(Integer, primary_key=True, index=True)
```

```
    title = Column(String(255), nullable=True)
```

```
    company_id = Column(Integer, ForeignKey("companies.id", ondelete="CASCADE"),
        nullable=True)
```

```
    company_name = Column(String(255), nullable=True)
```

```
    role = Column(String(255), nullable=True)
```

```
    package_lpa = Column(Numeric(10, 2), nullable=True)
```

```
    min_cgpa = Column(Numeric(3, 2), nullable=True)
```

```
    max_backlogs = Column(Integer, default=0)
```

```
    selection_process = Column(ARRAY(String), nullable=True)
```

```
    drive_date = Column(Date, nullable=True)
```

```
    mode = Column(String(30), nullable=True)
```

```
    location = Column(String(255), nullable=True)
```

```
    registration_deadline = Column(Date, nullable=True)
```

```
    apply_link = Column(Text, nullable=True)
```

```

details_pdf = Column(Text, nullable=True)
eligible_years = Column(ARRAY(Integer), nullable=True)
status = Column(String(30), default="open")
branches = Column(ARRAY(String), nullable=True)
created_by = Column(Integer, ForeignKey("users.user_id"), nullable=True)
closed_at = Column(DateTime(timezone=True), nullable=True)
created_at = Column(DateTime(timezone=True), server_default=func.now())

```

```

class StudentDrive(Base):

```

```

    __tablename__ = "student_drives"

    id = Column(Integer, primary_key=True, index=True)
    student_id = Column(Integer, ForeignKey("students.student_id", ondelete="CASCADE"),
        nullable=False)
    drive_id = Column(Integer, ForeignKey("placement_drives.id", ondelete="CASCADE"),
        nullable=False)
    is_eligible = Column(Boolean, default=False)
    applied = Column(Boolean, default=False)
    status = Column(String(30), default="Not Applied")
    probability_score = Column(Float, nullable=True)
    current_round = Column(Integer, default=0)
    final_result = Column(String(30), default="pending")
    created_at = Column(DateTime(timezone=True), server_default=func.now())
    updated_at = Column(DateTime(timezone=True), server_default=func.now(),
        onupdate=func.now())

    __table_args__ = (
        UniqueConstraint("student_id", "drive_id", name="unique_student_drive"),
    )

```

```

class DriveFacultyMap(Base):

```

```

    __tablename__ = "drive_faculty_map"

    id = Column(Integer, primary_key=True, index=True)

```

```

drive_id = Column(Integer, ForeignKey("placement_drives.id", ondelete="CASCADE"),
    nullable=False)
faculty_id = Column(Integer, ForeignKey("users.user_id", ondelete="CASCADE"),
    nullable=False)
role = Column(String(30), nullable=False, default="faculty")
department = Column(String(80), nullable=True)
assigned_from = Column(Date, nullable=True)
assigned_to = Column(Date, nullable=True)
assigned_by = Column(Integer, ForeignKey("users.user_id", ondelete="SET NULL"),
    nullable=True)
is_active = Column(Boolean, default=True)
assigned_at = Column(DateTime(timezone=True), server_default=func.now())

```

```

class DriveApplication(Base):

```

```

    __tablename__ = "drive_applications"

```

```

    id = Column(Integer, primary_key=True, index=True)
    drive_id = Column(Integer, ForeignKey("placement_drives.id", ondelete="CASCADE"),
        nullable=False)
    student_id = Column(Integer, ForeignKey("students.student_id", ondelete="CASCADE"),
        nullable=False)
    application_status = Column(String(30), default="not_applied")
    application_type = Column(String(30), default="normal")
    current_round = Column(Integer, default=0)
    final_status = Column(String(30), default="pending")
    updated_by = Column(Integer, ForeignKey("users.user_id", ondelete="SET NULL"),
        nullable=True)
    updated_at = Column(DateTime(timezone=True), server_default=func.now(),
        onupdate=func.now())

    __table_args__ = (
        UniqueConstraint("drive_id", "student_id", name="unique_drive_application"),
    )

```

```

class DriveAuditLog(Base):
    __tablename__ = "drive_audit_logs"

    id = Column(Integer, primary_key=True, index=True)
    drive_id = Column(Integer, ForeignKey("placement_drives.id", ondelete="CASCADE"),
        nullable=False)
    action = Column(String(50), nullable=False)
    old_data = Column(JSON, nullable=True)
    new_data = Column(JSON, nullable=True)
    user_id = Column(Integer, ForeignKey("users.user_id", ondelete="SET NULL"),
        nullable=True)
    timestamp = Column(DateTime(timezone=True), server_default=func.now())

```

```

class AssignmentHistory(Base):
    __tablename__ = "assignment_history"

    id = Column(Integer, primary_key=True, index=True)
    drive_id = Column(Integer, ForeignKey("placement_drives.id", ondelete="CASCADE"),
        nullable=False)
    faculty_id = Column(Integer, ForeignKey("users.user_id", ondelete="CASCADE"),
        nullable=False)
    role = Column(String(30), nullable=False)
    action = Column(String(30), nullable=False)
    timestamp = Column(DateTime(timezone=True), server_default=func.now())

```

```

class AuditLog(Base):
    __tablename__ = "audit_logs"

    id = Column(Integer, primary_key=True, index=True)
    action_type = Column(String(80), nullable=False)
    performed_by = Column(Integer, ForeignKey("users.user_id", ondelete="SET NULL"),
        nullable=True)
    target_id = Column(Integer, ForeignKey("users.user_id", ondelete="SET NULL"),
        nullable=True)

```

```
details = Column("metadata", JSON, nullable=True)
created_at = Column(DateTime(timezone=True), server_default=func.now())
```

```
class DriveCoordinatorMap(Base):
```

```
    __tablename__ = "drive_coordinator_map"

    id = Column(Integer, primary_key=True, index=True)
    drive_id = Column(Integer, ForeignKey("placement_drives.id", ondelete="CASCADE"),
        nullable=True)
    faculty_id = Column(Integer, ForeignKey("users.user_id", ondelete="CASCADE"),
        nullable=False)
    assigned_from = Column(DateTime(timezone=True), nullable=False)
    assigned_to = Column(DateTime(timezone=True), nullable=False)
    is_active = Column(Boolean, default=True)
    created_by = Column(Integer, ForeignKey("users.user_id", ondelete="SET NULL"),
        nullable=True)
    created_at = Column(DateTime(timezone=True), server_default=func.now())
```

```
class PlacementFeedback(Base):
```

```
    __tablename__ = "placement_feedback"

    id = Column(Integer, primary_key=True, index=True)
    student_id = Column(Integer, ForeignKey("students.student_id", ondelete="CASCADE"),
        nullable=False)
    drive_id = Column(Integer, ForeignKey("placement_drives.id", ondelete="CASCADE"),
        nullable=False)
    faculty_id = Column(Integer, ForeignKey("users.user_id", ondelete="SET NULL"),
        nullable=True)
    comment = Column(Text, nullable=True)
    rating = Column(Integer, nullable=True)
    round_reached = Column(String(120), nullable=True)
    difficulty = Column(String(30), nullable=True)
    issues_faced = Column(Text, nullable=True)
    submitted_by = Column(String(30), nullable=True)
```

```

    created_at = Column(DateTime(timezone=True), server_default=func.now())

# -----
# STUDENT ALERTS
# -----
class StudentAlert(Base):
    __tablename__ = "student_alerts"

    id = Column(Integer, primary_key=True, index=True)
    student_id = Column(Integer, ForeignKey("students.student_id"))
    reason = Column(String)
    created_at = Column(DateTime, default=datetime.utcnow)

# -----
# FACULTY (EXTENSION)
# -----
class Faculty(Base):
    __tablename__ = "faculty"

    faculty_id = Column(Integer, ForeignKey("users.user_id"), primary_key=True)
    employee_id = Column(String, unique=True, nullable=False)

    designation = Column(String)
    qualifications = Column(String)
    experience = Column(String)

    phone = Column(String)
    bio = Column(Text)

    expertise = Column(Text)    # comma-separated
    certifications = Column(Text) # JSON string
    publications = Column(Text) # JSON string
    classes = Column(Text)     # JSON string

```

```

linkedin = Column(String)
github = Column(String)
portfolio = Column(String)

#  NEW (SOFT DELETE)
#is_deleted = Column(Boolean, default=False)
#deleted_at = Column(DateTime, nullable=True)

# -----
# TIMETABLE
# -----
class Timetable(Base):
    __tablename__ = "timetables"

    id = Column(Integer, primary_key=True, index=True)
    faculty_id = Column(Integer, ForeignKey("faculty.faculty_id"), nullable=True)

    # BASIC INFO
    title = Column(String(255), nullable=False)    # e.g. "III Year Class Timetable"
    timetable_type = Column(String(50), nullable=False) # class / exam / fest / event

    # OPTIONAL CLASS DETAILS
    department = Column(String(50), nullable=True)
    year = Column(String(20), nullable=True)
    section = Column(String(10), nullable=True)
    semester = Column(String(20), nullable=True)

    # FILE / LINK DETAILS
    file_name = Column(String(255), nullable=True)
    file_url = Column(Text, nullable=False)
    file_type = Column(String(20), nullable=False) # pdf / excel / doc / image / link

    # AUDIENCE
    audience = Column(String(50), default="students") # students / faculty / both / all

```

```

# ADMIN META
uploaded_by = Column(Integer, ForeignKey("users.user_id"), nullable=False)
uploaded_at = Column(DateTime(timezone=True), server_default=func.now())

is_active = Column(Boolean, default=True)

# =====
# SYSTEM SETTINGS
class SystemSetting(Base):
    __tablename__ = "system_settings"
    __table_args__ = (UniqueConstraint("key"),)

    id = Column(Integer, primary_key=True, index=True)
    key = Column(String(255), unique=True, nullable=False)
    value = Column(JSON, nullable=False)
    updated_at = Column(DateTime(timezone=True), server_default=func.now(),
        onupdate=func.now())

# =====
# MARKS UPLOADS
# =====
# SETTINGS AUDIT LOGS
class SettingsAuditLog(Base):
    __tablename__ = "settings_audit_logs"

    id = Column(Integer, primary_key=True, index=True)
    key = Column(String(255), nullable=False)
    old_value = Column(String, nullable=True)
    new_value = Column(String, nullable=True)
    updated_by = Column(String, default="admin")
    timestamp = Column(DateTime(timezone=True), default=datetime.utcnow)

```

```

class MarksUpload(Base):
    __tablename__ = "marks_uploads"

    id = Column(Integer, primary_key=True, index=True)
    faculty_id = Column(Integer, ForeignKey("users.user_id"), nullable=False)
    subject_id = Column(Integer, ForeignKey("subjects.subject_id"), nullable=False)
    year = Column(Integer, nullable=False)
    section = Column(String(10), nullable=False)
    exam = Column(String(50), nullable=False)
    file_hash = Column(String(64), nullable=False)
    uploaded_at = Column(DateTime(timezone=True), server_default=func.now())

# =====
# ALERTS
# =====

class Alert(Base):
    __tablename__ = "alerts"

    id = Column(Integer, primary_key=True, index=True)

    title = Column(String(255), nullable=False)
    message = Column(Text, nullable=False)

    type = Column(String(50), nullable=False)

    target_role = Column(String(50), nullable=False)
    target_type = Column(String(50), nullable=False)

    department = Column(String(50), nullable=True)

    faculty_id = Column(
        Integer,
        ForeignKey("faculty.faculty_id"),
        nullable=True

```

```

)
student_id = Column(
    Integer,
    ForeignKey("students.student_id"),
    nullable=True
)

file_name = Column(String, nullable=True)
file_path = Column(String, nullable=True)
file_type = Column(String, nullable=True)
drive_id = Column(Integer, ForeignKey("placement_drives.id"), nullable=True)
is_read = Column(Boolean, default=False)

created_at = Column(DateTime(timezone=True), server_default=func.now())

# -----
# ALERT RECIPIENTS
# -----
class AlertRecipient(Base):
    __tablename__ = "alert_recipients"

    id = Column(Integer, primary_key=True)
    alert_id = Column(Integer, ForeignKey("alerts.id"))
    user_id = Column(Integer, ForeignKey("users.user_id"))
    is_read = Column(Boolean, default=False)

# -----
# FACULTY SUBJECTS (MANY-TO-MANY)
# -----
class FacultySubject(Base):
    __tablename__ = "faculty_subjects"

    id = Column(Integer, primary_key=True, index=True)

    faculty_id = Column(Integer, ForeignKey("faculty.faculty_id"))

```

```

subject_id = Column(Integer, ForeignKey("subjects.subject_id"))
year = Column(Integer, nullable=False)
section = Column(String, nullable=False)
is_active = Column(Boolean, default=True)
subject = relationship("Subject")
assigned_at = Column(DateTime(timezone=True), server_default=func.now())
updated_at = Column(DateTime(timezone=True), onupdate=func.now())

# -----
# SUBJECTS
# -----

class Subject(Base):
    __tablename__ = "subjects"

    subject_id = Column(Integer, primary_key=True)
    subject_code = Column(String)
    subject_name = Column(String)
    semester = Column(Integer)
    credits = Column(Integer)
    department_id = Column(Integer)

class Attendance(Base):
    __tablename__ = "attendance"

    __table_args__ = (
        UniqueConstraint(
            "student_id",
            "subject_id",
            "attendance_date",
            name="unique_attendance_per_day"
        ),
    )
    attendance_id = Column(Integer, primary_key=True, index=True)

```

```

student_id = Column(Integer, ForeignKey("students.student_id"), nullable=False)
subject_id = Column(Integer, ForeignKey("subjects.subject_id"), nullable=False)
faculty_id = Column(Integer, ForeignKey("faculty.faculty_id"), nullable=False)
attendance_date = Column(Date, nullable=False)
status = Column(Boolean, nullable=False)
student = relationship("Student")
subject = relationship("Subject")
faculty = relationship("Faculty")

```

```

class AttendanceWarning(Base):

```

```

    __tablename__ = "attendance_warnings"

```

```

    id = Column(Integer, primary_key=True, index=True)
    student_id = Column(Integer)
    subject_id = Column(Integer)
    semester = Column(Integer)
    level = Column(String)
    last_sent = Column(DateTime, default=datetime.utcnow)

```

```

class FacultyMonthlyAttendanceAlert(Base):

```

```

    __tablename__ = "faculty_monthly_attendance_alerts"
    id = Column(Integer, primary_key=True, index=True)
    faculty_id = Column(Integer)
    subject_id = Column(Integer)
    year = Column(Integer)
    section = Column(String)
    month = Column(Integer)
    year_value = Column(Integer)
    last_sent = Column(DateTime, default=datetime.utcnow)

```

```

# -----

```

```

# ASSIGNMENTS

```

```

# -----

```

```

class Assignment(Base):

```

```

__tablename__ = "assignments"

id = Column(Integer, primary_key=True, index=True)

# Assignment Basic Info
title = Column(String(255), nullable=False)
description = Column(Text, nullable=True)

# Class Details
faculty_id = Column(Integer, ForeignKey("faculty.faculty_id"), nullable=False)
subject_id = Column(Integer, ForeignKey("subjects.subject_id"), nullable=False)
year = Column(Integer, nullable=False)
section = Column(String(10), nullable=False)

# Dates
created_at = Column(DateTime, default=datetime.utcnow)
due_date = Column(DateTime, nullable=False)

# File
file_name = Column(String(255), nullable=True)
file_path = Column(Text, nullable=True)
file_type = Column(String(20), nullable=True)

# Status
is_active = Column(Boolean, default=True)

# Relationships
faculty = relationship("Faculty")
subject = relationship("Subject")

class AssignmentSubmission(Base):
    __tablename__ = "assignment_submissions"
    id = Column(Integer, primary_key=True, index=True)

```

```

assignment_id = Column(Integer, ForeignKey("assignments.id"))
student_id = Column(Integer, ForeignKey("students.student_id"))
file_name = Column(String, nullable=True)
file_path = Column(String, nullable=True)
file_type = Column(String, nullable=True)
submission_text = Column(Text, nullable=True)
submitted_at = Column(DateTime)
is_late = Column(Boolean, default=False)
is_submitted = Column(Boolean, default=True)
status = Column(String(20), default="pending")
__table_args__ = (
    UniqueConstraint("assignment_id", "student_id", name="unique_submission"),
)

```

```

class AssignmentDeadlineAlert(Base):

```

```

    __tablename__ = "assignment_deadline_alerts"
    id = Column(Integer, primary_key=True, index=True)
    assignment_id = Column(Integer)
    faculty_id = Column(Integer)
    alert_sent = Column(Boolean, default=False)
    sent_at = Column(DateTime)

```

```

class Resource(Base):

```

```

    __tablename__ = "resources"

    id = Column(Integer, primary_key=True, index=True)
    title = Column(String)
    description = Column(Text)
    subject_id = Column(Integer)
    faculty_id = Column(Integer)
    type = Column(String)
    file_url = Column(Text)
    created_at = Column(DateTime)

```

```

class ResourceAccess(Base):
    __tablename__ = "resource_access"
    id = Column(Integer, primary_key=True, index=True)
    resource_id = Column(Integer)
    student_id = Column(Integer)
    action_type = Column(String, default="view")
    accessed_at = Column(DateTime)

# =====
# EVENTS
# =====

class Event(Base):
    __tablename__ = "events"
    id = Column(Integer, primary_key=True, index=True)
    title = Column(String(255), nullable=False)
    description = Column(Text, nullable=True)
    event_type = Column(String(100), nullable=False)
    event_date = Column(Date, nullable=False)
    location = Column(String(255), nullable=True)
    venue = Column(String(255), nullable=True)
    organizer = Column(String(255), nullable=True)
    max_participants = Column(Integer, nullable=True)
    registration_deadline = Column(DateTime, nullable=True)
    external_registration_link = Column(String, nullable=True)
    year = Column(String(20), nullable=True)
    section = Column(String(10), nullable=True)
    created_by = Column(Integer, ForeignKey("faculty.faculty_id"), nullable=False)
    status = Column(String(50), default="upcoming") # upcoming, ongoing, completed
    created_at = Column(DateTime(timezone=True), server_default=func.now())
    faculty = relationship("Faculty")

class EventAttendance(Base):
    __tablename__ = "event_attendance"

```

```

id = Column(Integer, primary_key=True, index=True)
event_id = Column(Integer, ForeignKey("events.id", ondelete="CASCADE"),
    nullable=False)
student_id = Column(Integer, ForeignKey("students.student_id", ondelete="CASCADE"),
    nullable=False)
status = Column(String(50), default="absent") # present / absent
result = Column(String(50), nullable=True) # winner, runner_up, participant, None
marked_at = Column(DateTime(timezone=True), onupdate=func.now())
event = relationship("Event")
student = relationship("Student")

```

```

class EventRegistration(Base):

```

```

    __tablename__ = "event_registrations"
    id = Column(Integer, primary_key=True, index=True)
    event_id = Column(Integer, ForeignKey("events.id", ondelete="CASCADE"),
        nullable=False)
    student_id = Column(Integer, ForeignKey("students.student_id", ondelete="CASCADE"),
        nullable=False)
    registered_at = Column(DateTime(timezone=True), server_default=func.now())
    attendance = Column(String(50), default=None, nullable=True)
    result = Column(String(50), nullable=True)
    certificate_uploaded = Column(String, nullable=True)
    faculty_verified = Column(Boolean, default=False)

    event = relationship("Event")
    student = relationship("Student")

```

```

class ExternalEventSubmission(Base):

```

```

    __tablename__ = "external_event_submissions"
    id = Column(Integer, primary_key=True, index=True)
    student_id = Column(Integer, ForeignKey("students.student_id", ondelete="CASCADE"),
        nullable=False)
    event_name = Column(String(255), nullable=False)
    organizer = Column(String(255), nullable=True)

```

```

event_date = Column(Date, nullable=False)
achievement_type = Column(String(100), nullable=True) # Participation, Winner, etc.
position = Column(String(100), nullable=True)
certificate_file = Column(String, nullable=True)
proof_file = Column(String, nullable=True)
status = Column(String(50), default="pending") # pending / approved / rejected
submitted_at = Column(DateTime(timezone=True), server_default=func.now())
faculty_reviewed_by = Column(Integer, ForeignKey("faculty.faculty_id"), nullable=True)

student = relationship("Student")

# -----
# MARKS
# -----
class Mark(Base):
    __tablename__ = "marks"
    id = Column(Integer, primary_key=True, index=True)
    student_id = Column(Integer, ForeignKey("students.student_id"))
    subject_id = Column(Integer, ForeignKey("subjects.subject_id"))
    exam = Column(String) # Mid-1, Mid-2, Assignment, Semester
    marks = Column(Numeric(5,2), nullable=True)
    extra_data = Column(JSON, default={})
    assignment_total = Column(Numeric(5,2), default=0.00)
    mid1 = Column(Numeric(5,2), default=0.00)
    mid2 = Column(Numeric(5,2), default=0.00)
    semester = Column(Numeric(5,2), default=0.00)
    total = Column(Numeric(5,2), default=0.00)
    sgpa = Column(Numeric(4,2), default=0.00)
    cgpa = Column(Numeric(4,2), default=0.00)
    year = Column(Integer)
    section = Column(String)
    faculty_id = Column(Integer, ForeignKey("faculty.faculty_id"))
    created_at = Column(DateTime, default=datetime.utcnow)

```

```

# Relationships
student = relationship("Student")
subject = relationship("Subject")
faculty = relationship("Faculty")
# -----
# SCALING SYSTEM
# -----
class ScaledMark(Base):
    __tablename__ = "scaled_marks"
    id = Column(Integer, primary_key=True, index=True)
    student_id = Column(Integer, ForeignKey("students.student_id"))
    subject_id = Column(Integer, ForeignKey("subjects.subject_id"))
    year = Column(Integer)
    section = Column(String)
    assignment_scaled = Column(Numeric(5,2))
    mid1_scaled = Column(Numeric(5,2))
    mid2_scaled = Column(Numeric(5,2))
    mid_combined = Column(Numeric(5,2))
    internal_total = Column(Numeric(5,2))
    semester_marks = Column(Numeric(5,2))
    final_total = Column(Numeric(5,2))
    student = relationship("Student")
    subject = relationship("Subject")

class ScalingLog(Base):
    __tablename__ = "scaling_logs"
    id = Column(Integer, primary_key=True, index=True)
    faculty_id = Column(Integer, ForeignKey("faculty.faculty_id"))
    action_type = Column(String) # "apply" | "recalculate" | "undo"
    year = Column(Integer)
    section = Column(String)
    subject_id = Column(Integer, ForeignKey("subjects.subject_id"))
    file_name = Column(String, nullable=True) # New column for excel scaling
    timestamp = Column(DateTime, default=datetime.utcnow)

```

```

        snapshot_data = Column(JSON, nullable=True) # Stores previous state for undo.
# -----
# GAMIFICATION PROGRESS
# -----
class StudentProgress(Base):
    __tablename__ = "student_progress"
    student_id = Column(Integer, ForeignKey("students.student_id", ondelete="CASCADE"),
        primary_key=True)
    total_xp = Column(Integer, default=0, nullable=False)
    streak_days = Column(Integer, default=0, nullable=False)
    last_active_date = Column(Date, nullable=True)

class TaskLog(Base):
    __tablename__ = "task_logs"
    id = Column(Integer, primary_key=True, index=True)
    student_id = Column(Integer, ForeignKey("students.student_id", ondelete="CASCADE"),
        nullable=False, index=True)
    task_id = Column(String(255), nullable=False, index=True)
    completed = Column(Boolean, default=False, nullable=False)
    verified = Column(Boolean, default=False, nullable=False)
    xp_earned = Column(Integer, default=0, nullable=False)
    date = Column(Date, default=lambda: datetime.utcnow().date(), nullable=False)
    __table_args__ = (
        UniqueConstraint("student_id", "task_id", "date",
            name="uq_task_log_student_task_date"),
    )

```

8.5 Schemas.py

```

from __future__ import annotations
from pydantic import BaseModel
from typing import List, Optional, Any, Union, Dict
from datetime import datetime, date
DateType = date

```

```

class LoginRequest(BaseModel):
    email: str
    password: str

class StudentSignupRequest(BaseModel):
    name: str
    roll_no: str
    email: str
    password: str

class StudentProfileResponse(BaseModel):
    name: str
    email: str
    roll_no: str | None
    year: int
    semester: int
    skills: list[str] = []
    linkedin: str | None = None
    github: str | None = None
    portfolio: str | None = None
    bio: str | None = None

class Certificate(BaseModel):
    title: str
    link: str

class StudentProfileUpdate(BaseModel):
    year: int
    semester: int
    skills: list[str]
    certificates: list[Certificate]
    linkedin: str | None = None
    github: str | None = None
    portfolio: str | None = None

```

```
class TeacherSignupRequest(BaseModel):
```

```
    name: str
```

```
    email: str
```

```
    password: str
```

```
    employee_id: str
```

```
    department_id: int
```

```
class FacultyCertificate(BaseModel):
```

```
    title: str
```

```
    link: str
```

```
class FacultyProfileResponse(BaseModel):
```

```
    name: str
```

```
    email: str
```

```
    employee_id: str
```

```
    designation: str | None = None
```

```
    qualifications: str | None = None
```

```
    experience: int | None = None
```

```
    phone: str | None = None
```

```
    bio: str | None = None
```

```
    expertise: list[str] = []
```

```
    certifications: list[FacultyCertificate] = []
```

```
    linkedin: str | None = None
```

```
    website: str | None = None
```

```
class NameLink(BaseModel):
```

```
    name: str
```

```
    link: str
```

```
class ClassItem(BaseModel):
```

```
    year: str
```

```
    section: str
```

```
    subject: str
```

```
    students: str
```

attendance: Optional[float] = None # auto later

class FacultyProfileUpdate(BaseModel):

name: Optional[str] = None

phone: Optional[str] = None

bio: Optional[str] = None

qualifications: Optional[str] = None

experience: Optional[str] = None

linkedin: Optional[str] = None

github: Optional[str] = None

portfolio: Optional[str] = None

expertise: Optional[List[str]] = None

certifications: Optional[List[NameLink]] = None

publications: Optional[List[NameLink]] = None

classes: Optional[List[ClassItem]] = None

class AdminLoginRequest(BaseModel):

email: str

password: str

access_key: str

class ResetPasswordRequest(BaseModel):

token: str

new_password: str

class SettingsUpdateRequest(BaseModel):

attendance_threshold: Optional[float] = None

cgpa_threshold: Optional[float] = None

attendance_alert_enabled: Optional[bool] = None

cgpa_alert_enabled: Optional[bool] = None

alert_frequency: Optional[str] = None

report_retention_days: Optional[int] = None

analytics_refresh_interval: Optional[str] = None

session_timeout: Optional[int] = None

```

report_format: Optional[str] = None
marks_format: Optional[str] = None
attendance_format: Optional[str] = None
assignment_format: Optional[str] = None
resources_format: Optional[str] = None
class Config:
    extra = "forbid"
class SettingsPreviewRequest(BaseModel):
    attendance_threshold: float
    cgpa_threshold: float

class AdminOverviewMetrics(BaseModel):
    at_risk_students: int
    attendance_risk_percent: float
    data_completeness: float
    active_alerts: int
    total_students: int
    total_teachers: int
    active_events: int
    events_today: int
    events_this_week: int

class AdminAcademicHealth(BaseModel):
    avg_attendance: float
    avg_cgpa: float
    at_risk_students: int

class AdminFacultyHealth(BaseModel):
    avg_classes: float
    overloaded: int
    underutilized: int

class AdminSystemHealth(BaseModel):
    active_users: int

```

```
last_sync: str
data_completeness: float
```

```
class AdminOverviewAlert(BaseModel):
```

```
    title: str
    type: str
    severity: str
    timestamp: str
    action: str
```

```
class AdminOverviewTrendPoint(BaseModel):
```

```
    date: str
    attendance: float
```

```
class AdminOverviewResponse(BaseModel):
```

```
    metrics: AdminOverviewMetrics
    academic_health: AdminAcademicHealth
    faculty_health: AdminFacultyHealth
    system_health: AdminSystemHealth
    alerts: List[AdminOverviewAlert]
    trend: List[AdminOverviewTrendPoint]
```

```
# =====
```

```
# TIMETABLE SCHEMAS
```

```
# =====
```

```
# ♦ Admin Upload (metadata only, file handled separately)
```

```
class TimetableCreate(BaseModel):
```

```
    title: str                # "III Year Class Timetable"
    timetable_type: str        # class / exam / fest / event
    department: Optional[str] = None
    year: Optional[str] = None
    section: Optional[str] = None
    semester: Optional[str] = None
```

```
audience: str = "students"      # students / faculty / both / all
```

```
# ♦ Timetable Response (single item)
```

```
class TimetableResponse(BaseModel):
```

```
    id: int
    title: str
    timetable_type: str
    faculty_id: Optional[int] = None # ✅ ADD THIS
    department: Optional[str]
    year: Optional[str]
    section: Optional[str]
    semester: Optional[str]
    file_name: Optional[str]
    file_url: str
    file_type: str
    audience: str
    uploaded_at: datetime
    is_active: bool
```

```
# ♦ Timetable List Response
```

```
class TimetableListResponse(BaseModel):
```

```
    timetables: List[TimetableResponse]
```

```
# =====
```

```
# STUDENT PROMOTION SCHEMAS
```

```
# =====
```

```
class StudentPromotionRequest(BaseModel):
```

```
    student_ids: list[int]
    new_semester: int | None = None
    new_section: str | None = None
```

```
class DriveCreate(BaseModel):
```

```
    title: str
```

company_name: str
role: str
package: float
min_cgpa: float
max_backlogs: int
date: DateType
location: str
registration_deadline: DateType
eligible_years: List[int]
branches: List[str]
mode: str
status: str
selection_process: Optional[List[str]] = None
apply_link: Optional[str] = None
details_pdf: Optional[str] = None

```
class DriveUpdate(BaseModel):  
    title: Optional[str] = None  
    company_name: Optional[str] = None  
    role: Optional[str] = None  
    package: Optional[float] = None  
    min_cgpa: Optional[float] = None  
    max_backlogs: Optional[int] = None  
    date: Optional[DateType] = None  
    location: Optional[str] = None  
    registration_deadline: Optional[DateType] = None  
    apply_link: Optional[str] = None  
    details_pdf: Optional[str] = None  
    eligible_years: Optional[List[int]] = None  
    branches: Optional[List[str]] = None  
    mode: Optional[str] = None  
    status: Optional[str] = None  
    selection_process: Optional[List[str]] = None
```

```
class DrivePreviewResponse(BaseModel):
    total_students: int
    eligible_count: int
    branch_wise_count: Dict[str, int]
    recipients: Optional[List[Dict[str, Any]]] = None
    stats: Optional[Dict[str, Any]] = None
```

```
class DriveClosePreviewResponse(BaseModel):
    total_students: int
    eligible: int
    applied: int
    selected: int
```

```
class DriveEditPreviewResponse(BaseModel):
    old_data: Dict[str, Any]
    new_data: Dict[str, Any]
    changed_fields: Dict[str, Dict[str, Any]]
```

```
class PlacementDriveFeedbackCreate(BaseModel):
    round_reached: str
    difficulty: str
    issues_faced: Optional[str] = None
    comment: Optional[str] = None
    rating: Optional[int] = None
```

```
class FacultyAssignmentItem(BaseModel):
    faculty_id: int
    assigned_from: DateType
    assigned_to: DateType
```

```
class AssignFacultyRequest(BaseModel):
    assignments: List[FacultyAssignmentItem]
```

```
class DriveRoleAssignmentRequest(BaseModel):
```

```
    faculty_id: int
```

```
    role: str
```

```
    assigned_from: DateType
```

```
    assigned_to: DateType
```

```
class FacultyAssignmentUpdateRequest(BaseModel):
```

```
    assigned_from: Optional[DateType] = None
```

```
    assigned_to: Optional[DateType] = None
```

```
    is_active: Optional[bool] = None
```

```
class CoordinatorAssignRequest(BaseModel):
```

```
    faculty_id: int
```

```
    drive_id: Optional[int] = None
```

```
    assigned_from: datetime
```

```
    assigned_to: datetime
```

```
class CoordinatorExtendRequest(BaseModel):
```

```
    assigned_from: Optional[datetime] = None
```

```
    assigned_to: datetime
```

```
class DriveStudentUpdateRequest(BaseModel):
```

```
    current_round: Optional[int] = None
```

```
    final_status: Optional[str] = None
```

```
class DriveNotifyFilteredRequest(BaseModel):
```

```
    branch: Optional[List[str]] = None
```

```
    year: Optional[List[int]] = None
```

```
    status: Optional[List[str]] = None
```

```
    title: Optional[str] = None
```

```
    message: Optional[str] = None
```

```
class StudentNotifyRequest(BaseModel):
```

```
    title: str
```

```

    message: str
    drive_id: Optional[int] = None

#class BulkPromoteRequest(BaseModel):
#    year: str
#    department: str
#    section: Optional[str] = None
#    new_year: str
#    new_section: Optional[str] = None

# =====
# STUDENT DELETE SCHEMA
# =====

class StudentDeleteRequest(BaseModel):
    student_ids: list[int]

# =====
# TEACHER UPDATE (ADMIN)
# =====

class TeacherAdminUpdate(BaseModel):
    designation: Optional[str] = None
    department_id: Optional[int] = None

# =====
# TEACHER DELETE SCHEMA (ADMIN)
# =====

class TeacherDeleteRequest(BaseModel):
    teacher_ids: List[int]

# =====
# ALERT SCHEMA
# =====

class AlertCreate(BaseModel):
    title: str

```

```

message: str
type: str
target_role: str
target_type: str
department: Optional[str] = None
faculty_id: Optional[int] = None
student_id: Optional[int] = None

```

```

class AlertSendRequest(BaseModel):
    type: str # Emergency | Announcement | Info | Reminder
    message: str
    target: str # class | multiple_classes | students
    subject_id: Optional[int] = None
    subject_ids: Optional[List[int]] = None
    student_ids: Optional[List[int]] = None

```

```

class StudentSearchResponse(BaseModel):
    student_id: int
    name: str
    roll_no: Optional[str] = None

```

```

# =====
# ATTENDANCE SCHEMAS
# =====

```

```

class AttendanceItem(BaseModel):
    student_id: int
    status: bool # TRUE = present, FALSE = absent

```

```

class AttendanceCreate(BaseModel):
    subject_id: int
    date: date
    year: int
    section: str
    records: List[AttendanceItem]

```

```

class AssignSubjectRequest(BaseModel):
    faculty_id: int
    subject_id: int
    year: int
    section: str

class SubjectCreate(BaseModel):
    subject_code: str
    subject_name: str
    semester: int
    credits: int
    department_id: int

# =====
# STUDENT ATTENDANCE TREND SCHEMAS
# =====

class AttendanceTrendItem(BaseModel):
    date: date
    percentage: float

class SubjectComparisonItem(BaseModel):
    subject: str
    percentage: float

class AttendancePrediction(BaseModel):
    projected_percentage: float
    confidence: str

class AttendanceAnalyticsResponse(BaseModel):
    trend: List[AttendanceTrendItem]
    subject_comparison: List[SubjectComparisonItem]
    prediction: AttendancePrediction

```

```

# =====
# ASSIGNMENT SCHEMAS
# =====

class AssignmentCreate(BaseModel):
    title: str
    description: Optional[str] = None
    subject_id: int
    year: int
    section: str
    due_date: datetime
    file_path: Optional[str] = None


class AssignmentResponse(BaseModel):
    id: int
    title: str
    description: Optional[str] = None
    subject_id: int
    year: int
    section: str
    due_date: datetime
    created_at: datetime
    is_active: bool
    faculty_id: int
    file_path: Optional[str] = None


class Config:
    from_attributes = True


class AssignmentSubmissionCreate(BaseModel):
    assignment_id: int
    submission_text: Optional[str] = None


class AssignmentSubmissionResponse(BaseModel):
    id: int

```

```

assignment_id: int
student_id: int
file_path: Optional[str] = None
submission_text: Optional[str] = None
submitted_at: datetime
is_late: bool
is_submitted: bool
status: str

class Config:
    from_attributes = True

class StatusUpdateRequest(BaseModel):
    status: str

class AssignmentStatusDot(BaseModel):
    assignment_id: int
    title: str
    status: str # "approved", "rejected", "pending", "not_submitted", "future"

class StudentAssignmentSummary(BaseModel):
    student_id: int
    name: str
    roll: str
    year: int
    section: str
    recent_assignments: List[AssignmentStatusDot]

class StudentAssignmentSummaryResponse(BaseModel):
    status: str
    students: List[StudentAssignmentSummary]

class AssignmentDetailResponse(BaseModel):
    id: int

```

```
title: str
description: Optional[str] = None
subject_id: int
year: int
section: str
due_date: datetime
created_at: datetime
is_active: bool
faculty_id: int
```

```
total_students: int
submitted_count: int
pending_count: int
pending: List[dict] = []
submitted: List[dict] = []
class Config:
    from_attributes = True
```

```
class FacultySubjectResponse(BaseModel):
    subject_id: int
    subject_name: str
    year: int
    section: str
    department: str
```

```
class StudentAssignmentResponse(BaseModel):
    id: int
    title: str
    description: Optional[str]
    subject_name: str
    due_date: datetime
    file_path: Optional[str]
    is_submitted: bool
```

```

class ResourceResponse(BaseModel):
    id: int
    title: str
    type: str
    subject: str
    created_at: datetime
    accessed: int
    total_students: int

    class Config:
        orm_mode = True

class ResourceAccessRequest(BaseModel):
    action_type: str

class StudentResourceResponse(BaseModel):
    id: int
    title: str
    description: str
    type: str
    file_url: str
    created_at: datetime

    class Config:
        orm_mode = True

# =====
# EVENTS SCHEMAS
# =====

from typing import Optional, List, Union, Any

class EventCreate(BaseModel):
    title: str
    description: Optional[str] = None

```

```
event_type: str
organizer: Optional[str] = None
venue: Optional[str] = None
event_date: date
max_participants: Optional[int] = None
registration_deadline: Optional[datetime] = None
external_registration_link: Optional[str] = None
year: str
section: str
```

```
class EventResponse(BaseModel):
    id: int
    title: str
    description: Optional[str] = None
    event_type: str
    organizer: Optional[str] = None
    venue: Optional[str] = None
    event_date: date
    max_participants: Optional[int] = None
    registration_deadline: Optional[datetime] = None
    external_registration_link: Optional[str] = None
    location: Optional[str] = None
    year: Any
    section: Any
    created_by: int
    status: str
    created_at: datetime
    total_students: int = 0
    present_count: int = 0
    absent_count: int = 0
```

```
class Config:
    from_attributes = True
```

```
class EventAttendanceItem(BaseModel):
```

```
    student_id: int
```

```
    status: str
```

```
class EventAttendanceUpdate(BaseModel):
```

```
    student_id: int
```

```
    status: str
```

```
class BulkEventAttendanceUpdate(BaseModel):
```

```
    students: List[EventAttendanceItem]
```

```
class EventStudentDetail(BaseModel):
```

```
    student_id: int
```

```
    name: str
```

```
    roll_no: Optional[str] = None
```

```
    attendance_status: str
```

```
    result: Optional[str] = None
```

```
class EventAttendanceResponse(BaseModel):
```

```
    event_id: int
```

```
    title: str
```

```
    date: date
```

```
    location: Optional[str] = None
```

```
    students: List[EventStudentDetail]
```

```
    message: Optional[str] = None
```

```
class EventAlertRequest(BaseModel):
```

```
    type: str
```

```
    message: str
```

```
    target: str # "all", "present", "absent"
```

```
class EventResultUpdate(BaseModel):
```

```
    student_id: int
```

```
    result: Optional[str] = None
```

```

class EventRegistrationRequest(BaseModel):
    event_id: int


class StudentEventResponse(BaseModel):
    id: int
    title: str
    event_type: str
    organizer: Optional[str] = None
    venue: Optional[str] = None
    event_date: date
    max_participants: Optional[int] = None
    registration_deadline: Optional[datetime] = None
    external_registration_link: Optional[str] = None
    location: Optional[str] = None
    description: Optional[str] = None
    status: str
    is_registered: bool = False
    attendance_status: Optional[str] = None
    result: Optional[str] = None


class Config:
    from_attributes = True


class ExternalEventSubmissionCreate(BaseModel):
    event_name: str
    organizer: Optional[str] = None
    event_date: date
    achievement_type: Optional[str] = None # New field
    position: Optional[str] = None


class ExternalEventSubmissionResponse(BaseModel):
    id: int
    student_id: int

```

```

event_name: str
organizer: Optional[str] = None
event_date: date
achievement_type: Optional[str] = None # New field
position: Optional[str] = None
certificate_file: Optional[str] = None
proof_file: Optional[str] = None
status: str
submitted_at: datetime
faculty_reviewed_by: Optional[int] = None

class Config:
    from_attributes = True

class FacultyExternalSubmissionDetail(ExternalEventSubmissionResponse):
    student_name: str
    student_roll_no: Optional[str] = None

# =====
# MARKS SCHEMAS
# =====

class MarkCreate(BaseModel):
    student_id: int
    marks: int

class MarksUpload(BaseModel):
    year: str
    section: str
    subject: str
    exam: str
    marks: List[MarkCreate]

class FacultyMarkResponse(BaseModel):
    id: int

```

```
student_id: int
student_name: str
roll_no: Optional[str]
assignment_total: float
mid1: float
mid2: float
semester: float
total: float
sgpa: float
cgpa: float
```

```
class StudentMarkResponse(BaseModel):
```

```
    subject: str
    exam: str
    assignment_total: float
    mid1: float
    mid2: float
    semester: float
    total: float
    sgpa: float
    cgpa: float
```

```
class TaskCompleteRequest(BaseModel):
```

```
    type: str
    priority: str
    verificationType: str
```

```
class StudentXpResponse(BaseModel):
```

```
    total_xp: int
```

```
class StudentStreakResponse(BaseModel):
```

```
    streak_days: int
```

```
class LeaderboardItem(BaseModel):
```

```

student_id: int
name: str
xp: int
rank: int

```

8.6 Security.py

```

import hashlib
from passlib.context import CryptContext
from passlib.exc import UnknownHashError

```

```

# Bcrypt context for SHA256 pre-hashed passwords (new method)
# Old bcrypt hashes are supported via deprecated schemes
pwd_context = CryptContext(
    schemes=["bcrypt"],
    deprecated="auto",
    bcrypt__rounds=12
)

```

```

def hash_password(password: str) -> str:
    """Hash password using SHA256 + bcrypt (new secure method).

```

Steps:

1. Hash plain password with SHA256 first
2. Then hash the SHA256 result with bcrypt

This prevents password length limits and provides two layers of hashing.

```

"""
print("🔵 PASSWORD RECEIVED LENGTH:", len(password))
# Step 1: SHA256 hash
sha_password = hashlib.sha256(password.encode("utf-8")).hexdigest()
# Step 2: bcrypt hash
return pwd_context.hash(sha_password)

```

```

def verify_password(plain_password: str, hashed_password: str) -> bool | str:
    """Verify password with backward compatibility for old hashes.

```

Returns:

```

    True: Password valid with current method
    "upgrade": Password valid with OLD method (needs rehashing)
    False: Password invalid

```

Supports both:

- NEW METHOD: SHA256 + bcrypt
- OLD METHOD: Direct bcrypt (auto-upgrades on successful login)

```

"""
print(f'[*] Verifying password (length: {len(plain_password)})')

```

```

# TRY NEW METHOD FIRST: SHA256 + bcrypt
try:
    sha_password = hashlib.sha256(plain_password.encode("utf-8")).hexdigest()
    if pwd_context.verify(sha_password, hashed_password):
        print("[✓] Password verified with NEW method (SHA256+bcrypt)")
        return True
except Exception as e:
    print(f"[!] New method verification error: {type(e).__name__}")
    pass

# FALLBACK TO OLD METHOD: Direct bcrypt (for backward compatibility)
try:
    if pwd_context.verify(plain_password, hashed_password):
        print("[△] Password verified with OLD method (direct bcrypt) - UPGRADE NEEDED")
        return "upgrade"
except UnknownHashError:
    print(f"[!] Unknown hash format")
    return False
except Exception as e:
    print(f"[!] Old method verification error: {type(e).__name__}: {e}")
    return False

print("[X] Password verification failed - invalid credentials")
return False

```

8.7 Admin Dashboard.jsx

```

import { useEffect, useState } from "react";
import { useNavigate, useLocation, Outlet } from "react-router-dom";

/* ===== ADMIN PAGES ===== */
import Overview from "../pages/Admin/Overview";
import Students from "../pages/Admin/Students";
import Teachers from "../pages/Admin/Teachers";
import Academics from "../pages/Admin/Academics";
import Timetable from "../pages/Admin/Timetable";
import Alerts from "../pages/Admin/Alerts";
import Insights from "../pages/Admin/Insights";
import Settings from "../pages/Admin/Settings";
import ErrorBoundary from "../components/ErrorBoundary";
import Logout from "../pages/Logout";
import Smart404 from "../components/Smart404";
import { recordRouteVisit } from "../utils/navigationHistory";

/* ===== ICONS ===== */
import DashboardIcon from "@mui/icons-material/Dashboard";
import PeopleIcon from "@mui/icons-material/People";
import SchoolIcon from "@mui/icons-material/School";
import MenuBookIcon from "@mui/icons-material/MenuBook";

```

```

import EventAvailableIcon from "@mui/icons-material/EventAvailable";
import NotificationsIcon from "@mui/icons-material/Notifications";
import BarChartIcon from "@mui/icons-material/BarChart";
import WorkIcon from "@mui/icons-material/Work";
import SettingsIcon from "@mui/icons-material/Settings";
import LogoutIcon from "@mui/icons-material/Logout";
import MenuIcon from "@mui/icons-material/Menu";

/* ===== ADMIN DASHBOARD ===== */

export default function AdminDashboard() {
  const [sidebarOpen, setSidebarOpen] = useState(true);
  const [showLogout, setShowLogout] = useState(false);
  const navigate = useNavigate();
  const location = useLocation();

  const availableAdminPages = [
    "overview",
    "students",
    "teachers",
    "academics",
    "timetable",
    "alerts",
    "insights",
    "placement",
    "settings",
  ];

  const activePage = location.pathname.replace(/^\$/ , "").split("/")[2] || "overview";

  useEffect(() => {
    if (!availableAdminPages.includes(activePage)) {
      return;
    }

    const path = activePage === "overview" ? "/admin" : `/admin/${activePage}`;
    const labelMap = {
      overview: "Dashboard",
      students: "Manage Users",
      insights: "View Reports",
    };

    recordRouteVisit({ label: labelMap[activePage] || "Dashboard", path, role: "admin" });
  }, [activePage]);

  /* ===== LOGOUT FLOW ===== */
  if (showLogout) {
    return (
      <Logout
        role="admin"
        onBack={() => setShowLogout(false)}

```

```

    />
  );
}

if (!availableAdminPages.includes(activePage)) {
  return <Smart404 />;
}

return (
  <div className="h-screen w-screen bg-gradient-to-br from-slate-50 via-white to-indigo-50">
    <div className="flex h-full">

      { /* ===== SIDEBAR ===== */ }
      <aside
        className={`transition-all duration-300 flex flex-col ${sidebarOpen ? "w-72" : "w-20"} p-4 glass
          border-r border-white/40`}
      >
        { /* LOGO */ }
        <div className="flex items-center justify-between mb-8">
          {sidebarOpen && (
            <h2 className="text-xl font-semibold text-indigo-700">
              GVP-MAAA Admin
            </h2>
          )}
          <button
            onClick={() => setSidebarOpen(!sidebarOpen)}
            className="p-2 rounded-lg hover:bg-white/60"
          >
            <MenuIcon fontSize="small" />
          </button>
        </div>

        { /* MENU GROUP */ }
        <div className="flex-1 overflow-y-auto flex flex-col">
          <div className="flex flex-col gap-2">
            <SidebarSection title="Management" open={sidebarOpen}>
              <MenuItem
                icon={DashboardIcon}
                label="Overview"
                open={sidebarOpen}
                active={activePage === "overview"}
                onClick={() => navigate("/admin")}
              />

              <MenuItem
                icon={PeopleIcon}
                label="Students"
                open={sidebarOpen}
                active={activePage === "students"}
                onClick={() => navigate("/admin/students")}
              />

```

```

<MenuItem
  icon={SchoolIcon}
  label="Teachers"
  open={sidebarOpen}
  active={activePage === "teachers"}
  onClick={() => navigate("/admin/teachers")}
/>
</SidebarSection>

<SidebarSection title="Academics" open={sidebarOpen}>
  <MenuItem
    icon={MenuBookIcon}
    label="Academics"
    open={sidebarOpen}
    active={activePage === "academics"}
    onClick={() => navigate("/admin/academics")}
  />

  <MenuItem
    icon={EventAvailableIcon}
    label="Timetable"
    open={sidebarOpen}
    active={activePage === "timetable"}
    onClick={() => navigate("/admin/timetable")}
  />
</SidebarSection>

<SidebarSection title="Monitoring" open={sidebarOpen}>
  <MenuItem
    icon={NotificationsIcon}
    label="Alerts"
    open={sidebarOpen}
    active={activePage === "alerts"}
    onClick={() => navigate("/admin/alerts")}
  />

  <MenuItem
    icon={BarChartIcon}
    label="Insights"
    open={sidebarOpen}
    active={activePage === "insights"}
    onClick={() => navigate("/admin/insights")}
  />

  <MenuItem
    icon={WorkIcon}
    label="Placement"
    open={sidebarOpen}
    active={activePage === "placement"}
    onClick={() => navigate("/admin/placement")}
  />

```

```

    />
  </SidebarSection>

  <SidebarSection title="System" open={sidebarOpen}>
    <MenuItem
      icon={SettingsIcon}
      label="Settings"
      open={sidebarOpen}
      active={activePage === "settings"}
      onClick={() => navigate("/admin/settings")}
    />
  </SidebarSection>
</div>

<div className="flex-1" />
</div>

<div className="pt-4 border-t border-white/40">
  <button
    onClick={() => setShowLogout(true)}
    className="w-full flex items-center gap-3 px-3 py-2 rounded-xl text-red-600 hover:bg-red-50"
  >
    <LogoutIcon fontSize="small" />
    {sidebarOpen && <span>Logout</span>}
  </button>
</div>
</aside>

  { /* ===== MAIN ===== */ }
  <main className="flex-1 p-8 overflow-y-auto">
    <Outlet />
  </main>
</div>
</div>
);
}

/* ===== HELPERS ===== */

function SidebarSection({ title, open, children }) {
  return (
    <div className="mb-6">
      {open && (
        <p className="text-xs uppercase text-gray-400 mb-2 tracking-wider">
          {title}
        </p>
      )}
      <div className="space-y-2">{children}</div>
    </div>
  );
}

```

```

function MenuItem({ icon: Icon, label, open, active, onClick, danger }) {
  return (
    <div
      onClick={onClick}
      className={`flex items-center gap-3 px-3 py-2 rounded-xl cursor-pointer transition
        ${active ? "bg-indigo-500/10 text-indigo-700" : "hover:bg-white/60"}
        ${danger && "text-red-600 hover:bg-red-50"}
        ${!open && "justify-center"}}`
    >
      <Icon fontSize="small" />
      {open && <span>{label}</span>}
    </div>
  );
}

```

8.8 Faculty Dashboard.jsx

```

import { useState, useEffect } from "react";
import { useLocation, useNavigate } from "react-router-dom";
import api from "../utils/axios";
import Overview from "../pages/Teacher/Overview";
import Timetable from "../pages/Teacher/Timetable";
import Attendance from "../pages/Teacher/Attendance";
import Assignment from "../pages/Teacher/Assignment";
import Marks from "../pages/Teacher/Marks";
import Resources from "../pages/Teacher/Resources";
import Events from "../pages/Teacher/Events";
import Insights from "../pages/Teacher/Insights";
import UploadResourceModal from "../pages/Teacher/UploadResourceModal";
import GiveAlertModal from "../pages/Teacher/GiveAlertModal";
import Alerts from "../pages/Teacher/Alerts";
import Logout from "../pages/Logout";
import TeacherProfilePage from "../pages/Teacher/TeacherProfilePage";
import Placement from "../pages/Teacher/Placement";
import PlacementCoordinator from "../pages/Teacher/PlacementCoordinator";
import Smart404 from "../components/Smart404";
import { recordRouteVisit } from "../utils/navigationHistory";

/* ICONS */
import DashboardIcon from "@mui/icons-material/Dashboard";
import EventAvailableIcon from "@mui/icons-material/EventAvailable";
import AssignmentIcon from "@mui/icons-material/Assignment";
import BarChartIcon from "@mui/icons-material/BarChart";
import InsightsIcon from "@mui/icons-material/Insights";
import NotificationsIcon from "@mui/icons-material/Notifications";
import MenuIcon from "@mui/icons-material/Menu";
import LogoutIcon from "@mui/icons-material/Logout";
import CloseIcon from "@mui/icons-material/Close";
import LinkedInIcon from "@mui/icons-material/LinkedIn";

```

```

import LanguageIcon from "@mui/icons-material/Language";
import PersonIcon from "@mui/icons-material/Person";

/* EXTRA ICONS */
import FolderIcon from "@mui/icons-material/Folder";
import EventIcon from "@mui/icons-material/Event";
import UploadFileIcon from "@mui/icons-material/UploadFile";
import CampaignIcon from "@mui/icons-material/Campaign";
import WorkIcon from "@mui/icons-material/Work";

export default function TeacherDashboard() {
  const [sidebarOpen, setSidebarOpen] = useState(true);
  const [activePage, setActivePage] = useState("overview");
  const [showProfile, setShowProfile] = useState(true);
  const [showLogout, setShowLogout] = useState(false);
  const [showUploadResource, setShowUploadResource] = useState(false);
  const [showGiveAlert, setShowGiveAlert] = useState(false);
  const [showFullProfile, setShowFullProfile] = useState(false);
  const [profile, setProfile] = useState(null);
  const [alerts, setAlerts] = useState([]);
  const unreadCount = alerts.filter(a => !a.is_read).length;

  const [loading, setLoading] = useState(true);
  const navigate = useNavigate();
  const location = useLocation();

  const availableTeacherPages = [
    "overview",
    "timetable",
    "attendance",
    "assignments",
    "marks",
    "resources",
    "events",
    "insights",
    "alerts",
    "placement",
    "placement-coordinator",
  ];

  const resolvePageFromPath = (pathname) => {
    const parts = pathname.split("/").filter(Boolean);
    const page = parts[1] || "overview";
    return availableTeacherPages.includes(page) ? page : "overview";
  };

  const isValidPage = availableTeacherPages.includes(resolvePageFromPath(location.pathname));

  const goToPage = (page) => {
    const targetPath = page === "overview" ? "/teacher" : `/teacher/${page}`;
    setActivePage(page);
  };

```

```

    navigate(targetPath);
  };

  useEffect(() => {
    setActivePage(resolvePageFromPath(location.pathname));
  }, [location.pathname]);

  useEffect(() => {
    if (!isValidPage) {
      return;
    }

    const page = resolvePageFromPath(location.pathname);
    const path = page === "overview" ? "/teacher" : `/teacher/${page}`;
    const labelMap = {
      attendance: "Take Attendance",
      marks: "Upload Marks",
      overview: "View Classes",
    };

    recordRouteVisit({ label: labelMap[page] || "View Classes", path, role: "teacher" });
  }, [isValidPage, location.pathname]);

  useEffect(() => {
    const fetchProfile = async () => {
      try {
        const res = await api.get("/faculty/profile");
        setProfile(res.data);
      } catch (err) {
        console.error("Failed to fetch profile", err);
      }
    };

    fetchProfile();
  }, []);

  useEffect(() => {
    const fetchAlerts = async () => {
      try {
        const res = await api.get("/faculty/alerts");
        setAlerts(res.data);
      } catch (error) {
        console.error("Error fetching alerts", error);
      } finally {
        setLoading(false);
      }
    };

    fetchAlerts();
  }, []);

```

```

if (showLogout) {
  return <Logout onBack={() => setShowLogout(false)} role="teacher" />;
}

if (!isValidPage) {
  return <Smart404 />;
}

if (!profile) {
  return (
    <div className="h-screen flex items-center justify-center">
      <p className="text-gray-500">Loading dashboard...</p>
    </div>
  );
}

return (
  <div className="h-screen w-screen bg-gradient-to-br from-indigo-50 via-white to-cyan-50">
    <div className="flex h-full">

      { /* ===== SIDEBAR ===== */ }
      <aside
        className={`transition-[width] duration-300 ease-out flex flex-col ${sidebarOpen ? "w-72" : "w-20"}
          } p-4 glass border-r border-white/40` >
        <div className="flex items-center justify-between mb-8">
          {sidebarOpen && (
            <h2 className="text-xl font-semibold text-indigo-600">
              GVP-MAAA
            </h2>
          )}
          <button
            onClick={() => setSidebarOpen(!sidebarOpen)}
            className="p-2 rounded-lg hover:bg-white/60"
          >
            <MenuIcon fontSize="small" />
          </button>
        </div>

        <div className="flex-1 overflow-y-auto">
          <SidebarSection title="Academics" open={sidebarOpen}>
            <MenuItem
              icon={DashboardIcon}
              label="Overview"
              open={sidebarOpen}
              active={activePage === "overview"}
              onClick={() => goToPage("overview")}
            />
            <MenuItem

```

```

        icon={DashboardIcon}
        label="Timetable"
        open={sidebarOpen}
        active={activePage === "timetable"}
        onClick={() => goToPage("timetable")}
      />
      <MenuItem
        icon={EventAvailableIcon}
        label="Attendance"
        open={sidebarOpen}
        active={activePage === "attendance"}
        onClick={() => goToPage("attendance")}
      />
      <MenuItem
        icon={AssignmentIcon}
        label="Assignments"
        open={sidebarOpen}
        active={activePage === "assignments"}
        onClick={() => goToPage("assignments")}
      />
      <MenuItem
        icon={BarChartIcon}
        label="Marks"
        open={sidebarOpen}
        active={activePage === "marks"}
        onClick={() => goToPage("marks")}
      />
    </SidebarSection>

    <SidebarSection title="Teaching Hub" open={sidebarOpen}>
      <MenuItem
        icon={FolderIcon}
        label="Resources"
        open={sidebarOpen}
        active={activePage === "resources"}
        onClick={() => goToPage("resources")}
      />
      <MenuItem
        icon={EventIcon}
        label="Events"
        open={sidebarOpen}
        active={activePage === "events"}
        onClick={() => goToPage("events")}
      />
      <MenuItem
        icon={InsightsIcon}
        label="Insights"
        open={sidebarOpen}
        active={activePage === "insights"}
        onClick={() => goToPage("insights")}
      />
    </SidebarSection>
  </div>
</div>

```

```

<MenuItem
  icon={WorkIcon}
  label="Placement"
  open={sidebarOpen}
  active={activePage === "placement"}
  onClick={() => goToPage("placement")}
/>
<MenuItem
  icon={WorkIcon}
  label="Placement Coordinator"
  open={sidebarOpen}
  active={activePage === "placement-coordinator"}
  onClick={() => goToPage("placement-coordinator")}
/>
</SidebarSection>

<div className="mt-6 mb-6">
  {sidebarOpen && (
    <p className="text-xs uppercase text-gray-400 tracking-wider mb-3">
      Quick Actions
    </p>
  )}

  <div className="space-y-3">
    <button
      onClick={() => setShowUploadResource(true)}
      className="w-full flex items-center gap-3 px-3 py-2.5 rounded-xl border border-indigo-200 text-indigo-700 bg-indigo-50 hover:bg-indigo-100 transition"
    >
      <UploadFileIcon fontSize="small" />
      {sidebarOpen && <span>Upload Resource</span>}
    </button>

    <button
      onClick={() => setShowGiveAlert(true)}
      className="w-full flex items-center gap-3 px-3 py-2.5 rounded-xl border border-amber-200 text-amber-700 bg-amber-50 hover:bg-amber-100 transition"
    >
      <CampaignIcon fontSize="small" />
      {sidebarOpen && <span>Give Alert</span>}
    </button>

  </div>
</div>

<SidebarSection title="System" open={sidebarOpen}>
  <div className="relative">
    <MenuItem
      icon={NotificationsIcon}
      label="Alerts"

```

```

        open={sidebarOpen}
        active={activePage === "alerts"}
        onClick={() => goToPage("alerts")}
      />
      {unreadCount > 0 && (
        <span className="absolute top-0 right-4 bg-red-500 text-white text-xs px-2 py-0.5 rounded-full">
          {unreadCount}
        </span>
      )}
    </div>

    <MenuItem
      icon={LogoutIcon}
      label="Logout"
      open={sidebarOpen}
      danger
      onClick={() => setShowLogout(true)}
    />
  </SidebarSection>
</div>
</aside>

{ /* ===== MAIN ===== */}
<main className="flex-1 p-8 flex gap-6 overflow-y-auto">

  { /* ===== MAIN CONTENT (FIXED SAFE PADDING) ===== */}
  <div
    className={ `flex-1 relative z-10 transition-all ${!showProfile ? "pr-16" : ""}
  ` }
  >
    {activePage === "overview" && <Overview profile={profile} />}
    {activePage === "timetable" && <Timetable />}
    {activePage === "attendance" && <Attendance />}
    {activePage === "assignments" && <Assignment />}
    {activePage === "marks" && <Marks />}
    {activePage === "resources" && <Resources />}
    {activePage === "events" && <Events />}
    {activePage === "insights" && <Insights />}
    {activePage === "placement" && <Placement />}
    {activePage === "placement-coordinator" && <PlacementCoordinator />}
    {activePage === "alerts" && (
      <Alerts
        alerts={alerts}
        setAlerts={setAlerts}
        loading={loading}
      />
    )}
  </div>

  { /* ===== PROFILE ===== */}
  <div

```

```

        className={`transition-all duration-300 ${showProfile ? "w-80" : "w-14"}
        } overflow-hidden`}
      >
      {showProfile ? (
        <TeacherProfile
          profile={profile}
          onClose={() => setShowProfile(false)}
          onViewFullProfile={() => setShowFullProfile(true)}
        />
      ) : (
        <CollapsedTeacherProfile onOpen={() => setShowProfile(true)} />
      )}
    </div>

  </main>
  {/* ===== FULL PROFILE PAGE ===== */}
  {showFullProfile && (
    <TeacherProfilePage
      profile={profile}
      onBack={() => setShowFullProfile(false)}
    />
  )}
  {/* ===== UPLOAD RESOURCE MODAL ===== */}
  {showUploadResource && (
    <UploadResourceModal
      onClose={() => setShowUploadResource(false)}
    />
  )}
  {/* ===== GIVE ALERT MODAL ===== */}
  {showGiveAlert && (
    <GiveAlertModal
      onClose={() => setShowGiveAlert(false)}
    />
  )}
</div>
</div>
);
}

/* ===== PROFILE ===== */

function TeacherProfile({ profile, onClose, onViewFullProfile }) {
  return (
    <div className="h-full rounded-2xl overflow-hidden pointer-events-auto">
      <div className="h-full glass p-6 flex flex-col justify-between">
        <button
          onClick={onClose}
          className="self-end p-2 rounded-full hover:bg-white/60 transition"
        >
          <CloseIcon fontSize="small" />

```

```

</button>

<div className="text-center">
  <div className="w-24 h-24 mx-auto rounded-full bg-indigo-500 text-white flex items-center justify-
    center text-3xl font-semibold shadow-inner border-4 border-white/20">
    {profile?.name ? profile.name.charAt(0).toUpperCase() : "T"}
  </div>
  <h3 className="mt-4 text-lg font-semibold">
    {profile?.name || "Faculty"}
  </h3>
  <p className="text-sm text-gray-500">
    {profile?.designation || "Faculty"}
  </p>

  <div className="mt-6 space-y-3 text-sm text-gray-600 text-left">
    <ProfileRow label="Email" value={profile?.email || ""} />
    <ProfileRow label="Phone" value={profile?.phone || ""} />
    <ProfileRow label="Expertise" value={profile?.expertise?.join(", ") || ""} />
  </div>

  <div className="mt-6 flex justify-center gap-4">
    {profile?.linkedin && (
      <IconButton
        icon={LinkedInIcon}
        color="bg-blue-600"
        onClick={() => window.open(profile.linkedin, "_blank")}
      />
    )}

    {profile?.github && (
      <IconButton
        icon={GitHubIcon}
        color="bg-gray-800"
        onClick={() => window.open(profile.github, "_blank")}
      />
    )}

    {profile?.portfolio && (
      <IconButton
        icon={LanguageIcon}
        color="bg-indigo-600"
        onClick={() => window.open(profile.portfolio, "_blank")}
      />
    )}

  </div>
</div>

<div className="space-y-3">
  <button
    onClick={onViewFullProfile}

```

```

        className="w-full py-2 rounded-xl bg-indigo-600 text-white hover:bg-indigo-700 transition"
    >
        View Full Profile
    </button>

    <button className="w-full py-2 rounded-xl border border-gray-300 text-gray-600">
        View Resume
    </button>
</div>
</div>
</div>
);
}

/* ===== HELPERS ===== */

function SidebarSection({ title, open, children }) {
    return (
        <div className="mb-6">
            {open && <p className="text-xs uppercase text-gray-400 mb-2">{title}</p>}
            <div className="space-y-2">{children}</div>
        </div>
    );
}

function MenuItem({ icon: Icon, label, open, active, onClick, danger }) {
    return (
        <div
            onClick={onClick}
            className={flex items-center gap-3 px-3 py-2 rounded-xl cursor-pointer transition
                ${active ? "bg-indigo-500/10 text-indigo-700" : "hover:bg-white/60"}
                ${danger && "text-red-600 hover:bg-red-50"}
                ${!open && "justify-center"}}`
        >
            <Icon fontSize="small" />
            {open && <span>{label}</span>}
        </div>
    );
}

function ProfileRow({ label, value }) {
    return (
        <div className="flex justify-between">
            <span className="text-gray-400">{label}</span>
            <span className="font-medium">{value}</span>
        </div>
    );
}

function IconButton({ icon: Icon, color }) {
    return (

```

```

    <button
      className={`w-10 h-10 rounded-full ${color} text-white flex items-center justify-center`}
    >
      <Icon fontSize="small" />
    </button>
  );
}

function CollapsedTeacherProfile({ onOpen }) {
  return (
    <div className="h-full glass flex items-center justify-center pointer-events-auto">
      <button
        onClick={onOpen}
        className="w-10 h-10 rounded-full bg-indigo-600 text-white flex items-center justify-center shadow"
      >
        <PersonIcon fontSize="small" />
      </button>
    </div>
  );
}

```

8.9 Student Dashboard.jsx

```

import { useState } from "react";
import { useEffect } from "react";
import { useNavigate, useLocation } from "react-router-dom";
import api from "../utils/axios";
import { recordRouteVisit } from "../utils/navigationHistory";

/* ===== STUDENT PAGES ===== */
import StudentOverview from "../pages/Student/Overview";
import Attendance from "../pages/Student/Attendance";
import Marks from "../pages/Student/Marks";
import Timetable from "../pages/Student/Timetable";
import Assignments from "../pages/Student/Assignments";
import Events from "../pages/Student/Events";
import Resources from "../pages/Student/Resources";
import Placement from "../pages/Student/Placement";
import Insights from "../pages/Student/Insights";
import Alerts from "../pages/Student/Alerts";
import ViewProfile from "../pages/Student/ViewProfile";
import Logout from "../pages/Logout";
import Smart404 from "../components/Smart404";

/* ===== Material UI Icons ===== */
import DashboardIcon from "@mui/icons-material/Dashboard";
import EventAvailableIcon from "@mui/icons-material/EventAvailable";
import BarChartIcon from "@mui/icons-material/BarChart";
import AssignmentIcon from "@mui/icons-material/Assignment";

```

```

import ScheduleIcon from "@mui/icons-material/Schedule";
import WorkIcon from "@mui/icons-material/Work";
import EventIcon from "@mui/icons-material/Event";
import InsightsIcon from "@mui/icons-material/Insights";
import MenuBookIcon from "@mui/icons-material/MenuBook";
import NotificationsIcon from "@mui/icons-material/Notifications";
import LinkedInIcon from "@mui/icons-material/LinkedIn";
import GitHubIcon from "@mui/icons-material/GitHub";

import CloseIcon from "@mui/icons-material/Close";
import PersonIcon from "@mui/icons-material/Person";
import MenuIcon from "@mui/icons-material/Menu";
import LogoutIcon from "@mui/icons-material/Logout";

export default function StudentDashboard() {
  const [sidebarOpen, setSidebarOpen] = useState(true);
  const [activePage, setActivePage] = useState("overview");
  const [showProfile, setShowProfile] = useState(true);
  const [showFullProfile, setShowFullProfile] = useState(false);
  const [showLogout, setShowLogout] = useState(false);
  const [profile, setProfile] = useState(null);
  const [alerts, setAlerts] = useState([]);
  const [loadingAlerts, setLoadingAlerts] = useState(true);

  const unreadCount = alerts.filter(a => !a.is_read).length;

  const navigate = useNavigate();
  const location = useLocation();

  const availableStudentPages = [
    "overview",
    "attendance",
    "marks",
    "assignments",
    "timetable",
    "placement",
    "events",
    "insights",
    "resources",
    "alerts",
  ];

  const resolvePageFromPath = (pathname) => {
    const parts = pathname.split("/").filter(Boolean);
    const page = parts[1] || "overview";
    return availableStudentPages.includes(page) ? page : "overview";
  };

  const isValidPage = availableStudentPages.includes(resolvePageFromPath(location.pathname));

  const goToPage = (page) => {

```

```

const targetPath = page === "overview" ? "/student" : `/student/${page}`;
navigate(targetPath);
};

useEffect(() => {
  setActivePage(resolvePageFromPath(location.pathname));
}, [location.pathname]);

useEffect(() => {
  if (!isValidPage) {
    return;
  }

  const page = resolvePageFromPath(location.pathname);
  const path = page === "overview" ? "/student" : `/student/${page}`;
  const labelMap = {
    attendance: "View Attendance",
    marks: "Check Results",
    overview: "Dashboard",
  };

  recordRouteVisit({ label: labelMap[page] || "Dashboard", path, role: "student" });
}, [isValidPage, location.pathname]);

// Fetch student profile on mount
const fetchProfile = async () => {
  try {
    const res = await api.get("/student/profile");
    setProfile(res.data);
  } catch (err) {
    console.error(err);
  }
};

useEffect(() => {
  fetchProfile();
}, []);

useEffect(() => {
  const fetchAlerts = async () => {
    try {
      const res = await api.get("/student/alerts");
      setAlerts(res.data);
    } catch (error) {
      console.error("Error fetching alerts", error);
    } finally {
      setLoadingAlerts(false);
    }
  };

  fetchAlerts();

```

```
}, []);
```

```
// Prevent back navigation
```

```
useEffect(() => {  
  window.history.pushState(null, "", window.location.href);  
  
  const handleBack = () => {  
    window.history.pushState(null, "", window.location.href);  
  };  
  
  window.addEventListener("popstate", handleBack);  
  
  return () => window.removeEventListener("popstate", handleBack);  
}, []);
```

```
if (showFullProfile) {  
  return (  
    <ViewProfile  
      profile={profile}  
      onClose={() => {  
        setShowFullProfile(false);  
        fetchProfile(); // 🔥 refresh sidebar profile  
      }}  
    />  
  );  
}
```

```
if (showLogout) {  
  return <Logout onBack={() => setShowLogout(false)} />;  
}
```

```
if (!isValidPage) {  
  return <Smart404 />;  
}
```

```
if (!profile) {  
  return (  
    <div className="h-screen flex items-center justify-center">  
      <p className="text-slate-500">Loading profile...</p>  
    </div>  
  );  
}
```

```
return (  
  <div className="h-screen w-screen bg-gradient-to-br from-indigo-50 via-white to-cyan-50">  
    <div className="flex h-full">
```

```

{ /* ===== SIDEBAR ===== */}
<aside
  className={`transition-all duration-300 flex flex-col ${sidebarOpen ? "w-72" : "w-20"}
    } p-4 glass border-r border-white/40`}
>
  <div className="flex items-center justify-between mb-8">
    {sidebarOpen && (
      <h2 className="text-xl font-semibold text-indigo-600">
        GVP-MAAA
      </h2>
    )}
    <button
      onClick={() => setSidebarOpen(!sidebarOpen)}
      className="p-2 rounded-lg hover:bg-white/60"
    >
      <MenuIcon fontSize="small" />
    </button>
  </div>

  <div className="flex-1 overflow-y-auto">
    <SidebarSection title="Academics" open={sidebarOpen}>
      <MenuItem icon={DashboardIcon} label="Overview" open={sidebarOpen}
        active={activePage === "overview"} onClick={() => goToPage("overview")} />
      <MenuItem icon={EventAvailableIcon} label="Attendance" open={sidebarOpen}
        active={activePage === "attendance"} onClick={() => goToPage("attendance")} />
      <MenuItem icon={BarChartIcon} label="Marks" open={sidebarOpen}
        active={activePage === "marks"} onClick={() => goToPage("marks")} />
      <MenuItem icon={AssignmentIcon} label="Assignments" open={sidebarOpen}
        active={activePage === "assignments"} onClick={() => goToPage("assignments")} />
      <MenuItem icon={ScheduleIcon} label="Timetable" open={sidebarOpen}
        active={activePage === "timetable"} onClick={() => goToPage("timetable")} />
    </SidebarSection>

    <SidebarSection title="Career & Growth" open={sidebarOpen}>
      <MenuItem icon={WorkIcon} label="Placement" open={sidebarOpen}
        active={activePage === "placement"} onClick={() => goToPage("placement")} />
      <MenuItem icon={EventIcon} label="Events" open={sidebarOpen}
        active={activePage === "events"} onClick={() => goToPage("events")} />
      <MenuItem icon={InsightsIcon} label="Insights" open={sidebarOpen}
        active={activePage === "insights"} onClick={() => goToPage("insights")} />
      <MenuItem icon={MenuBookIcon} label="Resources" open={sidebarOpen}
        active={activePage === "resources"} onClick={() => goToPage("resources")} />
    </SidebarSection>

    <SidebarSection title="System" open={sidebarOpen}>
      <div className="relative">
        <MenuItem
          icon={NotificationsIcon}
          label="Alerts"
          open={sidebarOpen}

```

```

        active={activePage === "alerts"}
        onClick={() => goToPage("alerts")}
      />
      {unreadCount > 0 && (
        <span className="absolute top-0 right-4 bg-red-500 text-white text-xs px-2 py-0.5 rounded-full">
          {unreadCount}
        </span>
      )}
    </div>
  </SidebarSection>
</div>

<div className="pt-4 border-t border-white/40">
  <button
    onClick={() => setShowLogout(true)}
    className="w-full flex items-center gap-3 px-3 py-2 rounded-xl text-red-600 hover:bg-red-50"
  >
    <LogoutIcon fontSize="small" />
    {sidebarOpen && <span>Logout</span>}
  </button>
</div>
</aside>

{ /* ===== MAIN ===== */}
<main className="flex-1 p-8 flex gap-6 overflow-hidden">

  <div className="flex-1 overflow-y-auto pr-2">
    {activePage === "overview" && <StudentOverview profile={profile} />}
    {activePage === "attendance" && <Attendance />}
    {activePage === "marks" && <Marks />}
    {activePage === "assignments" && <Assignments />}
    {activePage === "timetable" && <Timetable />}
    {activePage === "events" && <Events />}
    {activePage === "resources" && <Resources />}
    {activePage === "placement" && <Placement />}
    {activePage === "insights" && <Insights />}
    {activePage === "alerts" && (<Alerts alerts={alerts} setAlerts={setAlerts} loading={loadingAlerts} />)}
  </div>

  <div
    className={`transition-all duration-300 ${showProfile ? "w-80" : "w-14"}
      overflow-hidden`}
  >
    {showProfile ? (
      <StudentProfile
        profile={profile}
        onClose={() => setShowProfile(false)}
        onViewProfile={() => {
          setShowFullProfile(true);
          setShowProfile(false);

```

```

    }}
  />

  ): (
    <CollapsedProfile onOpen={() => setShowProfile(true)} />
  )}
</div>
</main>
</div>
</div>
);
}

/* ===== SMALL HELPERS ===== */

function SidebarSection({ title, open, children }) {
  return (
    <div className="mb-6">
      {open && (
        <p className="text-xs uppercase text-gray-400 mb-2 tracking-wider">
          {title}
        </p>
      )}
      <div className="space-y-2">{children}</div>
    </div>
  );
}

function MenuItem({ icon: Icon, label, open, active, onClick }) {
  return (
    <div
      onClick={onClick}
      className={`flex items-center gap-3 px-3 py-2 rounded-xl cursor-pointer transition
        ${active ? "bg-indigo-500/10 text-indigo-700" : "hover:bg-white/60"}
        ${!open && "justify-center"}`>
      >
      <Icon fontSize="small" />
      {open && <span>{label}</span>}
    </div>
  );
}

function StudentProfile({ profile, onClose, onViewProfile }) {
  return (
    <div className="h-full glass rounded-2xl p-6 flex flex-col justify-between">
      <button onClick={onClose} className="self-end p-2 hover:bg-white/60 rounded-full">
        <CloseIcon fontSize="small" />
      </button>

      <div className="text-center">
        <div className="w-24 h-24 mx-auto rounded-full bg-indigo-500 text-white flex items-center justify-

```

```

                center text-3xl font-semibold">
                {profile.name?.charAt(0).toUpperCase()}
            </div>
            <h3 className="mt-4 text-lg font-semibold">
                {profile.name}
            </h3>
            <p className="text-sm text-gray-500">
                {profile.roll_no}
            </p>

            <div className="mt-6 space-y-3 text-sm text-gray-600 text-left">
                <ProfileRow label="Roll No" value={profile.roll_no} />
                <ProfileRow label="Year" value={profile.year} />
                <ProfileRow label="Semester" value={profile.semester} />
                <ProfileRow
                    label="Certificates"
                    value={profile.certificates ? profile.certificates.length : 0}
                />
            </div>

            <div className="mt-6 flex justify-center gap-4">
                <IconLink
                    icon={LinkedInIcon}
                    color="bg-blue-600"
                    link={profile.linkedin}
                />

                <IconLink
                    icon={GitHubIcon}
                    color="bg-gray-800"
                    link={profile.github}
                />

                <IconLink
                    icon={PersonIcon}
                    color="bg-indigo-600"
                    link={profile.portfolio}
                />

            </div>
        </div>

        <button
            onClick={onViewProfile}
            className="w-full py-2 rounded-xl bg-indigo-600 text-white hover:bg-indigo-700"
        >
            View Full Profile
        </button>
    </div>
);

```

```

}

function CollapsedProfile({ onOpen }) {
  return (
    <div className="h-full glass flex items-center justify-center">
      <button
        onClick={onOpen}
        className="w-10 h-10 rounded-full bg-indigo-500 text-white flex items-center justify-center"
      >
        <PersonIcon fontSize="small" />
      </button>
    </div>
  );
}

function ProfileRow({ label, value }) {
  return (
    <div className="flex justify-between">
      <span className="text-gray-400">{label}</span>
      <span className="font-medium">{value}</span>
    </div>
  );
}

function IconButton({ icon: Icon, color, url }) {
  if (!url) return null;

  return (
    <a
      href={url}
      target="_blank"
      rel="noopener noreferrer"
      className={`w-10 h-10 flex items-center justify-center rounded-full ${color} text-white shadow
        hover:scale-110 transition`}
    >
      <Icon fontSize="small" />
    </a>
  );
}

function IconLink({ icon: Icon, color, link }) {
  if (!link) {
    // No link → disabled icon
    return (
      <div
        className={`w-10 h-10 flex items-center justify-center rounded-full
          bg-gray-300 text-white cursor-not-allowed`}
        title="Link not added"
      >
        <Icon fontSize="small" />
      </div>
    );
  }

```

```

    );
  }

  return (
    <a
      href={link}
      target="_blank"
      rel="noreferrer"
      className={`w-10 h-10 flex items-center justify-center rounded-full
        ${color} text-white shadow hover:scale-110 transition`}
      title={link}
    >
      <Icon fontSize="small" />
    </a>
  );
}

```

9.CONCLUSION

9. CONCLUSION

This paper presented GVP-MAAA, a Multi-Agent Academic Assistant that transforms academic management into an intelligent, automated, and data-driven process. The system's five-agent architecture — Notification, Analytics, Attendance, Placement, and Resource — each with rolespecific context prompts and keyword-based routing, delivers contextually appropriate responses across Student, Faculty, and Administrator roles. The MAAAAgent React component (MAAA_Agent_3.jsx) serves as the core implementation artifact, demonstrating a fully functional multi-agent chatbot with role selection, agent detection, LLM integration, and a polished gamified UI. Agent routing accuracy of 94.5% across 200 test queries validates the effectiveness of the keyword-classification approach. User acceptance testing confirmed high satisfaction with dashboard usability, chatbot relevance, and notification timeliness. Planned future enhancements include:

- **Semantic Embedding-Based Agent Routing:** Replace keyword matching in detectAgent() with vector similarity routing (using OpenAI Embeddings or a local sentence-transformer model) to handle ambiguous and cross-domain queries.
- **Machine Learning Dropout Risk Prediction:** Replace rulebased risk detection with supervised ML models trained on historical institutional performance data.
- **Biometric Attendance Integration:** Incorporate IoT-based biometric or RFID hardware for fully automated attendance capture feeding directly to the Attendance Agent.
- **Mobile Application:** Cross-platform React Native app with offline sync support and mobile-optimized agent chat UI.
- **Voice-Based Chatbot Agent:** Add speech-to-text input and text-to-speech output to the chatbot interface for hands-free academic assistance.

10. REFERENCES

- [1] N. R. Jennings, K. Sycara, and M. Wooldridge, "A Roadmap of Agent Research and Development," *Autonomous Agents and Multi-Agent Systems*, vol. 1, no. 1, pp. 7-38, 1998.
- [2] G. Siemens and R. S. J. D. Baker, "Learning Analytics and Educational Data Mining: Towards Communication and Collaboration," in *Proc. 2nd Int. Conf. Learning Analytics and Knowledge*, Vancouver, 2012, pp. 252-254.
- [3] A. Al-Dokhny, A. Al-Sayed, and O. Drwish, "Adaptive MultiAgent Smart Academic Advising System Based on Ontology and Case-Based Reasoning," *International Journal of Computer Applications*, vol. 131, no. 14, pp. 1-8, 2015.
- [4] P. Smutny and P. Schreiberova, "Chatbots for Learning: A Review of Educational Chatbots for the Facebook Messenger," *Computers and Education*, vol. 151, 2020.
- [5] T. Alario-Hoyos, M. Perez-Sanagustin, and C. Kloos, "Analyzing the Deployment of Massive Open Online Courses and the Role of LMS," *IEEE Trans. Learning Technologies*, vol. 7, no. 4, pp. 380-391, 2014.
- [6] M. Wooldridge and N. R. Jennings, "Intelligent Agents: Theory and Practice," *The Knowledge Engineering Review*, vol. 10, no. 2, pp. 115-152, 1995.
- [7] D. Gasevic, S. Dawson, and G. Siemens, "Let's Not Forget: Learning Analytics Are About Learning," *TechTrends*, vol. 59, no. 1, pp. 64-71, 2015.
- [8] Anthropic, "Claude API Documentation," Available: <https://docs.anthropic.com>, 2024.
- [9] LangChain, "LangChain Documentation: Building Applications with LLMs," Available: <https://docs.langchain.com>, 2024.
- [10] FastAPI, "FastAPI Modern Web Framework for Building APIs with Python," Available: <https://fastapi.tiangolo.com>, 2024.
- [11] R. Luckin, W. Holmes, M. Griffiths, and L. Forcier, "Intelligence Unleashed: An Argument for AI in Education," Pearson Education, London, 2016.
- [12] M. A. Al-Emran, H. M. Malik, and M. N. Al-Kabi, "A Survey of Internet of Things (IoT) in Education: Opportunities and Challenges," *Advances in IoT-Based Technologies for Smart Educational Environments*, 2018.



Moogambigai Charitable And Educational Trust
RAJARAJESWARI COLLEGE OF ENGINEERING

(An Autonomous Institution, Under Visvesvaraya Technological University, Belagavi)
No. 14, Ramohalli Cross, Kumbalagodu, Mysore Road, Bengaluru - 560074



**International Conference on Recent Research in
Computational Sciences & Engineering (ICRRCE -2025)**

CERTIFICATE OF PARTICIPATION

This is to certify that Mr./Ms./Dr./Prof. Ch. Bhonu Kumar
From Gyayatri Vidya Parishad UG, Vilakapadnam has presented the paper entitled
AI powered personalized Learning platform for Dyslexia in the International Conference on
Recent Research in Computational Sciences and Engineering (ICRRCE -2025) held on 12th April 2025 organized by
Rajarajeswari College of Engineering, Bangalore-560074.



C. Rangaiah

Dr. L. Rangaiah
Convener

Dr. R Balakrishna

Dr. R Balakrishna
Conference Chair



Moogambigai Charitable And Educational Trust



RAJARAJESWARI COLLEGE OF ENGINEERING

(An Autonomous Institution, Under Visvesvaraya Technological University, Belagavi)

No. 14, Ramohalli Cross, Kumbalagodu, Mysore Road, Bengaluru - 560074



International Conference on Recent Research in
Computational Sciences & Engineering (ICRRCE -2025)



CERTIFICATE OF PARTICIPATION

This is to certify that Mr./Ms./Dr./Prof. *M. S. H. Kalahangani*
From *Jayashree Vidya Parishad* presented the paper entitled
AI-Powered Personalized Learning Platform for Recent Research in Computational Sciences and Engineering in the International Conference on
Recent Research in Computational Sciences and Engineering (ICRRCE -2025) held on 12th April 2025 organized by
Rajarajeswari College of Engineering, Bangalore-560074.

C. Rangaiah

Dr. L. Rangaiah
Convener



[Signature]
Dr. R Balakrishna
Conference Chair






Moogambigai Charitable And Educational Trust

RAJARAJESWARI COLLEGE OF ENGINEERING

(An Autonomous Institution, Under Visvesvaraya Technological University, Belagavi)
No. 14, Ramohalli Cross, Kumbalagodu, Mysore Road, Bengaluru - 560074





International Conference on Recent Research in
Computational Sciences & Engineering (ICRRCE -2025)

CERTIFICATE OF PARTICIPATION

This is to certify that Mr./Ms./Dr./Prof. B. Bhavani.....

From Gayatri Vidya Parishad of Vishwakopadnam has presented the paper entitled
AI based Personalized Learning platform for..... In the International Conference on
Recent Research in Computational Sciences and Engineering (ICRRCE -2025) held on 12th April 2025 organized by
Rajarajeswari College of Engineering, Bangalore-560074.



Dr. L. Rangaiah
Convener



Dr. R. Balakrishna
Conference Chair

Moogambigai Charitable And Educational Trust
RAJARAJESWARI COLLEGE OF ENGINEERING
(An Autonomous Institution, Under Visvesvaraya Technological University, Belagavi)
No. 14, Ramohalli Cross, Kumbalagodu, Mysore Road, Bengaluru - 560074



*International Conference on Recent Research in
Computational Sciences & Engineering (ICRRCE -2025)*

CERTIFICATE OF PARTICIPATION

This is to certify that Mr./Ms./Dr./Prof. V. Siva Ram
From Jayathi Vidya Parichalad Uq Vishakupadanam has presented the paper entitled
AI powered Personalized Learning platform for Dyslexia In the International Conference on
Recent Research in Computational Sciences and Engineering (ICRRCE -2025) held on 12th April 2025 organized by
Rajarajeswari College of Engineering, Bangalore-560074.


Dr. L. Rangaiah
Convener




Dr. R Balakrishna
Conference Chair



Moogambigai Charitable And Educational Trust



RAJARAJESWARI COLLEGE OF ENGINEERING

(An Autonomous Institution, Under Visvesvaraya Technological University, Belagavi)

No. 14, Ramohalli Cross, Kumbalagodu, Mysore Road, Bengaluru - 560074



**International Conference on Recent Research in
Computational Sciences & Engineering (ICRRCE -2025)**



CERTIFICATE OF PARTICIPATION

This is to certify that Mr./Ms./Dr./Prof.**M. Pallavi**.....
From **Gayatri Vidya Peetham (G.V.P.) Lakshapattanam** has presented the paper entitled
AI driven Personalized Learning platform for Dyslexia in the International Conference on
Recent Research in Computational Sciences and Engineering (ICRRCE -2025) held on 12th April 2025 organized by
Rajarajeswari College of Engineering, Bangalore-560074.



C. Rangaiah

Dr. L. Rangaiah
Convener

Dr. R Balakrishna

Dr. R Balakrishna
Conference Chair

